io/kubernetes/client/simplified/Deployments.java

```
/*
Copyright 2023 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.simplified;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.models.V1Deployment;
import io.kubernetes.client.openapi.models.V1DeploymentList;
import io.kubernetes.client.openapi.models.V1Status;
import io.kubernetes.client.custom.V1Patch;
import io.kubernetes.client.openapi.apis.AppsV1Api;
/*
* This class is used to overload the AppsV1Api class methods for Deployments.
*/
public class Deployments {
private AppsV1Api api;
Deployments(AppsV1Api api){
this.api = api;
}
public V1Deployment createNamespacedDeployment(String namespace, V1Deployment body) throws
ApiException {
return api.createNamespacedDeployment(namespace, body).execute();
}
public V1Status deleteNamespacedDeployment(String name, String namespace) throws ApiException
{
return api.deleteNamespacedDeployment(name, namespace).execute();
}
public V1DeploymentList listNamespacedDeployment(String namespace) throws ApiException {
return api.listNamespacedDeployment(namespace).execute();
}
public V1DeploymentList listDeploymentForAllNamespaces() throws ApiException {
return api.listDeploymentForAllNamespaces().execute();
}
public V1Deployment replaceNamespacedDeployment(String name, String namespace,
V1Deployment body) throws ApiException {
return api.replaceNamespacedDeployment(name, namespace, body).execute();
}
public V1Deployment patchNamespacedDeployment(String name, String namespace, V1Patch body)
throws ApiException {
return api.patchNamespacedDeployment(name, namespace, body).execute();
}
```

}

io/kubernetes/client/simplified/Pods.java

```
/*
Copyright 2023 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.simplified;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.models.V1Pod;
import io.kubernetes.client.openapi.models.V1PodList;
import io.kubernetes.client.openapi.models.V1PodTemplate;
import io.kubernetes.client.openapi.apis.CoreV1Api;
/*
* This class is used to overload the CoreV1Api class methods for Pods.
*/
public class Pods{
private CoreV1Api api;
Pods(CoreV1Api api){
this.api = api;
}
public V1Pod createNamespacedPod(String namespace, V1Pod body) throws ApiException {
return api.createNamespacedPod(namespace, body).execute();
}
public V1Pod deleteNamespacedPod(String name, String namespace) throws ApiException {
return api.deleteNamespacedPod(name, namespace).execute();
}
public V1PodList listNamespacedPod(String namespace) throws ApiException {
return api.listNamespacedPod(namespace).execute();
}
public V1PodTemplate createNamespacedPodTemplate(String namespace, V1PodTemplate body)
throws ApiException {
return api.createNamespacedPodTemplate(namespace, body).execute();
}
}
```

io/kubernetes/client/simplified/Namespaces.java

```
/*
Copyright 2023 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
```

you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.simplified;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.models.V1Namespace;
import io.kubernetes.client.openapi.models.V1Status;
import io.kubernetes.client.openapi.apis.CoreV1Api;
/*
* This class is used to overload the CoreV1Api class methods for Namespaces.
*/
public class Namespaces {
private CoreV1Api api;
Namespaces(CoreV1Api api){
this.api = api;
}
public V1Namespace createNamespace(V1Namespace body) throws ApiException {
return api.createNamespace(body).execute();
}
public V1Status deleteNamespace(String name) throws ApiException {
return api.deleteNamespace(name).execute();
}
}

io/kubernetes/client/ProtoClient.java

/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client;
import com.google.protobuf.CodedOutputStream;
import com.google.protobuf.Message;
import com.google.protobuf.Message.Builder;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.Configuration;

```
import io.kubernetes.client.openapi.Pair;
import io.kubernetes.client.proto.Meta.DeleteOptions;
import io.kubernetes.client.proto.Meta.Status;
import io.kubernetes.client.proto.Runtime.TypeMeta;
import io.kubernetes.client.proto.Runtime.Unknown;
import java.io.IOException;
import java.util.ArrayList;
import java.util.HashMap;
import okhttp3.MediaType;
import okhttp3.Request;
import okhttp3.RequestBody;
import okhttp3.Response;
import okio.BufferedSource;
import okio.ByteString;
public class ProtoClient {
/**
* ObjectOrStatus is an object that is the return from a method call it holds either an API Object
* or an API Status object, but not both. Only one field may be non-null at a time.
*
* <p>Oh, how I long for multi-return...
*/
public static class ObjectOrStatus<T extends Message> {
public ObjectOrStatus(T obj, Status status) {
this.object = obj;
this.status = status;
}
public T object;
public Status status;
public String toString() {
if (object != null) {
return object.toString();
}
return status.toString();
}
}
private ApiClient apiClient;
// Magic number for the beginning of proto encoded.
// https://github.com/kubernetes/apimachinery/blob/release-1.13/pkg/runtime/serializer/protobuf/protobu
f.go#L44
private static final byte[] MAGIC = new byte[] {0x6b, 0x38, 0x73, 0x00};
private static final ByteString MAGIC_BYTESTRING = ByteString.of(MAGIC);
private static final String MEDIA_TYPE = "application/vnd.kubernetes.protobuf";
/** Simple Protocol Buffers API client constructor, uses default configuration */
public ProtoClient() {
this(Configuration.getDefaultApiClient());
}
/**
* ProtocolBuffer Client Constructor
*
* @param apiClient The api client to use.
*/
public ProtoClient(ApiClient apiClient) {
this.apiClient = apiClient;
```

```
}
/**
* Get the API client for these ProtocolBuffer operations.
*
* @return The API client that will be used.
*/
public ApiClient getApiClient() {
return apiClient;
}
/**
* Set the API client for subsequent ProtocolBuffer operations.
*
* @param apiClient The new API client to use.
*/
public void setApiClient(ApiClient apiClient) {
this.apiClient = apiClient;
}
/**
* Get a Kubernetes API object using protocol buffer encoding.
*
* @param builder The appropriate Builder for the object received from the request.
* @param path The URL path to call (e.g. /api/v1/namespaces/default/pods/pod-name)
* @return An ObjectOrStatus which contains the Object requested, or a Status about the request.
*/
public <T extends Message> ObjectOrStatus<T> get(T.Builder builder, String path)
throws ApiException, IOException {
return request(builder, path, "GET", null, null, null);
}
/**
* List is fluent, semantic sugar method on top of get, which is intended to convey that the
* object is a List of objects rather than a single object
*
* @param builder The appropriate Builder for the object received from the request.
* @param path The URL path to call (e.g. /api/v1/namespaces/default/pods/pod-name)
* @return An ObjectOrStatus which contains the Object requested, or a Status about the request.
*/
public <T extends Message> ObjectOrStatus<T> list(T.Builder builder, String path)
throws ApiException, IOException {
return get(builder, path);
}
/**
* Create a Kubernetes API object using protocol buffer encoding. Performs a POST
*
* @param obj The object to create
* @param path The URL path to call
* @param apiVersion The api version to use
* @param kind The kind of the object
* @return An ObjectOrStatus which contains the Object requested, or a Status about the request.
*/
public <T extends Message> ObjectOrStatus<T> create(
T obj, String path, String apiVersion, String kind) throws ApiException, IOException {
return request(obj.newBuilderForType(), path, "POST", obj, apiVersion, kind);
}
```

```
/**
* Update a Kubernetes API object using protocol buffer encoding. Performs a PUT
*
* @param obj The object to create
* @param path The URL path to call
* @param apiVersion The api version to use
* @param kind The kind of the object
* @return An ObjectOrStatus which contains the Object requested, or a Status about the request.
*/
public <T extends Message> ObjectOrStatus<T> update(
T obj, String path, String apiVersion, String kind) throws ApiException, IOException {
return request(obj.newBuilderForType(), path, "PUT", obj, apiVersion, kind);
}
/**
* Merge a Kubernetes API object using protocol buffer encoding. Performs a PATCH
*
* @param obj The object to merge
* @param path The URL path to call
* @param apiVersion The api version to use
* @param kind The kind of the object
* @return An ObjectOrStatus which contains the Object requested, or a Status about the request.
*/
public <T extends Message> ObjectOrStatus<T> merge(
T obj, String path, String apiVersion, String kind) throws ApiException, IOException {
return request(obj.newBuilderForType(), path, "PATCH", obj, apiVersion, kind);
}
/**
* Delete a kubernetes API object using protocol buffer encoding.
*
* @param builder The builder for the response
* @param path The path to call in the API server
* @return The response status
*/
public <T extends Message> ObjectOrStatus<T> delete(T.Builder builder, String path)
throws ApiException, IOException {
return request(builder, path, "DELETE", null, null, null);
}
/**
* Delete a kubernetes API object using protocol buffer encoding.
*
* @param builder The builder for the response
* @param path The path to call in the API server
* @param deleteOptions optional deleteOptions
* @return The response status
*/
public <T extends Message> ObjectOrStatus<T> delete(
T.Builder builder, String path, DeleteOptions deleteOptions)
throws ApiException, IOException {
if (deleteOptions == null) {
return delete(builder, path);
}
HashMap<String, String> headers = new HashMap<>();
headers.put("Content-Type", MEDIA_TYPE);
```

```
headers.put("Accept", MEDIA_TYPE);
String[] localVarAuthNames = new String[] {"BearerToken"};
Request request =
apiClient.buildRequest(
apiClient.getBasePath(),
path,
"DELETE",
new ArrayList<Pair>(),
new ArrayList<Pair>(),
null,
headers,
new HashMap<String, String>(),
new HashMap<String, Object>(),
localVarAuthNames,
null);
byte[] bytes = encode(deleteOptions, "v1", "DeleteOptions");
request =
request.newBuilder().delete(RequestBody.create(MediaType.parse(MEDIA_TYPE), bytes)).build();
return getObjectOrStatusFromServer(builder, request);
}
/**
* Generic protocol buffer based HTTP request. Not intended for general consumption, but public
* for advance use cases.
*
* @param builder The appropriate Builder for the object received from the request.
* @param method The HTTP method (e.g. GET) for this request.
* @param path The URL path to call (e.g. /api/v1/namespaces/default/pods/pod-name)
* @param body The body to send with the request (optional)
* @param apiVersion The 'apiVersion' to use when encoding, required if body is non-null, ignored
* otherwise.
* @param kind The 'kind' field to use when encoding, required if body is non-null, ignored
* otherwise.
* @return An ObjectOrStatus which contains the Object requested, or a Status about the request.
*/
public <T extends Message> ObjectOrStatus<T> request(
T.Builder builder, String path, String method, T body, String apiVersion, String kind)
throws ApiException, IOException {
HashMap<String, String> headers = new HashMap<>();
headers.put("Content-Type", MEDIA_TYPE);
headers.put("Accept", MEDIA_TYPE);
String[] localVarAuthNames = new String[] {"BearerToken"};
Request request =
apiClient.buildRequest(
apiClient.getBasePath(),
path,
method,
new ArrayList<Pair>(),
new ArrayList<Pair>(),
null,
headers,
new HashMap<String, String>(),
new HashMap<String, Object>(),
localVarAuthNames,
```

```
null);
if (body != null) {
byte[] bytes = encode(body, apiVersion, kind);
switch (method) {
case "POST":
request =
request
.newBuilder()
.post(RequestBody.create(MediaType.parse(MEDIA_TYPE), bytes))
.build();
break;
case "PUT":
request =
request
.newBuilder()
.put(RequestBody.create(MediaType.parse(MEDIA_TYPE), bytes))
.build();
break;
case "PATCH":
request =
request
.newBuilder()
.patch(RequestBody.create(MediaType.parse(MEDIA_TYPE), bytes))
.build();
break;
default:
throw new ApiException("Unknown proto client API method: " + method);
}
}
return getObjectOrStatusFromServer(builder, request);
}
private <T extends Message> ObjectOrStatus<T> getObjectOrStatusFromServer(
Builder builder, Request request) throws IOException, ApiException {
Unknown u;
try (Response resp = apiClient.getHttpClient().newCall(request).execute()) {
// Note: closing the response, closes the body and the underlying source.
u = parse(resp.body().source());
}
if (u.getTypeMeta().getApiVersion().equals("v1")
&& u.getTypeMeta().getKind().equals("Status")) {
Status status = Status.newBuilder().mergeFrom(u.getRaw()).build();
return new ObjectOrStatus<>(null, status);
}
return new ObjectOrStatus<>((T) builder.mergeFrom(u.getRaw()).build(), null);
}
// This isn't really documented anywhere except the code, but
// the proto-buf format is:
// * 4 byte magic number
// * Protocol Buffer encoded object of type runtime.Unknown
// * the 'raw' field in that object contains a Protocol Buffer
// encoding of the actual object.
// TODO: Document this somewhere proper.
private static byte[] encode(Message msg, String apiVersion, String kind) throws IOException {
```

```
// It is unfortunate that we have to include apiVersion and kind,
// since we should be able to extract it from the Message, but
// for now at least, those fields are missing from the proto-buffer.
Unknown u =
Unknown.newBuilder()
.setTypeMeta(TypeMeta.newBuilder().setApiVersion(apiVersion).setKind(kind))
.setRaw(msg.toByteString())
.build();
// Encode directly to a sized array, to eliminate buffering
int serializedSize = u.getSerializedSize();
byte[] result = new byte[MAGIC.length + u.getSerializedSize()];
System.arraycopy(MAGIC, 0, result, 0, MAGIC.length);
u.writeTo(CodedOutputStream.newInstance(result, MAGIC.length, serializedSize));
return result;
}
private static Unknown parse(BufferedSource responseBody) throws ApiException, IOException {
if (!responseBody.request(MAGIC.length)) {
throw new ApiException("Truncated reading magic number");
}
// Check the magic without allocating a byte array
if (responseBody.rangeEquals(0, MAGIC_BYTESTRING)) {
responseBody.skip(MAGIC.length);
} else {
ByteString badMagic = responseBody.readByteString(MAGIC.length);
throw new ApiException("Unexpected magic number: " + badMagic.hex());
}
return Unknown.parseFrom(responseBody.inputStream());
}
}
```

io/kubernetes/client/Metrics.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client;
import io.kubernetes.client.custom.NodeMetrics;
import io.kubernetes.client.custom.NodeMetricsList;
import io.kubernetes.client.custom.PodMetrics;
import io.kubernetes.client.custom.PodMetricsList;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
```

```
import io.kubernetes.client.openapi.Configuration;
import io.kubernetes.client.util.generic.GenericKubernetesApi;
public class Metrics {
private ApiClient apiClient;
/** Simple Metrics API constructor, uses default configuration */
public Metrics() {
this(Configuration.getDefaultApiClient());
}
/**
* Metrics API Constructor
*
* @param apiClient The api client to use.
*/
public Metrics(ApiClient apiClient) {
this.apiClient = apiClient;
}
/**
* Get the API client for these Attach operations.
*
* @return The API client that will be used.
*/
public ApiClient getApiClient() {
return apiClient;
}
/**
* Set the API client for subsequent Attach operations.
*
* @param apiClient The new API client to use.
*/
public void setApiClient(ApiClient apiClient) {
this.apiClient = apiClient;
}
public NodeMetricsList getNodeMetrics() throws ApiException {
GenericKubernetesApi<NodeMetrics, NodeMetricsList> metricsClient =
new GenericKubernetesApi<>(
NodeMetrics.class,
NodeMetricsList.class,
"metrics.k8s.io",
"v1beta1",
"nodes",
apiClient);
return metricsClient.list().throwsApiException().getObject();
}
public PodMetricsList getPodMetrics(String namespace) throws ApiException {
GenericKubernetesApi<PodMetrics, PodMetricsList> metricsClient =
new GenericKubernetesApi<>(
PodMetrics.class, PodMetricsList.class, "metrics.k8s.io", "v1beta1", "pods", apiClient);
return metricsClient.list(namespace).throwsApiException().getObject();
}
}
```

io/kubernetes/client/Copy.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.Configuration;
import io.kubernetes.client.openapi.models.V1Pod;
import io.kubernetes.client.util.Streams;
import io.kubernetes.client.util.exception.CopyNotSupportedException;
import java.io.BufferedInputStream;
import java.io.BufferedReader;
import java.io.ByteArrayInputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import org.apache.commons.codec.binary.Base64InputStream;
import org.apache.commons.compress.archivers.ArchiveEntry;
import org.apache.commons.compress.archivers.ArchiveInputStream;
import org.apache.commons.compress.archivers.ArchiveOutputStream;
import org.apache.commons.compress.archivers.tar.TarArchiveEntry;
import org.apache.commons.compress.archivers.tar.TarArchiveInputStream;
import org.apache.commons.compress.archivers.tar.TarArchiveOutputStream;
import org.apache.commons.compress.compressors.gzip.GzipCompressorInputStream;
import org.apache.commons.io.FilenameUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class Copy extends Exec {
private static final Logger log = LoggerFactory.getLogger(Copy.class);
/** Simple Copy constructor, uses default configuration */
public Copy() {
super(Configuration.getDefaultApiClient());
}
/**
```

```
* Copy Constructor
*
* @param apiClient The api client to use.
*/
public Copy(ApiClient apiClient) {
super(apiClient);
}
public InputStream copyFileFromPod(String namespace, String pod, String srcPath)
throws ApiException, IOException {
return copyFileFromPod(namespace, pod, null, srcPath);
}
public InputStream copyFileFromPod(V1Pod pod, String srcPath) throws ApiException, IOException {
return copyFileFromPod(pod, null, srcPath);
}
public InputStream copyFileFromPod(V1Pod pod, String container, String srcPath)
throws ApiException, IOException {
return copyFileFromPod(
pod.getMetadata().getNamespace(), pod.getMetadata().getName(), container, srcPath);
}
public InputStream copyFileFromPod(String namespace, String pod, String container, String srcPath)
throws ApiException, IOException {
Process proc =
this.exec(
namespace,
pod,
new String[] {"sh", "-c", "cat " + srcPath + " | base64"},
container,
false,
false);
return new Base64InputStream(proc.getInputStream());
}
public void copyFileFromPod(
String namespace, String name, String container, String srcPath, Path destination)
throws ApiException, IOException {
try (InputStream is = copyFileFromPod(namespace, name, container, srcPath);
FileOutputStream fos = new FileOutputStream(destination.toFile())) {
Streams.copy(is, fos);
fos.flush();
}
}
public void copyDirectoryFromPod(V1Pod pod, String srcPath, Path destination)
throws ApiException, IOException, CopyNotSupportedException {
copyDirectoryFromPod(pod, null, srcPath, destination);
}
public void copyDirectoryFromPod(V1Pod pod, String container, String srcPath, Path destination)
throws ApiException, IOException, CopyNotSupportedException {
copyDirectoryFromPod(
pod.getMetadata().getNamespace(),
pod.getMetadata().getName(),
container,
srcPath,
destination);
}
```

```
public void copyDirectoryFromPod(String namespace, String pod, String srcPath, Path destination)
throws ApiException, IOException, CopyNotSupportedException {
copyDirectoryFromPod(namespace, pod, null, srcPath, destination);
}
public void copyDirectoryFromPod(
String namespace, String pod, String container, String srcPath, Path destination)
throws ApiException, IOException, CopyNotSupportedException {
// Test that 'tar' is present in the container?
if (!isTarPresentInContainer(namespace, pod, container)) {
throw new CopyNotSupportedException("Tar is not present in the target container");
}
copyDirectoryFromPod(namespace, pod, container, srcPath, destination, true);
}
/**
* Copy directory from a pod to local.
*
* @param namespace
* @param pod
* @param container
* @param srcPath
* @param destination
* @param enableTarCompressing: false if tar is not present in target container
* @throws IOException
* @throws ApiException
*/
public void copyDirectoryFromPod(
String namespace,
String pod,
String container,
String srcPath,
Path destination,
boolean enableTarCompressing)
throws IOException, ApiException {
if (!enableTarCompressing) {
TreeNode tree = new TreeNode(true, srcPath, true);
createDirectoryTree(tree, namespace, pod, container, srcPath);
createDirectoryStructureFromTree(tree, namespace, pod, container, srcPath, destination);
return;
}
Future<Integer> future =
copyDirectoryFromPodAsync(namespace, pod, container, srcPath, destination);
try {
int code = future.get().intValue();
if (code != 0) {
throw new IOException("Copy failed (" + code + ")");
}
} catch (InterruptedException | ExecutionException ex) {
throw new IOException(ex);
}
}
public Future<Integer> copyDirectoryFromPodAsync(
String namespace, String pod, String container, String srcPath, Path destination)
throws IOException, ApiException {
```

```
final Process proc =
this.exec(
namespace,
pod,
new String[] {"sh", "-c", "tar cz - " + srcPath + " | base64"},
container,
false,
false);
try (InputStream is = new Base64InputStream(new BufferedInputStream(proc.getInputStream()));
ArchiveInputStream archive = new TarArchiveInputStream(new GzipCompressorInputStream(is))) {
for (ArchiveEntry entry = archive.getNextEntry();
entry != null;
entry = archive.getNextEntry()) {
if (!archive.canReadEntryData(entry)) {
log.error("Can't read: {}", entry);
continue;
}
String normalName = FilenameUtils.normalize(entry.getName());
if (normalName == null) {
throw new IOException("Invalid entry: " + entry.getName());
}
File f = new File(destination.toFile(), normalName);
if (entry.isDirectory()) {
if (!f.isDirectory() && !f.mkdirs()) {
throw new IOException("create directory failed: " + f);
}
} else {
File parent = f.getParentFile();
if (!parent.isDirectory() && !parent.mkdirs()) {
throw new IOException("create directory failed: " + parent);
}
try (OutputStream fs = new FileOutputStream(f)) {
Streams.copy(archive, fs);
fs.flush();
}
}
}
}
return new ProcessFuture(proc);
}
// This creates directories and files using tree of files and directories under container
private void createDirectoryStructureFromTree(
TreeNode tree,
String namespace,
String pod,
String container,
String srcPath,
Path destination)
throws IOException, ApiException {
// Code for creating invidual directory and files
createDirectory(tree, destination);
createFiles(tree, namespace, pod, container, srcPath, destination);
}
```

```
// Method to create files from directories/files tree in destination
private void createFiles(
TreeNode node,
String namespace,
String pod,
String container,
String srcPath,
Path destination)
throws IOException, ApiException {
if (node == null) {
return;
}
for (TreeNode childNode : node.getChildren()) {
if (!childNode.isDir) {
// Create file which is under 'node'
String filePath = genericPathBuilder(destination.toString(), childNode.name);
File f = new File(filePath);
if (!f.createNewFile()) {
throw new IOException("Failed to create file: " + f);
}
String modifiedSrcPath = genericPathBuilder(srcPath, childNode.name);
try (InputStream is = copyFileFromPod(namespace, pod, modifiedSrcPath);
OutputStream fs = new FileOutputStream(f)) {
Streams.copy(is, fs);
fs.flush();
}
} else {
String newSrcPath = genericPathBuilder(srcPath, childNode.name);
// TODO: Change this once the method genericPathBuilder is changed to varargs
Path newDestination = Paths.get(destination.toString(), childNode.name);
createFiles(childNode, namespace, pod, container, newSrcPath, newDestination);
}
}
}
// Method to create directories in destination path
private void createDirectory(TreeNode node, Path destination) throws IOException {
if (node == null) {
return;
}
if (!node.isRoot) {
// String directoryPath = genericPathBuilder(destination.toString(), node.name);
File f = new File(destination.toString());
// File f = new File(directoryPath);
if (!f.mkdirs()) {
throw new IOException("Failed to create directory: " + f);
}
}
for (TreeNode childNode : node.getChildren()) {
if (childNode.isDir) {
String path = genericPathBuilder(destination.toString(), childNode.name);
Path newPath = Paths.get(path);
createDirectory(childNode, newPath);
}
```

```
}
}
// Method to create a tree of files and directory of location inside container
private void createDirectoryTree(
TreeNode node, String namespace, String pod, String container, String srcPath)
throws IOException, ApiException {
if (node.isDir) {
final Process proc =
this.exec(
namespace,
pod,
new String[] {"sh", "-c", "ls -F " + srcPath},
container,
false,
false);
try (InputStream is = proc.getInputStream();
BufferedReader reader = new BufferedReader(new InputStreamReader(is))) {
String line = "";
while ((line = reader.readLine()) != null) {
int len = line.length();
// line = stripFileSeparators(line);
if (line.charAt(line.length() - 1) == '/') {
TreeNode subDirTree =
new TreeNode(
true,
line.substring(0, len - 1),
false); // Stripping off '/' in the end of directory
String path = genericPathBuilder(srcPath, subDirTree.name);
createDirectoryTree(subDirTree, namespace, pod, container, path);
node.getChildren().add(subDirTree);
} else {
node.getChildren().add(new TreeNode(false, line, false));
}
}
}
}
}
/*
Generic method to create path.
TODO: Change this to varargs
*/
private String genericPathBuilder(String first, String second) {
StringBuilder pathBuilder = new StringBuilder(first);
pathBuilder.append(File.separator);
pathBuilder.append(second);
return pathBuilder.toString();
}
public static void copyFileFromPod(String namespace, String pod, String srcPath, Path dest)
throws ApiException, IOException {
Copy c = new Copy();
try (InputStream is = c.copyFileFromPod(namespace, pod, null, srcPath);
FileOutputStream os = new FileOutputStream(dest.toFile())) {
Streams.copy(is, os);
```

```
os.flush();
}
}
public void copyFileToPod(
String namespace, String pod, String container, Path srcPath, Path destPath)
throws ApiException, IOException {
try {
int exit = copyFileToPodAsync(namespace, pod, container, srcPath, destPath).get();
if (exit != 0) {
throw new IOException("Failed to copy: " + exit);
}
} catch (InterruptedException | ExecutionException ex) {
throw new IOException(ex);
}
}
public Future<Integer> copyFileToPodAsync(
String namespace, String pod, String container, Path srcPath, Path destPath)
throws ApiException, IOException {
// Run decoding and extracting processes
final Process proc = execCopyToPod(namespace, pod, container, destPath);
// Send encoded archive output stream
File srcFile = new File(srcPath.toUri());
try (ArchiveOutputStream archiveOutputStream =
new TarArchiveOutputStream(proc.getOutputStream());
FileInputStream input = new FileInputStream(srcFile)) {
ArchiveEntry tarEntry = new TarArchiveEntry(srcFile, destPath.getFileName().toString());
archiveOutputStream.putArchiveEntry(tarEntry);
Streams.copy(input, archiveOutputStream);
archiveOutputStream.closeArchiveEntry();
return new ProcessFuture(proc);
}
}
public void copyFileToPod(
String namespace, String pod, String container, byte[] src, Path destPath)
throws ApiException, IOException {
try {
int exit = copyFileToPodAsync(namespace, pod, container, src, destPath).get();
if (exit != 0) {
throw new IOException("Copy failed: " + exit);
}
} catch (InterruptedException | ExecutionException ex) {
throw new IOException(ex);
}
}
public Future<Integer> copyFileToPodAsync(
String namespace, String pod, String container, byte[] src, Path destPath)
throws ApiException, IOException {
// Run decoding and extracting processes
final Process proc = execCopyToPod(namespace, pod, container, destPath);
try (ArchiveOutputStream archiveOutputStream =
new TarArchiveOutputStream(proc.getOutputStream())) {
ArchiveEntry tarEntry = new TarArchiveEntry(new File(destPath.getFileName().toString()));
((TarArchiveEntry) tarEntry).setSize(src.length);
```

```
archiveOutputStream.putArchiveEntry(tarEntry);
Streams.copy(new ByteArrayInputStream(src), archiveOutputStream);
archiveOutputStream.closeArchiveEntry();
return new ProcessFuture(proc);
}
}
private Process execCopyToPod(String namespace, String pod, String container, Path destPath)
throws ApiException, IOException {
// TODO: This assumes Linux and won't work on Windows Containers (for many reasons...)
String parentPath = destPath.getParent() != null ? destPath.getParent().toString() : ".";
parentPath = parentPath.replace("\\", "/");
return this.exec(
namespace,
pod,
new String[] {"sh", "-c", "tar -xmf - -C " + parentPath},
container,
true,
false);
}
private boolean isTarPresentInContainer(String namespace, String pod, String container)
throws ApiException, IOException {
final Process proc =
this.exec(
namespace, pod, new String[] {"sh", "-c", "tar --version"}, container, false, false);
// This will work for POSIX based operating systems
try {
int status = proc.waitFor();
return status != 127;
} catch (InterruptedException ex) {
throw new IOException(ex);
} finally {
proc.destroy();
}
}
private static class ProcessFuture implements Future<Integer> {
private Process proc;
ProcessFuture(Process proc) {
this.proc = proc;
}
// TODO: support cancelling?
public boolean cancel(boolean interupt) {
return false;
}
public boolean isCancelled() {
return false;
}
public Integer get() throws InterruptedException {
proc.waitFor();
proc.destroy();
return proc.exitValue();
}
public Integer get(long timeout, TimeUnit unit) throws InterruptedException {
proc.waitFor(timeout, unit);
```

```
proc.destroy();
return proc.exitValue();
}
public boolean isDone() {
return proc.isAlive();
}
}
}
```

io/kubernetes/client/Exec.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client;
import static io.kubernetes.client.KubernetesConstants.*;
import com.google.gson.reflect.TypeToken;
import io.kubernetes.client.custom.IOTrio;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.Configuration;
import io.kubernetes.client.openapi.apis.CoreV1Api;
import io.kubernetes.client.openapi.models.V1Pod;
import io.kubernetes.client.openapi.models.V1Status;
import io.kubernetes.client.openapi.models.V1StatusCause;
import io.kubernetes.client.openapi.models.V1StatusDetails;
import io.kubernetes.client.util.Streams;
import io.kubernetes.client.util.WebSocketStreamHandler;
import io.kubernetes.client.util.WebSockets;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.io.Reader;
import java.io.UnsupportedEncodingException;
import java.lang.reflect.Type;
import java.net.URLEncoder;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
```

```
import java.util.Optional;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import java.util.function.BiConsumer;
import java.util.function.Consumer;
import java.util.function.Supplier;
import org.apache.commons.lang3.StringUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class Exec {
private static final Logger log = LoggerFactory.getLogger(Exec.class);
private ApiClient apiClient;
private Consumer<Throwable> onUnhandledError;
/** Simple Exec API constructor, uses default configuration */
public Exec() {
this(Configuration.getDefaultApiClient());
}
/**
* Exec API Constructor
*
* @param apiClient The api client to use.
*/
public Exec(ApiClient apiClient) {
this.apiClient = apiClient;
}
/**
* Get the API client for these exec operations.
*
* @return The API client that will be used.
*/
public ApiClient getApiClient() {
return apiClient;
}
/**
* Set the API client for subsequent exec operations.
*
* @param apiClient The new API client to use.
*/
public void setApiClient(ApiClient apiClient) {
this.apiClient = apiClient;
}
/**
* Get a {@link Consumer<Throwable>} that will be accepted if there is any unhandled exception
* while the websocket communication is happening.
*
* @return The {@link Consumer<Throwable>} that will be used.
*/
public Consumer<Throwable> getOnUnhandledError() {
return onUnhandledError;
}
/**
```

```
* Set the {@link Consumer<Throwable>} that will be accepted if there is any unhandled exception
* while the websocket communication is happening.
*
* @param onUnhandledError The new {@link Consumer<Throwable>} to use.
*/
public void setOnUnhandledError(Consumer<Throwable> onUnhandledError) {
this.onUnhandledError = onUnhandledError;
}
/**
* Setup a Builder for the given namespace, name and command
*
* @param namespace The namespace of the Pod
* @param name The name of the Pod
* @param command The command to run
*/
public ExecutionBuilder newExecutionBuilder(String namespace, String name, String[] command) {
return new ExecutionBuilder(namespace, name, command);
}
/**
* Execute a command in a container. If there are multiple containers in the pod, uses the first
* container in the Pod.
*
* @param namespace The namespace of the Pod
* @param name The name of the Pod
* @param command The command to run
* @param stdin If true, pass a stdin stream into the container
*/
public Process exec(String namespace, String name, String[] command, boolean stdin)
throws ApiException, IOException {
return exec(namespace, name, command, null, stdin, false);
}
/**
* Execute a command in a container. If there are multiple containers in the pod, uses the first
* container in the Pod.
*
* @param pod The pod where the command is run.
* @param command The command to run
* @param stdin If true, pass a stdin stream into the container
*/
public Process exec(V1Pod pod, String[] command, boolean stdin) throws ApiException, IOException {
return exec(pod, command, pod.getSpec().getContainers().get(0).getName(), stdin, false);
}
/**
* Execute a command in a container. If there are multiple containers in the pod, uses the first
* container in the Pod.
*
* @param namespace The namespace of the Pod
* @param name The name of the Pod
* @param command The command to run
* @param stdin If true, pass a stdin stream into the container
* @param tty If true, stdin is a tty.
*/
public Process exec(String namespace, String name, String[] command, boolean stdin, boolean tty)
```

```
throws ApiException, IOException {
return exec(namespace, name, command, null, stdin, tty);
}
/**
* Execute a command in a container. If there are multiple containers in the pod, uses the first
* container in the Pod.
*
* @param pod The pod where the command is run.
* @param command The command to run
* @param stdin If true, pass a stdin stream into the container
* @param tty If true, stdin is a tty.
*/
public Process exec(V1Pod pod, String[] command, boolean stdin, boolean tty)
throws ApiException, IOException {
return exec(pod, command, pod.getSpec().getContainers().get(0).getName(), stdin, tty);
}
/**
* Execute a command in a container. If there are multiple containers in the pod, uses the first
* container in the Pod.
*
* @param pod The pod where the command is run.
* @param command The command to run
* @param container The container in the Pod where the command is run.
* @param stdin If true, pass a stdin stream into the container.
* @param tty If true, stdin is a TTY (only applies if stdin is true)
*/
public Process exec(V1Pod pod, String[] command, String container, boolean stdin, boolean tty)
throws ApiException, IOException {
if (container == null) {
container = pod.getSpec().getContainers().get(0).getName();
}
return exec(
pod.getMetadata().getNamespace(),
pod.getMetadata().getName(),
command,
container,
stdin,
tty);
}
/**
* Execute a command in a container. If there are multiple containers in the pod, uses the first
* container in the Pod.
*
* @param namespace The namespace of the Pod
* @param name The name of the Pod
* @param command The command to run
* @param container The container in the Pod where the command is run.
* @param stdin If true, pass a stdin stream into the container.
* @param tty If true, stdin is a TTY (only applies if stdin is true)
*/
public Process exec(
String namespace, String name, String[] command, String container, boolean stdin, boolean tty)
throws ApiException, IOException {
```

```
return new ExecutionBuilder(namespace, name, command)
.setContainer(container)
.setStdin(stdin)
.setTty(tty)
.setOnUnhandledError(onUnhandledError)
.execute();
}
/**
* A convenience method. Executes a command remotely on a pod and monitors for events in that
* execution. The monitored events are: <br>
* - connection established (onOpen) <br>
* - connection closed (onClosed) <br>
* - execution error occurred (onError) <br>
* This method also allows to specify a MAX timeout for the execution and returns a future in
* order to monitor the execution flow. <br>
* onError and onClosed callbacks are invoked asynchronously, in a separate thread. <br>
*
* @param namespace a namespace the target pod "lives" in
* @param podName a name of the pod to exec the command on
* @param onOpen a callback invoked upon the connection established event.
* @param onClosed a callback invoked upon the process termination. Return code might not always
* be there. N.B. this callback is invoked before the returned {@link Future} is completed.
* @param onError a callback to handle k8s errors (NOT the command errors/stderr!)
* @param timeoutMs timeout in milliseconds for the execution. I.e. the execution will take this
* many ms or less. If the timeout command is running longer than the allowed timeout, the
* command will be "asked" to terminate gracefully. If the command is still running after the
* grace period, the sigkill will be issued. If null is passed, the timeout will not be used
* and will wait for process to exit itself.
* @param tty whether you need tty to pipe the data. TTY mode will trim some binary data in order
* to make it possible to show on screen (tty)
* @param command a tokenized command to run on the pod
* @return a {@link Future} promise representing this execution. Unless something goes south, the
* promise will contain the process return exit code. If the timeoutMs is non-null and the
* timeout expires before the process exits, promise will contain {@link Integer#MAX_VALUE}.
* @throws IOException
*/
public Future<Integer> exec(
String namespace,
String podName,
Consumer<IOTrio> onOpen,
BiConsumer<Integer, IOTrio> onClosed,
BiConsumer<Throwable, IOTrio> onError,
Long timeoutMs,
boolean tty,
String... command)
throws IOException {
CompletableFuture<Integer> future = new CompletableFuture<>();
IOTrio io = new IOTrio();
String cmdStr = Arrays.toString(command);
BiConsumer<Throwable, IOTrio> errHandler =
(err, errIO) -> {
if (onError != null) {
onError.accept(err, errIO);
```

```
}
};
try {
Process process = exec(namespace, podName, command, null, true, tty);
io.onClose(
(code, timeout) -> {
process.destroy();
waitForProcessToExit(process, timeout, cmdStr, err -> errHandler.accept(err, io));
// processWaitingThread will handle the rest
});
io.setStdin(process.getOutputStream());
io.setStdout(process.getInputStream());
io.setStderr(process.getErrorStream());
runAsync(
"Process-Waiting-Thread-" + command[0] + "-" + podName,
() -> {
Supplier<Integer> returnCode = process::exitValue;
try {
log.debug("Waiting for process to close in {} ms: {}", timeoutMs, cmdStr);
boolean beforeTimeout =
waitForProcessToExit(
process, timeoutMs, cmdStr, err -> errHandler.accept(err, io));
if (!beforeTimeout) {
returnCode = () -> Integer.MAX_VALUE;
}
} catch (Exception e) {
errHandler.accept(e, io);
}
log.debug("process.onExit({}): {}", returnCode.get(), cmdStr);
if (onClosed != null) {
onClosed.accept(returnCode.get(), io);
}
future.complete(returnCode.get());
});
if (onOpen != null) {
onOpen.accept(io);
}
} catch (ApiException e) {
errHandler.accept(e, io);
future.completeExceptionally(e);
}
return future;
}
protected void runAsync(String taskName, Runnable task) {
Thread thread = new Thread(task);
thread.setName(taskName);
thread.start();
}
private boolean waitForProcessToExit(
Process process, Long timeoutMs, String cmdStr, Consumer<Exception> onError) {
boolean beforeTimeout = true;
if (timeoutMs != null && timeoutMs >= 0) {
boolean exited = false;
```

```
try {
exited = process.waitFor(timeoutMs, TimeUnit.MILLISECONDS);
} catch (InterruptedException e) {
onError.accept(e);
}
log.debug("Process closed={}: {}", exited, cmdStr);
if (!exited && process.isAlive()) {
beforeTimeout = false;
log.warn("Process timed out after {} ms. Shutting down: {}", timeoutMs, cmdStr);
process.destroy();
}
} else {
try {
process.waitFor();
} catch (InterruptedException e) {
onError.accept(e);
}
}
return beforeTimeout;
}
public final class ExecutionBuilder {
private final String namespace;
private final String name;
private final String[] command;
private String container;
private boolean stdin;
private boolean stdout;
private boolean stderr;
private boolean tty;
private Consumer<Throwable> onUnhandledError;
private ExecutionBuilder(String namespace, String name, String[] command) {
this.namespace = Objects.requireNonNull(namespace, "namespace");
this.name = Objects.requireNonNull(name, "name");
this.command = Objects.requireNonNull(command, "command");
this.stdin = true;
this.stdout = true;
this.stderr = true;
}
public String getName() {
return name;
}
public String getNamespace() {
return namespace;
}
public String[] getCommand() {
return command;
}
public String getContainer() {
return container;
}
public ExecutionBuilder setContainer(String container) {
this.container = container;
return this;
```

```
}
public boolean getStdin() {
return stdin;
}
public ExecutionBuilder setStdin(boolean stdin) {
this.stdin = stdin;
return this;
}
public boolean getStdout() {
return stdout;
}
public ExecutionBuilder setStdout(boolean stdout) {
this.stdout = stdout;
return this;
}
public boolean getStderr() {
return stderr;
}
public ExecutionBuilder setStderr(boolean stderr) {
this.stderr = stderr;
return this;
}
public boolean getTty() {
return tty;
}
public ExecutionBuilder setTty(boolean tty) {
this.tty = tty;
return this;
}
public Consumer<Throwable> getOnUnhandledError() {
return onUnhandledError;
}
public ExecutionBuilder setOnUnhandledError(Consumer<Throwable> onUnhandledError) {
this.onUnhandledError = onUnhandledError;
return this;
}
private String makePath() {
String[] encodedCommand = new String[command.length];
for (int i = 0; i < command.length; i++) {
try {
encodedCommand[i] = URLEncoder.encode(command[i], "UTF-8");
} catch (UnsupportedEncodingException ex) {
throw new RuntimeException("some thing wrong happend: " + ex.getMessage());
}
}
return "/api/v1/namespaces/"
+ namespace
+ "/pods/"
+ name
+ "/exec?"
+ "stdin="
+ stdin
+ "&stdout="
```

```
+ stdout
+ "&stderr="
+ stderr
+ "&tty="
+ tty
+ (container != null ? "&container=" + container : "")
+ "&command="
+ StringUtils.join(encodedCommand, "&command=");
}
public Process execute() throws ApiException, IOException {
if (container == null) {
CoreV1Api api = new CoreV1Api(apiClient);
V1Pod pod = api.readNamespacedPod(name, namespace).execute();
container = pod.getSpec().getContainers().get(0).getName();
}
ExecProcess exec = new ExecProcess(apiClient, onUnhandledError);
WebSocketStreamHandler handler = exec.getHandler();
WebSockets.stream(makePath(), "GET", apiClient, handler);
return exec;
}
}
static int parseExitCode(ApiClient client, InputStream inputStream) {
try {
Type returnType = new TypeToken<V1Status>() {}.getType();
String body;
try (final Reader reader = new InputStreamReader(inputStream)) {
body = Streams.toString(reader);
}
V1Status status = client.getJSON().deserialize(body, returnType);
if (status == null) {
return -1;
}
if (V1STATUS_SUCCESS.equals(status.getStatus())) return 0;
if (V1STATUS_REASON_NONZEROEXITCODE.equals(status.getReason())) {
V1StatusDetails details = status.getDetails();
if (details != null) {
List<V1StatusCause> causes = details.getCauses();
if (causes != null) {
for (V1StatusCause cause : causes) {
if (V1STATUS_CAUSE_REASON_EXITCODE.equals(cause.getReason())) {
try {
return Integer.parseInt(cause.getMessage());
} catch (NumberFormatException nfe) {
log.error("Error parsing exit code from status channel response", nfe);
}
}
}
}
}
}
} catch (Throwable t) {
log.error("Error parsing exit code from status channel response", t);
}
```

```
// Unable to parse the exit code from the content
return -1;
}
public static class ExecProcess extends Process {
private final WebSocketStreamHandler streamHandler;
private final Consumer<Throwable> onUnhandledError;
private int statusCode = -1;
private boolean isAlive = true;
private final Map<Integer, InputStream> input = new HashMap<>();
private final CountDownLatch latch = new CountDownLatch(1);
public ExecProcess(final ApiClient apiClient) throws IOException {
this(apiClient, Throwable::printStackTrace);
}
public ExecProcess(final ApiClient apiClient, final Consumer<Throwable> onUnhandledError)
throws IOException {
this.onUnhandledError =
Optional.ofNullable(onUnhandledError).orElse(Throwable::printStackTrace);
this.streamHandler =
new WebSocketStreamHandler() {
@Override
protected void handleMessage(int stream, InputStream inStream) throws IOException {
if (stream == 3) {
int exitCode = parseExitCode(apiClient, inStream);
if (exitCode >= 0) {
// notify of process completion
synchronized (ExecProcess.this) {
statusCode = exitCode;
isAlive = false;
}
}
inStream.close();
// Stream ID of `3` delivers the status of exec connection from
// kubelet,
// closing the connection upon 0 exit-code.
this.close();
ExecProcess.this.latch.countDown();
} else super.handleMessage(stream, inStream);
}
@Override
public void failure(Throwable ex) {
super.failure(ex);
ExecProcess.this.onUnhandledError.accept(ex);
synchronized (ExecProcess.this) {
// Try for a pretty unique error code, so if someone searches
// they'll find this
// code.
statusCode = -1975219;
isAlive = false;
ExecProcess.this.latch.countDown();
}
}
@Override
public void close() {
```

```
// notify of process completion
synchronized (ExecProcess.this) {
if (isAlive) {
isAlive = false;
ExecProcess.this.latch.countDown();
}
}
super.close();
}
};
}
// Protected to facilitate unit testing.
protected WebSocketStreamHandler getHandler() {
return streamHandler;
}
@Override
public OutputStream getOutputStream() {
return streamHandler.getOutputStream(0);
}
@Override
public InputStream getInputStream() {
return getInputStream(1);
}
@Override
public InputStream getErrorStream() {
return getInputStream(2);
}
public InputStream getConnectionErrorStream() {
return getInputStream(3);
}
public OutputStream getResizeStream() {
return streamHandler.getOutputStream(4);
}
private synchronized InputStream getInputStream(int stream) {
if (!input.containsKey(stream)) {
input.put(stream, streamHandler.getInputStream(stream));
}
return input.get(stream);
}
@Override
public int waitFor() throws InterruptedException {
this.latch.await();
return statusCode;
}
@Override
public boolean waitFor(long timeout, TimeUnit unit) throws InterruptedException {
this.latch.await(timeout, unit);
return !isAlive();
}
@Override
public synchronized int exitValue() {
if (isAlive) throw new IllegalThreadStateException();
return statusCode;
```

```
}
@Override
public synchronized boolean isAlive() {
return isAlive;
}
@Override
public void destroy() {
streamHandler.close();
for (InputStream in : input.values()) {
try {
in.close();
} catch (IOException ex) {
log.error("Error on close", ex);
}
}
}
}
}
```

io/kubernetes/client/apimachinery/NamespaceName.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.apimachinery;
import static io.kubernetes.client.util.Preconditions.precondition;
import java.util.Objects;
public class NamespaceName {
private final String namespace;
private final String name;
public NamespaceName(String namespace, String name) {
this.namespace = precondition(namespace, Objects::isNull, () -> "namespace must not be null");
this.name = precondition(name, Objects::isNull, () -> "name must not be null");
}
public String getNamespace() {
return namespace;
}
public String getName() {
return name;
}
```

```
@Override
public boolean equals(Object o) {
if (this == o) return true;
if (o == null || getClass() != o.getClass()) return false;
NamespaceName that = (NamespaceName) o;
return Objects.equals(namespace, that.namespace) && Objects.equals(name, that.name);
}
@Override
public int hashCode() {
return Objects.hash(namespace, name);
}
}
```

io/kubernetes/client/apimachinery/GroupVersionKind.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.apimachinery;
import static io.kubernetes.client.util.Preconditions.precondition;
import java.util.Objects;
public class GroupVersionKind extends GroupVersion {
private final String kind;
public GroupVersionKind(String group, String version, String kind) {
super(group, version);
this.kind = precondition(kind, Objects::isNull, () -> "kind must not be null");
}
public String getKind() {
return kind;
}
@Override
public boolean equals(Object o) {
if (this == o) return true;
if (o == null || getClass() != o.getClass()) return false;
if (!super.equals(o)) return false;
GroupVersionKind that = (GroupVersionKind) o;
return Objects.equals(kind, that.kind);
}
@Override
public int hashCode() {
```

```
return Objects.hash(super.hashCode(), kind);
}
}
```

io/kubernetes/client/apimachinery/KubernetesVerb.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.apimachinery;
public enum KubernetesVerb {
GET("get"),
LIST("list"),
CREATE("create"),
UPDATE("update"),
DELETE("delete"),
PATCH("patch"),
WATCH("watch"),
DELETE_COLLECTION("deleteCollection");
private final String value;
KubernetesVerb(String value) {
this.value = value;
}
public static KubernetesVerb of(
String httpVerb, boolean hasNamePathParam, boolean hasWatchParam) {
if (hasWatchParam) {
return WATCH;
}
switch (httpVerb) {
case "GET":
if (!hasNamePathParam) {
return LIST;
}
return GET;
case "POST":
return CREATE;
case "PUT":
return UPDATE;
case "PATCH":
return PATCH;
```

```
case "DELETE":
if (!hasNamePathParam) {
return DELETE_COLLECTION;
}
return DELETE;
}
throw new IllegalArgumentException("invalid HTTP verb for kubernetes client");
}
public String value() {
return value;
}
}
```

io/kubernetes/client/apimachinery/ParseKubernetesResourceException.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.apimachinery;
public class ParseKubernetesResourceException extends Exception {}
```

io/kubernetes/client/apimachinery/KubernetesRequestDigest.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.apimachinery;
import io.kubernetes.client.util.Strings;
```

```
import java.util.regex.Pattern;
import okhttp3.Request;
public class KubernetesRequestDigest {
public static final Pattern RESOURCE_URL_PATH_PATTERN =
Pattern.compile("^/(api|apis)(/\\S+)?/v\\d\\w*/\\S+");
KubernetesRequestDigest(
String urlPath,
boolean isNonResourceRequest,
KubernetesResource resourceMeta,
KubernetesVerb verb) {
this.urlPath = urlPath;
this.isNonResourceRequest = isNonResourceRequest;
this.resourceMeta = resourceMeta;
this.verb = verb;
}
public static KubernetesRequestDigest parse(Request request) {
String urlPath = request.url().encodedPath();
if (!isResourceRequest(urlPath)) {
return nonResource(urlPath);
}
try {
KubernetesResource resourceMeta;
if (urlPath.startsWith("/api/v1")) {
resourceMeta = KubernetesResource.parseCoreResource(urlPath);
} else {
resourceMeta = KubernetesResource.parseRegularResource(urlPath);
}
return new KubernetesRequestDigest(
urlPath,
false,
resourceMeta,
KubernetesVerb.of(
request.method(), hasNamePathParameter(resourceMeta), hasWatchParameter(request)));
} catch (ParseKubernetesResourceException e) {
return nonResource(urlPath);
}
}
private static KubernetesRequestDigest nonResource(String urlPath) {
KubernetesRequestDigest digest = new KubernetesRequestDigest(urlPath, true, null, null);
return digest;
}
public static boolean isResourceRequest(String urlPath) {
return RESOURCE_URL_PATH_PATTERN.matcher(urlPath).matches();
}
private static boolean hasWatchParameter(Request request) {
return !Strings.isNullOrEmpty(request.url().queryParameter("watch"));
}
private static boolean hasNamePathParameter(KubernetesResource resource) {
return !Strings.isNullOrEmpty(resource.getName());
}
private final String urlPath;
private final boolean isNonResourceRequest;
private final KubernetesResource resourceMeta;
```

```
private final KubernetesVerb verb;
public String getUrlPath() {
return urlPath;
}
public boolean isNonResourceRequest() {
return isNonResourceRequest;
}
public KubernetesResource getResourceMeta() {
return resourceMeta;
}
public KubernetesVerb getVerb() {
return verb;
}
@Override
public String toString() {
if (isNonResourceRequest) {
return new StringBuilder().append(verb).append(' ').append(urlPath).toString();
}
String groupVersion;
if (Strings.isNullOrEmpty(resourceMeta.getGroupVersionResource().getGroup())) {
// core resource
groupVersion = "";
} else {
// regular resource
groupVersion =
resourceMeta.getGroupVersionResource().getGroup()
+ "/"
+ resourceMeta.getGroupVersionResource().getVersion();
}
String targetResourceName;
if (Strings.isNullOrEmpty(resourceMeta.getSubResource())) {
targetResourceName = resourceMeta.getGroupVersionResource().getResource();
} else { // subresource
targetResourceName =
resourceMeta.getGroupVersionResource().getResource()
+ "/"
+ resourceMeta.getSubResource();
}
return new StringBuilder()
.append(verb.value())
.append(' ')
.append(groupVersion)
.append(' ')
.append(targetResourceName)
.toString();
}
}
```

io/kubernetes/client/apimachinery/KubernetesResource.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.apimachinery;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
public class KubernetesResource {
public static final Pattern CORE_RESOURCE_URL_PATH_PATTERN =
Pattern.compile(
"^/api/v1(/namespaces/(?<namespace>[\\w-]+))?/(?<resource>[\\w-]+)(/(?<name>[\\w-]+))?(/(?<subres
ource>[\\w-]+))?");
public static final Pattern REGULAR_RESOURCE_URL_PATH_PATTERN =
Pattern.compile(
"^/apis/(?<group>\\S+?)/(?<version>\\S+?)(/namespaces/(?<namespace>[\\w-]+))?/(?<resource>[\\w-]+
)(/(?<name>[\\w-]+))?(/(?<subresource>[\\w-]+))?");
public static KubernetesResource parseCoreResource(String urlPath)
throws ParseKubernetesResourceException {
Matcher matcher = CORE_RESOURCE_URL_PATH_PATTERN.matcher(urlPath);
if (!matcher.matches()) {
throw new ParseKubernetesResourceException();
}
KubernetesResource resource =
new KubernetesResource(
new GroupVersionResource("", "v1", matcher.group("resource")),
matcher.group("subresource"),
matcher.group("namespace"),
matcher.group("name"));
return resource;
}
public static KubernetesResource parseRegularResource(String urlPath)
throws ParseKubernetesResourceException {
Matcher matcher = REGULAR_RESOURCE_URL_PATH_PATTERN.matcher(urlPath);
if (!matcher.matches()) {
throw new ParseKubernetesResourceException();
}
KubernetesResource resource =
new KubernetesResource(
new GroupVersionResource(
matcher.group("group"), matcher.group("version"), matcher.group("resource")),
matcher.group("subresource"),
matcher.group("namespace"),
matcher.group("name"));
return resource;
```

```
}
KubernetesResource(GroupVersionResource gvr, String subResource, String namespace, String
name) {
this.groupVersionResource = gvr;
this.subResource = subResource;
this.namespace = namespace;
this.name = name;
}
private final GroupVersionResource groupVersionResource;
private final String subResource;
private final String namespace;
private final String name;
public GroupVersionResource getGroupVersionResource() {
return groupVersionResource;
}
public String getSubResource() {
return subResource;
}
public String getNamespace() {
return namespace;
}
public String getName() {
return name;
}
}
```

io/kubernetes/client/apimachinery/GroupVersionResource.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.apimachinery;
import static io.kubernetes.client.util.Preconditions.precondition;
import java.util.Objects;
public class GroupVersionResource extends GroupVersion {
private final String resource;
public GroupVersionResource(String group, String version, String resource) {
super(group, version);
this.resource = precondition(resource, Objects::isNull, () -> "resource must not be null");
}
```

```
@Override
public boolean equals(Object o) {
if (this == o) return true;
if (o == null || getClass() != o.getClass()) return false;
if (!super.equals(o)) return false;
GroupVersionResource that = (GroupVersionResource) o;
return Objects.equals(resource, that.resource);
}
@Override
public int hashCode() {
return Objects.hash(super.hashCode(), resource);
}
public String getResource() {
return resource;
}
}
```

io/kubernetes/client/apimachinery/GroupVersion.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.apimachinery;
import static io.kubernetes.client.util.Preconditions.precondition;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.util.Strings;
import java.util.Objects;
public class GroupVersion {
private static final GroupVersion GROUP_VERSION_V1 = new GroupVersion("", "v1");
public static GroupVersion parse(String apiVersion) {
if ("v1".equals(apiVersion)) {
// legacy api group
return GROUP_VERSION_V1;
}
precondition(apiVersion, Strings::isNullOrEmpty, () -> "apiVersion can not be null");
String[] parts = apiVersion.split("/");
precondition(parts, x -> x.length != 2, () -> "Invalid apiVersion: " + apiVersion);
return new GroupVersion(parts[0], parts[1]);
}
public static GroupVersion parse(KubernetesObject obj) {
return parse(obj.getApiVersion());
}
```

```
public GroupVersion(String group, String version) {
this.group = precondition(group, Objects::isNull, () -> "group must not be null");
this.version = precondition(version, Objects::isNull, () -> "version must not be null");
}
private final String group;
private final String version;
public String getGroup() {
return group;
}
public String getVersion() {
return version;
}
@Override
public boolean equals(Object o) {
if (this == o) return true;
if (o == null || getClass() != o.getClass()) return false;
GroupVersion that = (GroupVersion) o;
return Objects.equals(group, that.group) && Objects.equals(version, that.version);
}
@Override
public int hashCode() {
return Objects.hash(group, version);
}
}
```

io/kubernetes/client/monitoring/PrometheusInterceptor.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.monitoring;
import io.kubernetes.client.apimachinery.KubernetesRequestDigest;
import io.kubernetes.client.util.Strings;
import io.prometheus.client.Counter;
import io.prometheus.client.Histogram;
import io.prometheus.client.SimpleTimer;
import java.io.IOException;
import okhttp3.Interceptor;
import okhttp3.Request;
import okhttp3.Response;
```

```
public class PrometheusInterceptor implements Interceptor {
static final String PREFIX = "k8s_java";
static final Counter requests =
Counter.build().name(PREFIX + "_requests_total").help("Total requests").register();
static final Counter codes =
Counter.build()
.name(PREFIX + "_response_code_total")
.help("Response code total")
.labelNames("code")
.register();
static final Histogram resourceRequestLatencyHistogram =
Histogram.build()
.name(PREFIX + "_resource_request_latency_seconds")
.help("Kubernetes resource request latency (seconds)")
.labelNames(
"http_response_code",
"group",
"version",
"resource",
"subresource",
"api_verb",
"namespace")
.register();
static final Histogram nonResourceRequestLatencyHistogram =
Histogram.build()
.name(PREFIX + "_non_resource_request_latency_seconds")
.help("Kubernetes non-resource request latency (seconds)")
.labelNames("http_response_code", "non_resource_url_path")
.register();
@Override
public Response intercept(Interceptor.Chain chain) throws IOException {
requests.inc();
Request request = chain.request();
KubernetesRequestDigest requestDigest = KubernetesRequestDigest.parse(request);
SimpleTimer timer = new SimpleTimer();
final double timecost;
final Response response;
try {
response = chain.proceed(chain.request());
} finally {
timecost = timer.elapsedSeconds();
}
codes.labels(Integer.toString(response.code())).inc();
if (requestDigest.isNonResourceRequest()) {
nonResourceRequestLatencyHistogram
.labels(Integer.toString(response.code()), requestDigest.getUrlPath())
.observe(timecost);
} else {
resourceRequestLatencyHistogram
.labels(
Integer.toString(response.code()),
Strings.nullToEmpty(
requestDigest.getResourceMeta().getGroupVersionResource().getGroup()),
```

```
Strings.nullToEmpty(
requestDigest.getResourceMeta().getGroupVersionResource().getVersion()),
Strings.nullToEmpty(
requestDigest.getResourceMeta().getGroupVersionResource().getResource()),
Strings.nullToEmpty(requestDigest.getResourceMeta().getSubResource()),
requestDigest.getVerb().value(),
Strings.nullToEmpty(requestDigest.getResourceMeta().getNamespace()))
.observe(timecost);
}
return response;
}
}
```

io/kubernetes/client/monitoring/Monitoring.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.monitoring;
import io.kubernetes.client.openapi.ApiClient;
import io.prometheus.client.exporter.HTTPServer;
import java.io.IOException;
public class Monitoring {
public static void startMetricsServer(int port) throws IOException {
new HTTPServer(port);
}
public static void startMetricsServer(String host, int port) throws IOException {
new HTTPServer(host, port);
}
public static void installMetrics(ApiClient client) throws IOException {
client.setHttpClient(
client.getHttpClient().newBuilder().addInterceptor(new PrometheusInterceptor()).build());
}
}
```

io/kubernetes/client/KubernetesConstants.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
```

```
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client;
public final class KubernetesConstants {
private KubernetesConstants() {}
// V1Status values
public static final String V1STATUS_SUCCESS = "Success";
public static final String V1STATUS_FAILURE = "Failure";
// V1Status reason values
public static final String V1STATUS_REASON_NONZEROEXITCODE = "NonZeroExitCode";
// V1Status cause reason values
public static final String V1STATUS_CAUSE_REASON_EXITCODE = "ExitCode";
}
```

io/kubernetes/client/PodLogs.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.Configuration;
import io.kubernetes.client.openapi.apis.CoreV1Api;
import io.kubernetes.client.openapi.models.V1Pod;
import java.io.IOException;
import java.io.InputStream;
import okhttp3.Call;
import okhttp3.Response;
/** Utility class offering streaming access to Pod logs. */
public class PodLogs {
private ApiClient apiClient;
private CoreV1Api coreClient;
/** Simple PodLogs API constructor, uses default configuration */
public PodLogs() {
this(Configuration.getDefaultApiClient());
```

```
}
/**
* PodLogs API Constructor
*
* @param apiClient The api client to use.
*/
public PodLogs(ApiClient apiClient) {
this.apiClient = apiClient;
this.coreClient = new CoreV1Api(apiClient);
}
/**
* Get the API client for these Logs operations.
*
* @return The API client that will be used.
*/
public ApiClient getApiClient() {
return apiClient;
}
public InputStream streamNamespacedPodLog(V1Pod pod) throws ApiException, IOException {
if (pod.getSpec() == null) {
throw new ApiException("pod.spec is null and container isn't specified.");
}
if (pod.getSpec().getContainers() == null || pod.getSpec().getContainers().size() < 1) {
throw new ApiException("pod.spec.containers has no containers");
}
return streamNamespacedPodLog(
pod.getMetadata().getNamespace(),
pod.getMetadata().getName(),
pod.getSpec().getContainers().get(0).getName());
}
// Important note. You must close this stream or else you can leak connections.
public InputStream streamNamespacedPodLog(String namespace, String name, String container)
throws ApiException, IOException {
return streamNamespacedPodLog(namespace, name, container, null, null, false);
}
// Important note. You must close this stream or else you can leak connections.
public InputStream streamNamespacedPodLog(
String namespace,
String name,
String container,
Integer sinceSeconds,
Integer tailLines,
boolean timestamps)
throws ApiException, IOException {
Call call =
coreClient.readNamespacedPodLog(
name,
namespace)
.container(container)
.follow(true)
.pretty("false")
.previous(false)
.sinceSeconds(sinceSeconds)
```

```
.tailLines(tailLines)
.timestamps(timestamps)
.buildCall(null);
Response response = call.execute();
if (!response.isSuccessful()) {
if (response.body() != null) {
response.close();
}
throw new ApiException(response.code(), "Logs request failed: " + response.code());
}
return response.body().byteStream();
}
}
```

io/kubernetes/client/Discovery.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.ApiResponse;
import io.kubernetes.client.openapi.Configuration;
import io.kubernetes.client.openapi.models.V1APIGroup;
import io.kubernetes.client.openapi.models.V1APIGroupList;
import io.kubernetes.client.openapi.models.V1APIResourceList;
import io.kubernetes.client.openapi.models.V1APIVersions;
import io.kubernetes.client.util.exception.IncompleteDiscoveryException;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.Set;
import java.util.stream.Collectors;
import okhttp3.Call;
public class Discovery {
```

```
private final ApiClient apiClient;
public Discovery() {
this(Configuration.getDefaultApiClient());
}
public Discovery(ApiClient apiClient) {
this.apiClient = apiClient;
}
public Set<APIResource> findAll() throws ApiException {
Set<APIResource> allResources = new HashSet<>();
for (String version : legacyCoreApi().getVersions()) {
allResources.addAll(findAll("", Arrays.asList(version), version, "/api/" + version));
}
IncompleteDiscoveryException incompleteDiscoveryException = null;
for (V1APIGroup group : groupDiscovery("/apis").getGroups()) {
try {
allResources.addAll(
findAll(
group.getName(),
group.getVersions().stream().map(v -> v.getVersion()).collect(Collectors.toList()),
group.getPreferredVersion().getVersion()));
} catch (ApiException e){
IncompleteDiscoveryException resourceDiscoveryException = new IncompleteDiscoveryException(
String.format("Unable to retrieve the complete list of server APIs: %s/%s : %s",
group.getName(), group.getPreferredVersion().getVersion(), e.getResponseBody()),
e, allResources);
if (incompleteDiscoveryException == null) {
incompleteDiscoveryException = resourceDiscoveryException;
}else {
incompleteDiscoveryException.addSuppressed(resourceDiscoveryException);
}
}
}
if (incompleteDiscoveryException != null) {
throw incompleteDiscoveryException;
}
return allResources;
}
public Set<APIResource> findAll(String group, List<String> versions, String preferredVersion)
throws ApiException {
return findAll(group, versions, preferredVersion, "/apis/" + group + "/" + preferredVersion);
}
public Set<APIResource> findAll(
String group, List<String> versions, String preferredVersion, String path)
throws ApiException {
V1APIResourceList resourceList = resourceDiscovery(path);
return groupResourcesByName(group, versions, preferredVersion, resourceList);
}
public Set<APIResource> groupResourcesByName(
String group,
List<String> versions,
String preferredVersion,
V1APIResourceList resourceList) {
// parse raw discovery responses to APIResource for better readability
```

```
Set<APIResource> resources =
resourceList.getResources().stream()
.filter(r -> getSubResourceNameIfPossible(r.getName()).isEmpty())
.map(
r ->
new APIResource(
group,
versions,
preferredVersion,
r.getKind(),
r.getNamespaced(),
r.getName(),
r.getSingularName()))
.collect(Collectors.toSet());
// wiring up connections between major-resource and sub-resources
Map<String, Set<String>> subResources =
manageRelationFromResourceToSubResources(resourceList);
resources.stream()
.forEach(
r -> {
if (subResources.containsKey(r.getResourcePlural())) {
r.subResources.addAll(subResources.get(r.getResourcePlural()));
}
});
return resources;
}
private Map<String, Set<String>> manageRelationFromResourceToSubResources(
V1APIResourceList resourceList) {
Map<String, Set<String>> subResources = new HashMap<>();
resourceList.getResources().stream()
.forEach(r -> subResources.put(r.getName(), new HashSet<>()));
resourceList.getResources().stream()
.forEach(
r -> {
getSubResourceNameIfPossible(r.getName())
.ifPresent(
subResourceName -> {
subResources
.computeIfAbsent(
getMajorResourceName(r.getName()),
majorResourceName -> new HashSet<>())
.add(subResourceName);
});
});
return subResources;
}
private String getMajorResourceName(String discoveredResourceName) {
String[] parts = discoveredResourceName.split("/", 2);
return parts[0];
}
private Optional<String> getSubResourceNameIfPossible(String discoveredResourceName) {
boolean isSubResource = discoveredResourceName.contains("/");
if (!isSubResource) {
```

```
return Optional.empty();
}
String[] parts = discoveredResourceName.split("/", 2);
return Optional.of(parts[1]);
}
public V1APIVersions legacyCoreApi() throws ApiException {
return versionDiscovery("/api");
}
public V1APIGroupList groupDiscovery(String path) throws ApiException {
return get(V1APIGroupList.class, path);
}
public V1APIVersions versionDiscovery(String path) throws ApiException {
return get(V1APIVersions.class, path);
}
public V1APIResourceList resourceDiscovery(String path) throws ApiException {
return get(V1APIResourceList.class, path);
}
private <T> T get(Class<T> returnTypeClass, String urlPath) throws ApiException {
Map<String, String> headers = new HashMap<>();
headers.put("Content-Type", "application/json");
Call call =
apiClient.buildCall(
apiClient.getBasePath(),
urlPath,
"GET",
Collections.emptyList(),
Collections.emptyList(),
null,
headers,
Collections.emptyMap(),
Collections.emptyMap(),
new String[] {"BearerToken"},
null);
ApiResponse<T> resourceList = apiClient.execute(call, returnTypeClass);
return resourceList.getData();
}
public static class APIResource {
private final String group;
private final String kind;
private final List<String> versions;
private final String preferredVersion;
private final Boolean isNamespaced;
private final String resourcePlural;
private final String resourceSingular;
private final List<String> subResources;
public APIResource(
String group,
List<String> versions,
String preferredVersion,
String kind,
Boolean isNamespaced,
String resourcePlural,
String resourceSingular) {
```

```
this.group = group;
this.versions = versions;
this.preferredVersion = preferredVersion;
this.kind = kind;
this.isNamespaced = isNamespaced;
this.resourcePlural = resourcePlural;
this.resourceSingular = resourceSingular;
this.subResources = new ArrayList<>();
}
public String getGroup() {
return group;
}
public List<String> getVersions() {
return versions;
}
public String getPreferredVersion() {
return preferredVersion;
}
public String getKind() {
return kind;
}
public Boolean getNamespaced() {
return isNamespaced;
}
public String getResourcePlural() {
return resourcePlural;
}
public String getResourceSingular() {
return resourceSingular;
}
public List<String> getSubResources() {
return subResources;
}
@Override
public boolean equals(Object o) {
if (this == o) return true;
if (o == null || getClass() != o.getClass()) return false;
APIResource that = (APIResource) o;
return Objects.equals(group, that.group)
&& Objects.equals(kind, that.kind)
&& Objects.equals(versions, that.versions)
&& Objects.equals(preferredVersion, that.preferredVersion)
&& Objects.equals(isNamespaced, that.isNamespaced)
&& Objects.equals(resourcePlural, that.resourcePlural)
&& Objects.equals(resourceSingular, that.resourceSingular)
&& Objects.equals(subResources, that.subResources);
}
@Override
public int hashCode() {
return Objects.hash(
group,
kind,
versions,
```

```
preferredVersion,
isNamespaced,
resourcePlural,
resourceSingular,
subResources);
}
}
}
```

io/kubernetes/client/util/Preconditions.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import java.util.function.Predicate;
import java.util.function.Supplier;
/**
* support for different invariant checks, like for a example a non-null argument being passed to a
* method that does not support nulls.
*
* @author wind57
*/
public final class Preconditions {
private Preconditions() {
throw new AssertionError("not supported");
}
/**
* @param validate argument to be validated
* @param predicate to check against
* @param errorSupplier the error message generator
* @param <T>
* @return T if Predicate does not match
* @throws IllegalArgumentException if predicate returns true
*/
public static <T> T precondition(
T validate, Predicate<? super T> predicate, Supplier<String> errorSupplier) {
if (predicate.test(validate)) {
throw new IllegalArgumentException(errorSupplier.get());
}
return validate;
}
```

}

io/kubernetes/client/util/Config.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.util.credentials.AccessTokenAuthentication;
import io.kubernetes.client.util.credentials.UsernamePasswordAuthentication;
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.Reader;
import java.nio.charset.StandardCharsets;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class Config {
private static final Logger log = LoggerFactory.getLogger(Config.class);
public static final String SERVICEACCOUNT_ROOT = "/var/run/secrets/kubernetes.io/serviceaccount";
public static final String SERVICEACCOUNT_CA_PATH = SERVICEACCOUNT_ROOT + "/ca.crt";
public static final String SERVICEACCOUNT_TOKEN_PATH = SERVICEACCOUNT_ROOT +
"/token";
public static final String SERVICEACCOUNT_NAMESPACE_PATH = SERVICEACCOUNT_ROOT +
"/namespace";
public static final String ENV_KUBECONFIG = "KUBECONFIG";
public static final String ENV_SERVICE_HOST = "KUBERNETES_SERVICE_HOST";
public static final String ENV_SERVICE_PORT = "KUBERNETES_SERVICE_PORT";
// The last resort host to try
public static final String DEFAULT_FALLBACK_HOST = "http://localhost:8080";
public static ApiClient fromCluster() throws IOException {
return ClientBuilder.cluster().build();
}
public static ApiClient fromUrl(String url) {
return fromUrl(url, true);
}
public static ApiClient fromUrl(String url, boolean validateSSL) {
return new ApiClient().setBasePath(url).setVerifyingSsl(validateSSL);
```

```
}
public static ApiClient fromUserPassword(String url, String user, String password) {
return fromUserPassword(url, user, password, true);
}
public static ApiClient fromUserPassword(
String url, String user, String password, boolean validateSSL) {
return new ClientBuilder()
.setBasePath(url)
.setAuthentication(new UsernamePasswordAuthentication(user, password))
.setVerifyingSsl(validateSSL)
.build();
}
public static ApiClient fromToken(String url, String token) {
return fromToken(url, token, true);
}
public static ApiClient fromToken(String url, String token, boolean validateSSL) {
return new ClientBuilder()
.setBasePath(url)
.setAuthentication(new AccessTokenAuthentication(token))
.setVerifyingSsl(validateSSL)
.build();
}
public static ApiClient fromConfig(String fileName) throws IOException {
try (BufferedReader bufferedReader =
new BufferedReader(
new InputStreamReader(new FileInputStream(fileName), StandardCharsets.UTF_8.name()))) {
KubeConfig config = KubeConfig.loadKubeConfig(bufferedReader);
config.setFile(new File(fileName));
return fromConfig(config);
}
}
public static ApiClient fromConfig(InputStream stream) throws IOException {
return fromConfig(new InputStreamReader(stream, StandardCharsets.UTF_8.name()));
}
public static ApiClient fromConfig(Reader input) throws IOException {
return fromConfig(KubeConfig.loadKubeConfig(input));
}
public static ApiClient fromConfig(KubeConfig config) throws IOException {
return ClientBuilder.kubeconfig(config).build();
}
/**
* Easy client creation, follows this plan
*
* <ul>
* <li>If $KUBECONFIG is defined, use that config file.
* <li>If $HOME/.kube/config can be found, use that.
* <li>If the in-cluster service account can be found, assume in cluster config.
* <li>Default to localhost:8080 as a last resort.
* </ul>
*
* @return The best APIClient given the previously described rules
*/
public static ApiClient defaultClient() throws IOException {
```

```
return ClientBuilder.standard().build();
}
}
```

io/kubernetes/client/util/Watch.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import com.google.gson.annotations.SerializedName;
import com.google.gson.reflect.TypeToken;
import com.google.gson.stream.JsonReader;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.JSON;
import io.kubernetes.client.openapi.models.V1Status;
import java.io.Closeable;
import java.io.IOException;
import java.io.StringReader;
import java.lang.reflect.Type;
import java.util.Iterator;
import okhttp3.Call;
import okhttp3.ResponseBody;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/**
* Watch class implements watch mechansim of kubernetes. For every list API call with a watch
* parameter you should be able to pass its call to this class and watch changes to that list. For
* example CoreV1Api.listNamespace has watch parameter, so you can create a call using
* CoreV1Api.listNamespaceCall and set watch to True and watch the changes to namespaces.
*/
public class Watch<T> implements Watchable<T>, Closeable {
private static final Logger log = LoggerFactory.getLogger(Watch.class);
/**
* Response class holds a watch response that has a `type` that can be ADDED, MODIFIED, DELETED
* and ERROR. It also hold the actual target object.
*/
public static class Response<T> {
@SerializedName("type")
public String type;
@SerializedName("object")
```

```
public T object;
public V1Status status;
public Response() {}
public Response(String type, T object) {
this.type = type;
this.object = object;
this.status = null;
}
public Response(String type, V1Status status) {
this.type = type;
this.object = null;
this.status = status;
}
}
Type watchType;
ResponseBody response;
JSON json;
Call call;
/**
* Creates a watch on a TYPENAME (T) using an API Client and a Call object.
*
* @param client the API client
* @param call the call object returned by api.{ListOperation}Call(...) method. Make sure watch
* flag is set in the call.
* @param watchType The type of the WatchResponse&lt;T&gt;. Use something like new
* TypeToken&lt;Watch.Response&lt;TYPENAME&gt;&gt;(){}.getType()
* @param <T> TYPENAME.
* @return Watch object on TYPENAME
* @throws ApiException on IO exceptions.
*/
public static <T> Watch<T> createWatch(ApiClient client, Call call, Type watchType)
throws ApiException {
if (client.isDebugging()) {
log.warn(
"Watch is (for now) incompatible with debugging mode active. Watches will not return data until the
watch connection terminates");
throw new ApiException("Watch is incompatible with debugging mode active.");
}
try {
okhttp3.Response response = call.execute();
if (!response.isSuccessful()) {
String respBody = null;
try (ResponseBody body = response.body()) {
if (body != null) {
respBody = body.string();
}
} catch (IOException e) {
throw new ApiException(
response.message(), e, response.code(), response.headers().toMultimap());
}
throw new ApiException(
response.message(), response.code(), response.headers().toMultimap(), respBody);
}
```

```
return new Watch<>(client.getJSON(), response.body(), watchType, call);
} catch (IOException e) {
throw new ApiException(e);
}
}
protected Watch(JSON json, ResponseBody body, Type watchType, Call call) {
this.response = body;
this.watchType = watchType;
this.json = json;
this.call = call;
}
public Response<T> next() {
try {
String line = response.source().readUtf8Line();
if (line == null) {
throw new RuntimeException("Null response from the server.");
}
return parseLine(line);
} catch (IOException e) {
throw new RuntimeException("IO Exception during next method.", e);
}
}
protected boolean isStatus(String line) throws IOException {
boolean found = false;
JsonReader reader = new JsonReader(new StringReader(line));
reader.beginObject();
// extract object data.
while (reader.hasNext()) {
String name = reader.nextName();
if (name.equals("object")) {
found = true;
break;
}
reader.skipValue();
}
if (!found) {
return false;
}
String kind = null;
String apiVersion = null;
reader.beginObject();
while (reader.hasNext()) {
String name = reader.nextName();
if (name.equals("kind")) {
kind = reader.nextString();
} else if (name.equals("apiVersion")) {
apiVersion = reader.nextString();
} else {
reader.skipValue();
}
if (apiVersion != null && kind != null) {
break;
}
```

```
}
return "Status".equals(kind) && "v1".equals(apiVersion);
}
protected Response<T> parseLine(String line) throws IOException {
if (!isStatus(line)) {
return json.deserialize(line, watchType);
}
Type statusType = new TypeToken<Response<V1Status>>() {}.getType();
Response<V1Status> status = json.deserialize(line, statusType);
return new Response<T>(status.type, status.object);
}
public boolean hasNext() {
try {
return !response.source().exhausted();
} catch (IOException e) {
throw new RuntimeException("IO Exception during hasNext method.", e);
}
}
public Iterator<Response<T>> iterator() {
return this;
}
public void remove() {
throw new UnsupportedOperationException("remove");
}
public void close() throws IOException {
this.call.cancel();
this.response.close();
}
}
```

io/kubernetes/client/util/Threads.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import java.util.concurrent.Executors;
import java.util.concurrent.ThreadFactory;
import java.util.concurrent.atomic.AtomicInteger;
public class Threads {
private static ThreadFactory defaultFactory = Executors.defaultThreadFactory();
// Users can inject their own thread
```

```
public static void setDefaultThreadFactory(ThreadFactory factory) {
defaultFactory = factory;
}
public static ThreadFactory threadFactory(String format) {
final AtomicInteger threadNumber = new AtomicInteger(1);
return r -> {
Thread thread = defaultFactory.newThread(r);
// Daemon status inherited from default
thread.setName(String.format(format, threadNumber.getAndIncrement()));
return thread;
};
}
}
```

io/kubernetes/client/util/KubeConfig.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import com.google.gson.JsonParseException;
import com.google.gson.JsonParser;
import io.kubernetes.client.persister.ConfigPersister;
import io.kubernetes.client.util.authenticators.Authenticator;
import io.kubernetes.client.util.authenticators.GCPAuthenticator;
import io.kubernetes.client.util.authenticators.OpenIDConnectAuthenticator;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.Reader;
import java.nio.charset.StandardCharsets;
import java.nio.file.FileSystems;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
```

```
import org.apache.commons.codec.binary.Base64;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.yaml.snakeyaml.LoaderOptions;
import org.yaml.snakeyaml.Yaml;
import org.yaml.snakeyaml.constructor.SafeConstructor;
/** KubeConfig represents a kubernetes client configuration */
public class KubeConfig {
private static final Logger log = LoggerFactory.getLogger(KubeConfig.class);
// Defaults for where to find a kubeconfig file
public static final String ENV_HOME = "HOME";
public static final String KUBEDIR = ".kube";
public static final String KUBECONFIG = "config";
public static final String CRED_TOKEN_KEY = "token";
public static final String CRED_CLIENT_CERTIFICATE_DATA_KEY = "clientCertificateData";
public static final String CRED_CLIENT_KEY_DATA_KEY = "clientKeyData";
private static Map<String, Authenticator> authenticators = new HashMap<>();
// Note to the reader: I considered creating a Config object
// and parsing into that instead of using Maps, but honestly
// this seemed cleaner than a bunch of boilerplate classes
private ArrayList<Object> clusters;
private ArrayList<Object> contexts;
private ArrayList<Object> users;
String currentContextName;
Map<String, Object> currentContext;
Map<String, Object> currentCluster;
Map<String, Object> currentUser;
String currentNamespace;
Object preferences;
ConfigPersister persister;
private File file;
public static void registerAuthenticator(Authenticator auth) {
synchronized (authenticators) {
authenticators.put(auth.getName(), auth);
}
}
static {
registerAuthenticator(new GCPAuthenticator());
registerAuthenticator(new OpenIDConnectAuthenticator());
}
/** Load a Kubernetes config from a Reader */
public static KubeConfig loadKubeConfig(Reader input) {
Yaml yaml = new Yaml(new SafeConstructor(new LoaderOptions()));
Object config = yaml.load(input);
Map<String, Object> configMap = (Map<String, Object>) config;
String currentContext = (String) configMap.get("current-context");
ArrayList<Object> contexts = (ArrayList<Object>) configMap.get("contexts");
ArrayList<Object> clusters = (ArrayList<Object>) configMap.get("clusters");
ArrayList<Object> users = (ArrayList<Object>) configMap.get("users");
Object preferences = configMap.get("preferences");
KubeConfig kubeConfig = new KubeConfig(contexts, clusters, users);
kubeConfig.setContext(currentContext);
kubeConfig.setPreferences(preferences);
```

```
return kubeConfig;
}
public KubeConfig(
ArrayList<Object> contexts, ArrayList<Object> clusters, ArrayList<Object> users) {
this.contexts = contexts;
this.clusters = clusters;
this.users = users;
}
public String getCurrentContext() {
return currentContextName;
}
public boolean setContext(String context) {
if (context == null) {
return false;
}
currentContextName = context;
currentCluster = null;
currentUser = null;
Map<String, Object> ctx = findObject(contexts, context);
if (ctx == null) {
return false;
}
currentContext = (Map<String, Object>) ctx.get("context");
if (currentContext == null) {
return false;
}
String cluster = (String) currentContext.get("cluster");
String user = (String) currentContext.get("user");
currentNamespace = (String) currentContext.get("namespace");
if (cluster != null) {
Map<String, Object> obj = findObject(clusters, cluster);
if (obj != null) {
currentCluster = (Map<String, Object>) obj.get("cluster");
}
}
if (user != null) {
Map<String, Object> obj = findObject(users, user);
if (obj != null) {
currentUser = (Map<String, Object>) obj.get("user");
}
}
return true;
}
public ArrayList<Object> getContexts() {
return contexts;
}
public ArrayList<Object> getClusters() {
return clusters;
}
public ArrayList<Object> getUsers() {
return users;
}
public String getNamespace() {
```

```
return currentNamespace;
}
public Object getPreferences() {
return preferences;
}
public String getServer() {
return getData(currentCluster, "server");
}
public String getCertificateAuthorityData() {
return getData(currentCluster, "certificate-authority-data");
}
public String getCertificateAuthorityFile() {
return getData(currentCluster, "certificate-authority");
}
public String getClientCertificateFile() {
return getData(currentUser, "client-certificate");
}
public String getClientCertificateData() {
return getData(currentUser, "client-certificate-data");
}
public String getClientKeyFile() {
return getData(currentUser, "client-key");
}
public String getClientKeyData() {
return getData(currentUser, "client-key-data");
}
public String getUsername() {
return getData(currentUser, "username");
}
public String getPassword() {
return getData(currentUser, "password");
}
@SuppressWarnings("unchecked")
public Map<String, String> getCredentials() {
if (currentUser == null) {
return null;
}
Map<String, String> credentials = new HashMap<>();
Object authProvider = currentUser.get("auth-provider");
if (authProvider != null) {
Map<String, Object> authProviderMap = (Map<String, Object>) authProvider;
Map<String, Object> authConfig = (Map<String, Object>) authProviderMap.get("config");
if (authConfig != null) {
String name = (String) authProviderMap.get("name");
Authenticator auth = authenticators.get(name);
if (auth != null) {
if (auth.isExpired(authConfig)) {
authConfig = auth.refresh(authConfig);
if (persister != null) {
try {
persister.save(contexts, clusters, users, preferences, currentContextName);
} catch (IOException ex) {
log.error("Failed to persist new token", ex);
```

```
}
}
}
credentials.put(CRED_TOKEN_KEY, auth.getToken(authConfig));
return credentials;
} else {
log.error("Unknown auth provider: {}", name);
}
}
}
Map<String, String> credentialsViaExecCredential =
getCredentialsViaExecCredential((Map<String, Object>) currentUser.get("exec"));
if (credentialsViaExecCredential != null) {
return credentialsViaExecCredential;
}
if (currentUser.containsKey("token")) {
credentials.put(CRED_TOKEN_KEY, (String) currentUser.get("token"));
return credentials;
}
if (currentUser.containsKey("tokenFile")) {
String tokenFile = (String) currentUser.get("tokenFile");
try {
byte[] data = Files.readAllBytes(FileSystems.getDefault().getPath(tokenFile));
credentials.put(CRED_TOKEN_KEY, new String(data, StandardCharsets.UTF_8));
return credentials;
} catch (IOException ex) {
log.error("Failed to read token file", ex);
}
}
return null;
}
/**
* Attempt to create an access token or client certificate by running a configured external
* program.
*
* @see <a
* href="https://kubernetes.io/docs/reference/access-authn-authz/authentication/#client-go-credential-pl
ugins">
* Authenticating » client-go credential plugins</a>
*/
@SuppressWarnings("unchecked")
private Map<String, String> getCredentialsViaExecCredential(Map<String, Object> execMap) {
if (execMap == null) {
return null;
}
Map<String, String> credentials = new HashMap<>();
String apiVersion = (String) execMap.get("apiVersion");
/**
* Some history here: v1 was added in kubernetes 1.21
* (https://github.com/kubernetes/kubernetes/pull/102890) v1alpha1 was removed in Kubernetes
* 1.24 (https://github.com/kubernetes/kubernetes/pull/108616) We need to leave v1alpha1 for now
* to support backwards compatability, but we will likely want to remove it eventually after
* Kubernetes 1.23 is no longer supported }
```

```
*/
if (!"client.authentication.k8s.io/v1".equals(apiVersion)
&& !"client.authentication.k8s.io/v1beta1".equals(apiVersion)
&& !"client.authentication.k8s.io/v1alpha1".equals(apiVersion)) {
log.error("Unrecognized user.exec.apiVersion: {}", apiVersion);
return null;
}
String command = (String) execMap.get("command");
JsonElement root = runExec(command, (List) execMap.get("args"), (List) execMap.get("env"));
if (root == null) {
return null;
}
if (!"ExecCredential".equals(root.getAsJsonObject().get("kind").getAsString())) {
log.error("Unrecognized kind in response");
return null;
}
if (!apiVersion.equals(root.getAsJsonObject().get("apiVersion").getAsString())) {
log.error("Mismatched apiVersion in response");
return null;
}
JsonObject status = root.getAsJsonObject().get("status").getAsJsonObject();
JsonElement token = status.get("token");
if (token == null) {
if (status.get("clientCertificateData") != null && status.get("clientKeyData") != null) {
log.debug("Obtained a client certificate from {}", command);
credentials.put(
CRED_CLIENT_CERTIFICATE_DATA_KEY, status.get("clientCertificateData").getAsString());
credentials.put(CRED_CLIENT_KEY_DATA_KEY, status.get("clientKeyData").getAsString());
return credentials;
} else {
log.warn("No token or certificates produced by {}", command);
return null;
}
}
log.debug("Obtained a token from {}", command);
credentials.put(CRED_TOKEN_KEY, token.getAsString());
return credentials;
// TODO cache tokens between calls, up to .status.expirationTimestamp
// TODO a 401 is supposed to force a refresh,
// but KubeconfigAuthentication hardcodes AccessTokenAuthentication which does not support
// that
// and anyway ClientBuilder only calls Authenticator.provide once per ApiClient;
// we would need to do it on every request
}
private JsonElement runExec(String command, List<String> args, List<Map<String, String>> env) {
List<String> argv = new ArrayList<>();
if (command.contains("/") || command.contains("\\")) {
// Spec is unclear on what should be treated as a “relative command path”.
// This clause should cover anything not resolved from $PATH / %Path%.
Path resolvedCommand;
if (file != null) {
// If we know where the Kubeconfig was located, use that as the base.
resolvedCommand = file.toPath().getParent().resolve(command).normalize();
```

```
} else {
// Otherwise, try the current working directory
resolvedCommand = Paths.get(command).normalize();
}
if (!Files.exists(resolvedCommand)) {
log.error("No such file: {}", resolvedCommand);
return null;
}
// Not checking isRegularFile or isExecutable here; leave that to ProcessBuilder.start.
log.debug("Resolved {} to {}", command, resolvedCommand);
argv.add(resolvedCommand.toString());
} else {
argv.add(command);
}
if (args != null) {
argv.addAll(args);
}
ProcessBuilder pb = new ProcessBuilder(argv);
if (env != null) {
for (Map<String, String> entry : env) {
pb.environment().put(entry.get("name"), entry.get("value"));
}
}
pb.redirectError(ProcessBuilder.Redirect.INHERIT);
try {
Process proc = pb.start();
JsonElement root;
try (InputStream is = proc.getInputStream();
Reader r = new InputStreamReader(is, StandardCharsets.UTF_8)) {
root = JsonParser.parseReader(r);
} catch (JsonParseException x) {
log.error("Failed to parse output of {}", command, x);
return null;
}
int r = proc.waitFor();
if (r != 0) {
log.error("{} failed with exit code {}", command, r);
return null;
}
return root;
} catch (IOException | InterruptedException x) {
log.error("Failed to run {}", command, x);
return null;
}
}
public boolean verifySSL() {
if (currentCluster == null) {
return false;
}
if (currentCluster.containsKey("insecure-skip-tls-verify")) {
return !((Boolean) currentCluster.get("insecure-skip-tls-verify")).booleanValue();
}
return true;
```

```
}
/**
* Set persistence for this config.
*
* @param persister The persistence to use, or null to not persist the config.
*/
public void setPersistConfig(ConfigPersister persister) {
this.persister = persister;
}
/**
* Indicates a file from which this configuration was loaded.
*
* @param file a file path, available for use when resolving relative file paths
*/
public void setFile(File file) {
this.file = file;
}
public void setPreferences(Object preferences) {
this.preferences = preferences;
}
private static String getData(Map<String, Object> obj, String key) {
if (obj == null) {
return null;
}
return (String) obj.get(key);
}
private static Map<String, Object> findObject(ArrayList<Object> list, String name) {
if (list == null) {
return null;
}
for (Object obj : list) {
Map<String, Object> map = (Map<String, Object>) obj;
if (name.equals(map.get("name"))) {
return map;
}
}
return null;
}
public byte[] getDataOrFileRelative(final String data, final String path) throws IOException {
String resolvedPath = path;
if (resolvedPath != null && this.file != null) {
resolvedPath = this.file.toPath().getParent().resolve(path).normalize().toString();
}
return getDataOrFile(data, resolvedPath);
}
private static byte[] getDataOrFile(final String data, final String file) throws IOException {
if (data != null) {
return Base64.decodeBase64(data);
}
if (file != null) {
return Files.readAllBytes(Paths.get(file));
}
return null;
```

```
}
}
```

io/kubernetes/client/util/RetryUtils.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.openapi.ApiException;
import java.net.HttpURLConnection;
public class RetryUtils {
/**
* Invokes kubernetes api and will retry upon CONFLICT(409) response.
*
* <p>Note that this helper method is mostly intended for update calls where CONFLICT can be
* returned when updating resources with a stale resource version.
*
* @param <ApiType> the type parameter
* @param apiTypeApiInvocation the api type api invocation lambda
* @param maxRetry the max retry count
* @return the returned api type
* @throws ApiException the api exception
*/
public static <ApiType> ApiType retryUponConflict(
ApiInvocation<ApiType> apiTypeApiInvocation, int maxRetry) throws ApiException {
int retries = 0;
while (true) {
try {
return apiTypeApiInvocation.call();
} catch (ApiException e) {
if (HttpURLConnection.HTTP_CONFLICT != e.getCode() || ++retries >= maxRetry) {
throw e;
}
}
}
}
}
```

io/kubernetes/client/util/conversion/Jobs.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.conversion;
import io.kubernetes.client.openapi.models.V1CronJob;
import io.kubernetes.client.openapi.models.V1CronJobSpec;
import io.kubernetes.client.openapi.models.V1Job;
import io.kubernetes.client.openapi.models.V1JobSpec;
import io.kubernetes.client.openapi.models.V1ObjectMeta;
import io.kubernetes.client.openapi.models.V1OwnerReference;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
public class Jobs {
/**
* Convert V1beta1CronJob object into V1Job object, based on kubectl code
* https://github.com/kubernetes/kubectl/blob/master/pkg/cmd/create/create_job.go
*
* @param cronJob cronJob object (required)
* @param jobName cronJob name
* @return V1Job object
*/
public static V1Job cronJobToJob(V1CronJob cronJob, String jobName) {
Map<String, String> annotations = new HashMap<>();
Map<String, String> labels = new HashMap<>();
V1JobSpec jobSpec = null;
V1CronJobSpec cronJobSpec = cronJob.getSpec();
if (cronJobSpec != null && cronJobSpec.getJobTemplate() != null) {
V1ObjectMeta metadata = cronJobSpec.getJobTemplate().getMetadata();
if (metadata != null) {
if (metadata.getAnnotations() != null) {
annotations.putAll(metadata.getAnnotations());
}
if (metadata.getLabels() != null) {
labels.putAll(metadata.getLabels());
}
}
jobSpec = cronJobSpec.getJobTemplate().getSpec();
}
annotations.put("cronjob.kubernetes.io/instantiate", "manual");
V1OwnerReference v1OwnerReference = new V1OwnerReference();
v1OwnerReference.setKind("CronJob");
v1OwnerReference.setName(cronJob.getMetadata().getName());
v1OwnerReference.setBlockOwnerDeletion(true);
```

```
v1OwnerReference.setController(true);
v1OwnerReference.setUid(cronJob.getMetadata().getUid());
v1OwnerReference.setApiVersion("batch/v1beta1");
V1ObjectMeta jobMetadata = new V1ObjectMeta();
jobMetadata.setName(jobName != null ? jobName : cronJob.getMetadata().getName() + "-manual");
jobMetadata.setAnnotations(annotations);
jobMetadata.setLabels(labels);
jobMetadata.setOwnerReferences(Arrays.asList(v1OwnerReference));
V1Job job = new V1Job();
job.setKind("Job");
job.setApiVersion("batch/v1");
job.setMetadata(jobMetadata);
job.setSpec(jobSpec);
return job;
}
}
```

io/kubernetes/client/util/Watchable.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import java.util.Iterator;
public interface Watchable<T>
extends Iterable<Watch.Response<T>>,
Iterator<Watch.Response<T>>,
java.io.Closeable,
AutoCloseable {}
```

io/kubernetes/client/util/WebSockets.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
```

```
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.Pair;
import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.Reader;
import java.io.StringReader;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import javax.annotation.Nullable;
import okhttp3.Call;
import okhttp3.Request;
import okhttp3.Response;
import okhttp3.WebSocket;
import okhttp3.WebSocketListener;
import okio.ByteString;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class WebSockets {
private static final Logger log = LoggerFactory.getLogger(WebSockets.class);
// Only support v4 stream protocol as it was available since k8s 1.4
public static final String V4_STREAM_PROTOCOL = "v4.channel.k8s.io";
public static final String STREAM_PROTOCOL_HEADER = "Sec-WebSocket-Protocol";
public static final String SPDY_3_1 = "SPDY/3.1";
public static final String CONNECTION = "Connection";
public static final String UPGRADE = "Upgrade";
/** A simple interface for a listener on a web socket */
public interface SocketListener {
/** Called when the socket is opened */
void open(String protocol, WebSocket socket);
/**
* Called when a binary media type message is received
*
* @param in The input stream containing the binary data
*/
void bytesMessage(InputStream in);
/**
* Called when a text media type message is received
*
* @param in The character stream containing the message
*/
void textMessage(Reader in);
/**
* Called when there has been a failure
*
* @param t the exception associated with the failure.
*/
```

```
void failure(Throwable t);
/** Called when the stream is closed. */
void close();
}
/**
* Create a new WebSocket stream
*
* @param path The HTTP Path to request from the API
* @param method The HTTP method to use for the call
* @param client The ApiClient for communicating with the API
* @param listener The socket listener to handle socket events
*/
public static void stream(String path, String method, ApiClient client, SocketListener listener)
throws ApiException, IOException {
stream(path, method, new ArrayList<Pair>(), client, listener);
}
public static void stream(
String path, String method, List<Pair> queryParams, ApiClient client, SocketListener listener)
throws ApiException, IOException {
HashMap<String, String> headers = new HashMap<String, String>();
headers.put(STREAM_PROTOCOL_HEADER, V4_STREAM_PROTOCOL);
headers.put(WebSockets.CONNECTION, WebSockets.UPGRADE);
headers.put(WebSockets.UPGRADE, SPDY_3_1);
String[] localVarAuthNames = new String[] {"BearerToken"};
Request request =
client.buildRequest(
client.getBasePath(),
path,
method,
queryParams,
new ArrayList<Pair>(),
null,
headers,
new HashMap<String, String>(),
new HashMap<String, Object>(),
localVarAuthNames,
null);
streamRequest(request, client, listener);
}
public static void stream(Call call, ApiClient client, SocketListener listener) {
streamRequest(call.request(), client, listener);
}
private static void streamRequest(Request request, ApiClient client, SocketListener listener) {
client.getHttpClient().newWebSocket(request, new Listener(listener));
}
public static class Listener extends WebSocketListener {
private SocketListener listener;
public Listener(SocketListener listener) {
this.listener = listener;
}
@Override
public void onOpen(final WebSocket webSocket, Response response) {
String protocol = response.header(STREAM_PROTOCOL_HEADER, "missing");
```

```
listener.open(protocol, webSocket);
}
@Override
public void onMessage(WebSocket webSocket, String text) {
listener.textMessage(new StringReader(text));
}
@Override
public void onMessage(WebSocket webSocket, ByteString bytes) {
listener.bytesMessage(new ByteArrayInputStream(bytes.toByteArray()));
}
@Override
public void onClosing(WebSocket webSocket, int code, String reason) {
super.onClosing(webSocket, code, reason);
if (code == 1000) {
webSocket.close(1000, "Normal close");
} else {
log.warn("Unexpected WebSocket ({}) closure: {} {}", webSocket, code, reason);
webSocket.close(1002, "Abnormal close");
}
}
@Override
public void onClosed(WebSocket webSocket, int code, String reason) {
listener.close();
}
@Override
public void onFailure(WebSocket webSocket, Throwable t, @Nullable Response response) {
listener.failure(t);
listener.close();
}
}
}
```

io/kubernetes/client/util/CSRUtils.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.apis.CertificatesV1Api;
import io.kubernetes.client.openapi.models.V1CertificateSigningRequest;
```

```
import io.kubernetes.client.openapi.models.V1CertificateSigningRequestCondition;
import io.kubernetes.client.openapi.models.V1CertificateSigningRequestSpec;
import io.kubernetes.client.openapi.models.V1ObjectMeta;
import io.kubernetes.client.util.exception.CSRNotApprovedException;
import io.kubernetes.client.util.exception.CSRSigningException;
import io.kubernetes.client.util.wait.Wait;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.PrintStream;
import java.security.KeyPair;
import java.time.Duration;
import java.time.OffsetDateTime;
import java.util.Base64;
import java.util.Objects;
import java.util.Optional;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.atomic.AtomicReference;
import javax.security.auth.x500.X500Principal;
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.OperatorCreationException;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;
import org.bouncycastle.pkcs.PKCS10CertificationRequest;
import org.bouncycastle.pkcs.PKCS10CertificationRequestBuilder;
import org.bouncycastle.pkcs.jcajce.JcaPKCS10CertificationRequestBuilder;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class CSRUtils {
private static final Logger LOG = LoggerFactory.getLogger(CSRUtils.class);
public static final String CSR_USAGE_CLIENT_AUTH = "client auth";
public static final String SIGNER_NAME_KUBE_APISERVER_CLIENT =
"kubernetes.io/kube-apiserver-client";
/**
* Approves a CertificateSigningRequest by requesting "/approve" subresource.
*
* @param apiClient the api client
* @param csrObjName the csr obj name
* @throws ApiException the api exception
*/
public static void approve(ApiClient apiClient, String csrObjName) throws ApiException {
CertificatesV1Api api = new CertificatesV1Api(apiClient);
OffsetDateTime now = OffsetDateTime.now();
V1CertificateSigningRequest current = api.readCertificateSigningRequest(csrObjName).execute();
current
.getStatus()
.addConditionsItem(
new V1CertificateSigningRequestCondition()
.type("Approved")
.status("True")
.reason("Kubernetes Java Client")
.lastTransitionTime(now)
.lastUpdateTime(now));
api.replaceCertificateSigningRequestApproval(csrObjName, current).execute();
```

```
}
/**
* Creates the given CertificateSigningRequest object if it doesn't exist, then waits until it's
* approved.
*
* <p>It's short-cut combo of CSRUtils#createIfAbsent and CSRUtils#waitUntilCertificateSigned
*
* @param bootstrapApiClient the bootstrap api client
* @param csr the csr
* @return the bytes of CSR [ ]
* @throws CSRNotApprovedException the csr not approved exception
* @throws ApiException the api exception
*/
public static byte[] createAndWaitUntilCertificateSigned(
ApiClient bootstrapApiClient, V1CertificateSigningRequest csr)
throws CSRNotApprovedException, ApiException {
// creates CSR or checks whether the existing one conflicts.
if (!CSRUtils.createIfAbsent(bootstrapApiClient, csr)) {
CertificatesV1Api api = new CertificatesV1Api(bootstrapApiClient);
V1CertificateSigningRequest existing =
api.readCertificateSigningRequest(csr.getMetadata().getName()).execute();
if (!CSRUtils.isIdentical(existing, csr)) {
LOG.error(
"Existing CertificateSigningRequest object is conflicting with the requesting object");
throw new IllegalStateException("Conflicting CSR object found in the cluster");
}
}
// wait until the certificates is approved.
return CSRUtils.waitUntilCertificateSigned(bootstrapApiClient, csr.getMetadata().getName());
}
/**
* Creates if the CertificateSigningRequest is absent in the cluster.
*
* @param apiClient the api client
* @param csr the csr
* @return the boolean represents whether the CSR is actually created.
* @throws ApiException the api exception
*/
public static boolean createIfAbsent(ApiClient apiClient, V1CertificateSigningRequest csr)
throws ApiException {
CertificatesV1Api api = new CertificatesV1Api(apiClient);
try {
api.createCertificateSigningRequest(csr).execute();
return true;
} catch (ApiException e) {
if (e.getCode() == 409) { // HTTP-Conflict
return false;
}
LOG.error(
"Failed creating CSR {} in the cluster: {}",
csr.getMetadata().getName(),
e.getResponseBody());
throw e;
```

```
}
}
/**
* Wait until the CertificateSigningRequest is approved within a timeout of 30 minutes.
*
* @param apiClient the api client
* @param csrObjectName the csr object name
* @return the byte [ ]
* @throws CSRNotApprovedException the csr not approved exception
*/
public static byte[] waitUntilCertificateSigned(ApiClient apiClient, String csrObjectName)
throws CSRNotApprovedException {
return waitUntilCertificateSigned(
apiClient, csrObjectName, Duration.ofSeconds(5), Duration.ofMinutes(30));
}
/**
* Wait until the CertificateSigningRequest is approved within a timeout of 30 minutes.
*
* @param apiClient the api client
* @param csrObjectName the csr object name
* @param retryInterval the retry interval
* @param timeout the timeout
* @return the byte [ ]
* @throws CSRNotApprovedException the csr not approved exception
*/
public static byte[] waitUntilCertificateSigned(
ApiClient apiClient, String csrObjectName, Duration retryInterval, Duration timeout)
throws CSRNotApprovedException {
CertificatesV1Api api = new CertificatesV1Api(apiClient);
ScheduledExecutorService service = Executors.newSingleThreadScheduledExecutor();
try {
AtomicReference<byte[]> certRef = new AtomicReference<>();
boolean certificateSigned =
Wait.poll(
retryInterval,
timeout,
() -> {
try {
V1CertificateSigningRequest current =
api.readCertificateSigningRequest(csrObjectName).execute();
CSRUtils.getCertificate(current).ifPresent(cert -> certRef.set(cert));
return true;
} catch (ApiException e) {
LOG.info(
"Failed acquiring latest state of CertificateSigningRequest resource {} from the cluster",
csrObjectName);
return false;
}
});
if (!certificateSigned) {
LOG.error(
"Timeout exceed but the CertificateSigningRequest {} is not approved", csrObjectName);
throw new CSRNotApprovedException(
```

```
"Timeout - CertificateSigningRequest not approved: " + csrObjectName);
}
LOG.info(
"Successfully acquired certificate from CertificateSigningRequest {}", csrObjectName);
return certRef.get();
} finally {
service.shutdown();
}
}
/**
* Gets certificate data from the CertificateSigningRequest object.
*
* @param csr the csr
* @return the certificate
*/
public static Optional<byte[]> getCertificate(V1CertificateSigningRequest csr) {
if (csr.getStatus() == null || csr.getStatus().getCertificate() == null) {
return Optional.empty();
}
return Optional.of(csr.getStatus().getCertificate());
}
/**
* Checks if the two CertificateSigningRequest is identical.
*
* @param left the left
* @param right the right
* @return the boolean
*/
public static boolean isIdentical(
V1CertificateSigningRequest left, V1CertificateSigningRequest right) {
// TODO: check CSR modulus
if (!Objects.equals(left.getSpec().getUsages(), right.getSpec().getUsages())) {
return false;
}
return Objects.equals(left.getSpec().getSignerName(), right.getSpec().getSignerName());
}
/**
* Sign CSR from the key-pair.
*
* @param keyPair the key pair
* @param commonName the common name
* @return the byte [ ]
* @throws CSRSigningException the csr signing exception
*/
public static byte[] sign(KeyPair keyPair, String commonName) throws CSRSigningException {
return sign(keyPair, "SHA512withRSA", commonName, "");
}
/**
* Sign CSR from the key-pair.
*
* @param keyPair the key pair
* @param csrAlgo the csr algo
* @param commonName the common name
```

```
* @param organizationName the organization name
* @return the byte [ ]
* @throws CSRSigningException the csr signing exception
*/
public static byte[] sign(
KeyPair keyPair, String csrAlgo, String commonName, String organizationName)
throws CSRSigningException {
return sign(
keyPair,
csrAlgo,
String.join(
", ",
new String[] {
"CN=" + commonName, "O=" + organizationName,
}));
}
/**
* Sign CSR from the key-pair.
*
* @param keyPair the key pair
* @param csrAlgo the csr algo
* @param subjects the subjects
* @return the byte [ ]
* @throws CSRSigningException the csr signing exception
*/
public static byte[] sign(KeyPair keyPair, String csrAlgo, String subjects)
throws CSRSigningException {
try {
PKCS10CertificationRequestBuilder p10Builder =
new JcaPKCS10CertificationRequestBuilder(
new X500Principal(subjects), keyPair.getPublic());
JcaContentSignerBuilder csBuilder = new JcaContentSignerBuilder(csrAlgo);
ContentSigner signer = csBuilder.build(keyPair.getPrivate());
PKCS10CertificationRequest csr = p10Builder.build(signer);
// NOTE: a work-around for https://github.com/kubernetes/kubernetes/pull/96747
ByteArrayOutputStream bs = new ByteArrayOutputStream();
PrintStream ps = new PrintStream(bs);
byte[] CRLF = new byte[] {'\r', '\n'};
ps.println("-----BEGIN CERTIFICATE REQUEST-----");
ps.println(Base64.getMimeEncoder(64, CRLF).encodeToString(csr.getEncoded()));
ps.println("-----END CERTIFICATE REQUEST-----");
return bs.toByteArray();
} catch (IOException | OperatorCreationException e) {
throw new CSRSigningException(e);
}
}
public static V1CertificateSigningRequest newV1CertificateSigningRequest(
String csrObjName, byte[] csrBytes) {
return newV1CertificateSigningRequest(
csrObjName, csrBytes, SIGNER_NAME_KUBE_APISERVER_CLIENT,
CSR_USAGE_CLIENT_AUTH);
}
public static V1CertificateSigningRequest newV1CertificateSigningRequest(
```

```
String csrObjName, byte[] csrBytes, String signerName, String usage) {
return new V1CertificateSigningRequest()
.metadata(new V1ObjectMeta().name(csrObjName))
.spec(
new V1CertificateSigningRequestSpec()
.request(csrBytes)
.signerName(signerName)
.addUsagesItem(usage));
}
}
```

io/kubernetes/client/util/Streams.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.io.Reader;
public class Streams {
public static final int BUFFER_SIZE = 4096;
public static void copy(InputStream in, OutputStream out) throws IOException {
byte[] buffer = new byte[BUFFER_SIZE];
int bytesRead;
while ((bytesRead = in.read(buffer)) != -1) {
out.write(buffer, 0, bytesRead);
}
out.flush();
}
public static String toString(Reader reader) throws IOException {
StringBuilder out = new StringBuilder(BUFFER_SIZE);
char[] buffer = new char[BUFFER_SIZE];
int charsRead;
while ((charsRead = reader.read(buffer)) != -1) {
out.append(buffer, 0, charsRead);
}
return out.toString();
}
public static void readFully(InputStream in, byte[] bytes) throws IOException {
int total = 0;
```

```
int len = bytes.length;
while (total < len) {
int result = in.read(bytes, total, len - total);
if (result == -1) {
break;
}
total += result;
}
}
}
```

io/kubernetes/client/util/CallGeneratorParams.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
public class CallGeneratorParams {
public Boolean watch;
public String resourceVersion;
public Integer timeoutSeconds;
public CallGeneratorParams(Boolean watch, String resourceVersion, Integer timeoutSeconds) {
this.watch = watch;
this.resourceVersion = resourceVersion;
this.timeoutSeconds = timeoutSeconds;
}
}
```

io/kubernetes/client/util/wait/Wait.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
```

```
*/
package io.kubernetes.client.util.wait;
import java.time.Duration;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.ScheduledFuture;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.function.Supplier;
public class Wait {
/**
* Poll tries a condition func until it returns true, an exception, or the timeout is reached.
*
* @param interval the check interval
* @param timeout the timeout period
* @param condition the condition func
*/
public static boolean poll(Duration interval, Duration timeout, Supplier<Boolean> condition) {
return poll(interval, interval, timeout, condition);
}
/**
* Poll tries a condition func until w/ the initial delay specified.
*
* @param initialDelay the initial delay
* @param interval the check interval
* @param timeout the timeout period
* @param condition the condition
* @return returns true if gracefully finished
*/
public static boolean poll(
Duration initialDelay, Duration interval, Duration timeout, Supplier<Boolean> condition) {
ScheduledExecutorService executorService = Executors.newSingleThreadScheduledExecutor();
AtomicBoolean result = new AtomicBoolean(false);
long dueDate = System.currentTimeMillis() + timeout.toMillis();
ScheduledFuture<?> future =
executorService.scheduleAtFixedRate(
() -> {
try {
result.set(condition.get());
} catch (Exception e) {
result.set(false);
}
},
initialDelay.toMillis(),
interval.toMillis(),
TimeUnit.MILLISECONDS);
try {
while (System.currentTimeMillis() < dueDate) {
if (result.get()) {
future.cancel(true);
return true;
}
}
```

```
} catch (Exception e) {
return result.get();
}
future.cancel(true);
return result.get();
}
}
```

io/kubernetes/client/util/okhttp/HTTP2ConsoleHandler.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.okhttp;
import java.util.logging.ConsoleHandler;
import java.util.logging.Level;
import java.util.logging.LogRecord;
class HTTP2ConsoleHandler extends ConsoleHandler {
public HTTP2ConsoleHandler() {
setLevel(Level.FINE);
setFormatter(new Formatter());
}
public static class Formatter extends java.util.logging.Formatter {
@Override
public String format(LogRecord record) {
return String.format("[%1$tF %1$tT] %2$s %n", record.getMillis(), record.getMessage());
}
}
}
```

io/kubernetes/client/util/okhttp/HTTP2Logging.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
```

```
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.okhttp;
import java.util.logging.Level;
import java.util.logging.Logger;
import okhttp3.internal.concurrent.TaskRunner;
import okhttp3.internal.http2.Http2;
public class HTTP2Logging {
private static final HTTP2ConsoleHandler consoleHandler = new HTTP2ConsoleHandler();
private static boolean debuggingHTTP2 = false;
private static final Logger http2Logger = Logger.getLogger(Http2.class.getName());
private static boolean debuggingTaskRunner = false;
private static final Logger taskRunnerLogger = Logger.getLogger(TaskRunner.class.getName());
public static void enableHTTP2Logging(boolean enable) {
if (debuggingHTTP2 == enable) {
return;
}
if (enable) {
http2Logger.addHandler(consoleHandler);
http2Logger.setLevel(Level.FINEST);
} else {
http2Logger.removeHandler(consoleHandler);
http2Logger.setLevel(Level.INFO);
}
debuggingHTTP2 = enable;
}
public static void enableTaskRunnerLogging(boolean enable) {
if (debuggingTaskRunner == enable) {
return;
}
if (enable) {
taskRunnerLogger.addHandler(consoleHandler);
taskRunnerLogger.setLevel(Level.FINEST);
} else {
taskRunnerLogger.removeHandler(consoleHandler);
taskRunnerLogger.setLevel(Level.INFO);
}
debuggingTaskRunner = enable;
}
}
```

io/kubernetes/client/util/Yaml.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
```

http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
```
package io.kubernetes.client.util;
import com.google.gson.annotations.SerializedName;
import io.kubernetes.client.common.KubernetesType;
import io.kubernetes.client.custom.IntOrString;
import io.kubernetes.client.custom.Quantity;
import io.kubernetes.client.openapi.models.V1JSONSchemaProps;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.io.Reader;
import java.io.StringReader;
import java.io.Writer;
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.time.OffsetDateTime;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import okio.ByteString;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.yaml.snakeyaml.DumperOptions;
import org.yaml.snakeyaml.DumperOptions.FlowStyle;
import org.yaml.snakeyaml.LoaderOptions;
import org.yaml.snakeyaml.TypeDescription;
import org.yaml.snakeyaml.constructor.BaseConstructor;
import org.yaml.snakeyaml.constructor.Constructor;
import org.yaml.snakeyaml.constructor.SafeConstructor;
import org.yaml.snakeyaml.introspector.Property;
import org.yaml.snakeyaml.nodes.MappingNode;
import org.yaml.snakeyaml.nodes.Node;
import org.yaml.snakeyaml.nodes.NodeTuple;
import org.yaml.snakeyaml.nodes.ScalarNode;
import org.yaml.snakeyaml.nodes.Tag;
import org.yaml.snakeyaml.representer.Represent;
import org.yaml.snakeyaml.representer.Representer;
/** The type Yaml. */
public class Yaml {
static final Logger logger = LoggerFactory.getLogger(Yaml.class);
/**
```

```
* Load an API object from a YAML string representation. Returns a concrete typed object (e.g.
* V1Pod)
*
* @param content The YAML content
* @return An instantiation of the object.
* @throws IOException If an error occurs while reading the YAML.
*/
public static Object load(String content) throws IOException {
return load(new StringReader(content));
}
/**
* Load an API object from a YAML file. Returns a concrete typed object (e.g. V1Pod)
*
* @param f The file to load.
* @return An instantiation of the object.
* @throws IOException If an error occurs while reading the YAML.
*/
public static Object load(File f) throws IOException {
return load(new FileReader(f));
}
/**
* Load an API object from a stream of data. Returns a concrete typed object (e.g. V1Pod)
*
* @param reader The stream to load.
* @return An instantiation of the object.
* @throws IOException If an error occurs while reading the YAML.
*/
public static Object load(Reader reader) throws IOException {
Map<String, Object> data = getSnakeYaml(null).load(reader);
return modelMapper(data);
}
/**
* Load an API object from a YAML string representation. Returns a concrete typed object using the
* type specified.
*
* @param content The YAML content
* @param clazz The class of object to return.
* @return An instantiation of the object.
* @throws IOException If an error occurs while reading the YAML.
*/
public static <T> T loadAs(String content, Class<T> clazz) {
return getSnakeYaml(clazz).loadAs(new StringReader(content), clazz);
}
/**
* Load an API object from a YAML file. Returns a concrete typed object using the type specified.
*
* @param f The YAML file
* @param clazz The class of object to return.
* @return An instantiation of the object.
* @throws IOException If an error occurs while reading the YAML.
*/
public static <T> T loadAs(File f, Class<T> clazz) throws IOException {
return getSnakeYaml(clazz).loadAs(new FileReader(f), clazz);
```

```
}
/**
* Load an API object from a YAML stream. Returns a concrete typed object using the type
* specified.
*
* @param reader The YAML stream
* @param clazz The class of object to return.
* @return An instantiation of the object.
* @throws IOException If an error occurs while reading the YAML.
*/
public static <T> T loadAs(Reader reader, Class<T> clazz) {
return getSnakeYaml(clazz).loadAs(reader, clazz);
}
/**
* Load list of instantiated API objects from a YAML string representation. Returns list of
* concrete typed objects (e.g. { V1Pod, V1SERVICE })
*
* <p>Order of API objects in list will be preserved according to order of objects in YAML string.
*
* @param content The YAML content
* @return List of instantiated objects.
* @throws IOException If an error occurs while reading the YAML.
*/
public static List<Object> loadAll(String content) throws IOException {
return loadAll(new StringReader(content));
}
/**
* Load list of instantiated API objects from a YAML file. Returns list of concrete typed objects
* (e.g. { V1Pod, V1SERVICE })
*
* <p>Order of API objects in list will be preserved according to order of objects in YAML file.
*
* @param f The file to load.
* @return List of instantiated of the objects.
* @throws IOException If an error occurs while reading the YAML.
*/
public static List<Object> loadAll(File f) throws IOException {
return loadAll(new FileReader(f));
}
/**
* Load list of instantiated API objects from a stream of data. Returns list of concrete typed
* objects (e.g. { V1Pod, V1SERVICE })
*
* <p>Order of API objects in list will be preserved according to order of objects in stream of
* data.
*
* @param reader The stream to load.
* @return List of instantiated of the objects.
* @throws IOException If an error occurs while reading the YAML.
*/
public static List<Object> loadAll(Reader reader) throws IOException {
Iterable<Object> iterable = getSnakeYaml(null).loadAll(reader);
List<Object> list = new ArrayList<Object>();
```

```
for (Object object : iterable) {
if (object != null) {
try {
list.add(modelMapper((Map<String, Object>) object));
} catch (ClassCastException ex) {
logger.error("Unexpected exception while casting: {}", ex);
}
}
}
return list;
}
/**
* Takes an API object and returns a YAML String representing that object.
*
* @param object The API object to dump.
* @return A YAML String representing the API object.
*/
public static String dump(Object object) {
return getSnakeYaml(object.getClass()).dump(object);
}
/**
* Takes an API object and writes a YAML string representing that object to the writer.
*
* @param object The API object to dump
* @param writer The writer to write the YAML to.
*/
public static void dump(Object object, Writer writer) {
getSnakeYaml(object.getClass()).dump(object, writer);
}
/**
* Takes an Iterator of YAML API objects and returns a YAML string representing all of them
*
* @param data The list of YAML API objects
* @return A String representing the list of YAML API objects.
*/
public static String dumpAll(Iterator<? extends KubernetesType> data) {
return getSnakeYaml(null).dumpAll(data);
}
/**
* Takes an Iterator of YAML API objects and writes a YAML String representing all of them.
*
* @param data The list of YAML API objects.
* @param output The writer to output the YAML String to.
*/
public static void dumpAll(Iterator<? extends KubernetesType> data, Writer output) {
getSnakeYaml(null).dumpAll(data, output);
}
/** Defines constructor logic for custom types in this library. */
public static class CustomConstructor extends Constructor {
public CustomConstructor(Class<?> type, LoaderOptions loaderConfig) {
super(type, loaderConfig);
}
@Override
```

```
protected Object constructObject(Node node) {
if (node.getType() == IntOrString.class) {
return constructIntOrString((ScalarNode) node);
}
if (node.getType() == byte[].class) {
return constructByteArray((ScalarNode) node);
}
if (node.getType() == OffsetDateTime.class) {
return constructDateTime((ScalarNode) node);
}
return super.constructObject(node);
}
private IntOrString constructIntOrString(ScalarNode node) {
try {
return new IntOrString(Integer.parseInt(node.getValue()));
} catch (NumberFormatException err) {
return new IntOrString(node.getValue());
}
}
private byte[] constructByteArray(ScalarNode node) {
return ByteString.decodeBase64(node.getValue()).toByteArray();
}
private Object constructDateTime(ScalarNode node) {
if (node.getValue() == null || "null".equalsIgnoreCase(node.getValue())) {
return null;
} else {
return OffsetDateTime.parse(node.getValue());
}
}
}
public static class CustomRepresenter extends Representer {
public CustomRepresenter() {
super(new DumperOptions());
this.setDefaultFlowStyle(DumperOptions.FlowStyle.BLOCK);
this.representers.put(IntOrString.class, new RepresentIntOrString());
this.representers.put(byte[].class, new RepresentByteArray());
this.representers.put(Quantity.class, new RepresentQuantity());
this.representers.put(OffsetDateTime.class, new RepresentDateTime());
}
private class RepresentDateTime implements Represent {
@Override
public Node representData(Object data) {
return CustomRepresenter.this.representData(data.toString());
}
}
private class RepresentIntOrString implements Represent {
@Override
public Node representData(Object data) {
IntOrString intOrString = (IntOrString) data;
if (intOrString.isInteger()) {
return CustomRepresenter.this.representData(intOrString.getIntValue());
} else {
return CustomRepresenter.this.representData(intOrString.getStrValue());
```

```
}
}
}
private class RepresentByteArray implements Represent {
@Override
public Node representData(Object data) {
String value = ByteString.of((byte[]) data).base64();
return representScalar(Tag.STR, value);
}
}
private class RepresentQuantity implements Represent {
@Override
public Node representData(Object data) {
Quantity quantity = (Quantity) data;
return representScalar(Tag.STR, quantity.toSuffixedString());
}
}
/**
* This returns the ordering of properties that by convention should appear at the beginning of
* a Yaml object in Kubernetes.
*/
private int getPropertyPosition(String property) {
switch (property) {
case "apiVersion":
return 0;
case "kind":
return 1;
case "metadata":
return 2;
case "spec":
return 3;
case "type":
return 4;
default:
return Integer.MAX_VALUE;
}
}
@Override
protected MappingNode representJavaBean(Set<Property> properties, Object javaBean) {
MappingNode node = super.representJavaBean(properties, javaBean);
// Always set the tag to MAP so that SnakeYaml doesn't print out the class name as a
// tag.
node.setTag(Tag.MAP);
// Sort the output of our map so that we put certain keys, such as apiVersion, first.
Collections.sort(
node.getValue(),
new Comparator<NodeTuple>() {
@Override
public int compare(NodeTuple a, NodeTuple b) {
String nameA = ((ScalarNode) a.getKeyNode()).getValue();
String nameB = ((ScalarNode) b.getKeyNode()).getValue();
int intCompare =
Integer.compare(getPropertyPosition(nameA), getPropertyPosition(nameB));
```

```
if (intCompare != 0) {
return intCompare;
} else {
return nameA.compareTo(nameB);
}
}
});
return node;
}
@Override
protected NodeTuple representJavaBeanProperty(
Object javaBean, Property property, Object propertyValue, Tag customTag) {
// returning null for a null property value means we won't output it in the Yaml
if (propertyValue == null) {
return null;
}
if (propertyValue instanceof List<?> && ((List<?>)propertyValue).size() == 0) {
return null;
}
return super.representJavaBeanProperty(javaBean, property, propertyValue, customTag);
}
}
/**
* Instantiate a snake yaml with a default {@link SafeConstructor}.
*
* <p>DEPRECATED: Use the parameterized "getSnakeYaml" constructing method below to get rid of
* vulnerability from dynamic type serialization.
*
* @return the snake yaml
*/
@Deprecated
public static org.yaml.snakeyaml.Yaml getSnakeYaml() {
return getSnakeYaml(null);
}
/**
* Instantiate a snake yaml with the target model type specified.. Optionally, passing custom type
* descriptions if the target serialized name differs from the actual field name.
*
* @param type the target model type
* @param typeDescriptions additional type descriptions for customizing the serializer
* @return the new snake yaml instance
*/
public static org.yaml.snakeyaml.Yaml getSnakeYaml(
Class<?> type, TypeDescription... typeDescriptions) {
return getSnakeYaml(type, null, null, typeDescriptions);
}
/**
* Instantiate a snake yaml with the target model type, and the dump/load option specified..
* Optionally, passing custom type descriptions if the target serialized name differs from the
* actual field name.
*
* @param type the target model type
* @param dumperOptions the dumper options
```

```
* @param loaderOptions the loader options
* @param typeDescriptions additional type descriptions for customizing the serializer
* @return the new snake yaml instance
*/
public static org.yaml.snakeyaml.Yaml getSnakeYaml(
Class<?> type,
DumperOptions dumperOptions,
LoaderOptions loaderOptions,
TypeDescription... typeDescriptions) {
if (loaderOptions == null) {
loaderOptions = new LoaderOptions();
}
loaderOptions.setEnumCaseSensitive(false);
BaseConstructor constructor = new SafeConstructor(loaderOptions);
Representer representer = new CustomRepresenter();
if (type != null) {
constructor = new CustomConstructor(type, loaderOptions);
}
// register builtin type descriptions
registerBuiltinGsonCompatibles(constructor, representer);
for (TypeDescription desc : typeDescriptions) {
registerCustomTypeDescriptions(constructor, representer, desc);
}
if (dumperOptions == null) {
dumperOptions = new DumperOptions();
dumperOptions.setDefaultFlowStyle(representer.getDefaultFlowStyle());
dumperOptions.setDefaultScalarStyle(representer.getDefaultScalarStyle());
dumperOptions.setAllowReadOnlyProperties(
representer.getPropertyUtils().isAllowReadOnlyProperties());
dumperOptions.setTimeZone(representer.getTimeZone());
}
return new org.yaml.snakeyaml.Yaml(constructor, representer, dumperOptions, loaderOptions);
}
/**
* Instantiate a new {@link TypeDescription} which will load the {@link SerializedName} via
* reflection so that yaml serialization can work for the custom gson serialized name.
*
* @param modelClass the kubenretes api model class
* @param gsonTaggedFields the custom serialized names tagged by gson
* @return the type description
*/
public static TypeDescription newGsonCompatibleTypeDescription(
Class modelClass, String... gsonTaggedFields) {
TypeDescription desc = new TypeDescription(modelClass);
List<String> excluding = new ArrayList<>();
for (String targetGsonAnnotation : gsonTaggedFields) {
Field field =
Arrays.stream(modelClass.getDeclaredFields())
.filter(f -> f.getAnnotation(SerializedName.class) != null)
.filter(
f -> targetGsonAnnotation.equals(f.getAnnotation(SerializedName.class).value()))
.findAny()
.orElseThrow(
```

```
() ->
new IllegalArgumentException(
"Api model class "
+ modelClass.getSimpleName()
+ " doesn't have field with Gson @SerializedName with value "
+ targetGsonAnnotation));
Method getterMethod =
tryFindGetterMethod(modelClass, field)
.orElseThrow(
() ->
new IllegalArgumentException(
"Cannot find getter method for "
+ targetGsonAnnotation
+ " on api model class "
+ modelClass.getSimpleName()));
Method setterMethod =
tryFindSetterMethod(modelClass, field)
.orElseThrow(
() ->
new IllegalArgumentException(
"Cannot find setter method for "
+ targetGsonAnnotation
+ " on api model class "
+ modelClass.getSimpleName()));
desc.substituteProperty(
targetGsonAnnotation, field.getType(), getterMethod.getName(), setterMethod.getName());
excluding.add(field.getName());
}
desc.setExcludes(excluding.toArray(new String[0]));
return desc;
}
private static void registerBuiltinGsonCompatibles(
BaseConstructor constructor, Representer representer) {
// TODO: Are there more builtin api classes need these explicit substitution below
String[] crdOpenApiExtensions =
new String[] {
"x-kubernetes-embedded-resource",
"x-kubernetes-int-or-string",
"x-kubernetes-list-map-keys",
"x-kubernetes-list-type",
"x-kubernetes-map-type",
"x-kubernetes-preserve-unknown-fields",
};
registerCustomTypeDescriptions(
constructor,
representer,
newGsonCompatibleTypeDescription(V1JSONSchemaProps.class, crdOpenApiExtensions));
}
private static void registerCustomTypeDescriptions(
BaseConstructor constructor,
Representer representer,
TypeDescription... addtionalTypeDescriptions) {
Arrays.stream(addtionalTypeDescriptions)
```

```
.forEach(
desc -> {
constructor.addTypeDescription(desc);
representer.addTypeDescription(desc);
});
}
private static Optional<Method> tryFindGetterMethod(Class modelClass, Field targetField) {
return Arrays.stream(modelClass.getDeclaredMethods())
.filter(f -> f.getName().startsWith("get"))
.filter(f -> f.getName().equalsIgnoreCase("get" + targetField.getName()))
.findAny();
}
private static Optional<Method> tryFindSetterMethod(Class modelClass, Field targetField) {
return Arrays.stream(modelClass.getDeclaredMethods())
.filter(f -> f.getName().startsWith("set"))
.filter(f -> f.getName().equalsIgnoreCase("set" + targetField.getName()))
.findAny();
}
/**
* @param data Map that will be converted to concrete API object.
* @return An instantiation of the object.
* @throws IOException
*/
private static Object modelMapper(Map<String, Object> data) throws IOException {
String kind = (String) data.get("kind");
if (kind == null) {
throw new IOException("Missing kind in YAML file!");
}
String apiVersion = (String) data.get("apiVersion");
if (apiVersion == null) {
throw new IOException("Missing apiVersion in YAML file!");
}
Class<?> clazz = ModelMapper.getApiTypeClass(apiVersion, kind);
if (clazz == null) {
throw new IOException(
"Unknown apiVersionKind " + apiVersion + "/" + kind + " is it registered?");
}
return loadAs(new StringReader(getSnakeYaml(clazz).dump(data)), clazz);
}
@Deprecated
public static void addModelMap(String apiGroupVersion, String kind, Class<?> clazz) {
ModelMapper.addModelMap(apiGroupVersion, kind, clazz);
}
}
```

io/kubernetes/client/util/ApiInvocation.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
```

You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.openapi.ApiException;
public interface ApiInvocation<ApiType> {
ApiType call() throws ApiException;
}

io/kubernetes/client/util/ClientBuilder.java

/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import static io.kubernetes.client.util.Config.ENV_KUBECONFIG;
import static io.kubernetes.client.util.Config.ENV_SERVICE_HOST;
import static io.kubernetes.client.util.Config.ENV_SERVICE_PORT;
import static io.kubernetes.client.util.Config.SERVICEACCOUNT_CA_PATH;
import static io.kubernetes.client.util.Config.SERVICEACCOUNT_TOKEN_PATH;
import static io.kubernetes.client.util.KubeConfig.ENV_HOME;
import static io.kubernetes.client.util.KubeConfig.KUBECONFIG;
import static io.kubernetes.client.util.KubeConfig.KUBEDIR;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.models.V1CertificateSigningRequest;
import io.kubernetes.client.persister.FilePersister;
import io.kubernetes.client.util.credentials.AccessTokenAuthentication;
import io.kubernetes.client.util.credentials.Authentication;
import io.kubernetes.client.util.credentials.ClientCertificateAuthentication;
import io.kubernetes.client.util.credentials.KubeconfigAuthentication;
import io.kubernetes.client.util.credentials.TokenFileAuthentication;
import io.kubernetes.client.util.exception.CSRNotApprovedException;
import java.io.BufferedReader;
import java.io.ByteArrayInputStream;
import java.io.File;
import java.io.FileInputStream;

```
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.Reader;
import java.net.URI;
import java.net.URISyntaxException;
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.security.PrivateKey;
import java.time.Duration;
import java.util.Arrays;
import java.util.List;
import okhttp3.Protocol;
import org.apache.commons.compress.utils.IOUtils;
import org.apache.commons.lang3.StringUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/** A Builder which allows the construction of {@link ApiClient}s in a fluent fashion. */
public class ClientBuilder {
private static final Logger log = LoggerFactory.getLogger(ClientBuilder.class);
private String basePath = Config.DEFAULT_FALLBACK_HOST;
private byte[] caCertBytes = null;
private boolean verifyingSsl = true;
private Authentication authentication;
private String keyStorePassphrase;
// defaulting client protocols to HTTP1.1 and HTTP 2
private List<Protocol> protocols = Arrays.asList(Protocol.HTTP_2, Protocol.HTTP_1_1);
// default to unlimited read timeout
private Duration readTimeout = Duration.ZERO;
// default health check is once a minute
private Duration pingInterval = Duration.ofMinutes(1);
// time to refresh exec based credentials
// TODO: Read the expiration from the credential itself
private Duration execCredentialRefreshPeriod = null;
/**
* Creates an {@link ApiClient} by calling {@link #standard()} and {@link #build()}.
*
* @return An <tt>ApiClient</tt> configured using the precedence specified for {@link
* #standard()}.
* @throws IOException if the configuration file or a file specified in a configuration file
* cannot be read.
*/
public static ApiClient defaultClient() throws IOException {
return ClientBuilder.standard().build();
}
/**
* Creates a builder which is pre-configured in the following way
*
* <ul>
* <li>If $KUBECONFIG is defined, use that config file.
* <li>If $HOME/.kube/config can be found, use that.
```

```
* <li>If the in-cluster service account can be found, assume in cluster config.
* <li>Default to localhost:8080 as a last resort.
* </ul>
*
* @return <tt>ClientBuilder</tt> pre-configured using the above precedence
* @throws IOException if the configuration file or a file specified in a configuration file
* cannot be read.
*/
public static ClientBuilder standard() throws IOException {
return standard(true);
}
public static ClientBuilder standard(boolean persistConfig) throws IOException {
final File kubeConfig = findConfigFromEnv();
ClientBuilder clientBuilderEnv = getClientBuilder(persistConfig, kubeConfig);
if (clientBuilderEnv != null) return clientBuilderEnv;
final File config = findConfigInHomeDir();
ClientBuilder clientBuilderHomeDir = getClientBuilder(persistConfig, config);
if (clientBuilderHomeDir != null) return clientBuilderHomeDir;
final File clusterCa = new File(SERVICEACCOUNT_CA_PATH);
if (clusterCa.exists()) {
return cluster();
}
return new ClientBuilder();
}
private static ClientBuilder getClientBuilder(boolean persistConfig, File kubeConfig)
throws IOException {
if (kubeConfig != null) {
try (BufferedReader kubeConfigReader =
new BufferedReader(
new InputStreamReader(
new FileInputStream(kubeConfig), StandardCharsets.UTF_8.name()))) {
KubeConfig kc = KubeConfig.loadKubeConfig(kubeConfigReader);
if (persistConfig) {
kc.setPersistConfig(new FilePersister(kubeConfig));
}
kc.setFile(kubeConfig);
return kubeconfig(kc);
}
}
return null;
}
private static File findConfigFromEnv() {
final KubeConfigEnvParser kubeConfigEnvParser = new KubeConfigEnvParser();
final String kubeConfigPath =
kubeConfigEnvParser.parseKubeConfigPath(System.getenv(ENV_KUBECONFIG));
if (kubeConfigPath == null) {
return null;
}
final File kubeConfig = new File(kubeConfigPath);
if (kubeConfig.exists()) {
return kubeConfig;
} else {
log.debug("Could not find file specified in $KUBECONFIG");
```

```
return null;
}
}
private static class KubeConfigEnvParser {
private String parseKubeConfigPath(String kubeConfigEnv) {
if (kubeConfigEnv == null) {
return null;
}
final String[] filePaths = kubeConfigEnv.split(File.pathSeparator);
final String kubeConfigPath = filePaths[0];
if (filePaths.length > 1) {
log.warn("Found multiple kubeconfigs files, $KUBECONFIG: {} using first: {}", kubeConfigEnv,
kubeConfigPath);
}
return kubeConfigPath;
}
}
private static File findHomeDir() {
final String envHome = System.getenv(ENV_HOME);
if (envHome != null && envHome.length() > 0) {
final File config = new File(envHome);
if (config.exists()) {
return config;
}
}
if (System.getProperty("os.name").toLowerCase().startsWith("windows")) {
String homeDrive = System.getenv("HOMEDRIVE");
String homePath = System.getenv("HOMEPATH");
if (homeDrive != null
&& homeDrive.length() > 0
&& homePath != null
&& homePath.length() > 0) {
File homeDir = new File(new File(homeDrive), homePath);
if (homeDir.exists()) {
return homeDir;
}
}
String userProfile = System.getenv("USERPROFILE");
if (userProfile != null && userProfile.length() > 0) {
File profileDir = new File(userProfile);
if (profileDir.exists()) {
return profileDir;
}
}
}
return null;
}
private static File findConfigInHomeDir() {
final File homeDir = findHomeDir();
if (homeDir != null) {
final File config = new File(new File(homeDir, KUBEDIR), KUBECONFIG);
if (config.exists()) {
return config;
```

```
}
}
log.debug("Could not find ~/.kube/config");
return null;
}
/**
* [DEPRECATED] Creates a builder which is pre-configured from the cluster configuration.
*
* @return <tt>ClientBuilder</tt> configured from the cluster configuration.
* @throws IOException if the Service Account Token Path or CA Path is not readable.
*/
public static ClientBuilder oldCluster() throws IOException {
final ClientBuilder builder = new ClientBuilder();
final String host = System.getenv(ENV_SERVICE_HOST);
final String port = System.getenv(ENV_SERVICE_PORT);
builder.setBasePath(host, port);
final String token =
new String(
Files.readAllBytes(Paths.get(SERVICEACCOUNT_TOKEN_PATH)), Charset.defaultCharset());
builder.setCertificateAuthority(Files.readAllBytes(Paths.get(SERVICEACCOUNT_CA_PATH)));
builder.setAuthentication(new AccessTokenAuthentication(token));
return builder;
}
/**
* Creates a builder which is pre-configured from the cluster configuration.
*
* @return <tt>ClientBuilder</tt> configured from the cluster configuration where service account
* token will be reloaded.
* @throws IOException if the Service Account Token Path or CA Path is not readable.
*/
public static ClientBuilder cluster() throws IOException {
final ClientBuilder builder = new ClientBuilder();
final String host = System.getenv(ENV_SERVICE_HOST);
final String port = System.getenv(ENV_SERVICE_PORT);
builder.setBasePath(host, port);
builder.setCertificateAuthority(Files.readAllBytes(Paths.get(SERVICEACCOUNT_CA_PATH)));
builder.setAuthentication(new TokenFileAuthentication(SERVICEACCOUNT_TOKEN_PATH));
return builder;
}
protected ClientBuilder setBasePath(String host, String port) {
try {
Integer iPort = Integer.valueOf(port);
URI uri = new URI("https", null, host, iPort, null, null, null);
this.setBasePath(uri.toString());
} catch (NumberFormatException | URISyntaxException e) {
throw new IllegalStateException(e);
}
return this;
}
/**
* Creates a builder which is pre-configured from a {@link KubeConfig}.
*
* <p>To load a <tt>KubeConfig</tt>, see {@link KubeConfig#loadKubeConfig(Reader)}.
```

```
*
* @param config The {@link KubeConfig} to configure the builder from.
* @return <tt>ClientBuilder</tt> configured from the provided <tt>KubeConfig</tt>
* @throws IOException if the files specified in the provided <tt>KubeConfig</tt> are not readable
*/
public static ClientBuilder kubeconfig(KubeConfig config) throws IOException {
return kubeconfig(config, null);
}
/**
* Creates a builder which is pre-configured from a {@link KubeConfig}.
*
* <p>To load a <tt>KubeConfig</tt>, see {@link KubeConfig#loadKubeConfig(Reader)}.
*
* @param config The {@link KubeConfig} to configure the builder from.
* @param tokenRefreshPeriod If the KubeConfig generates a bearer token, after this interval, it will be
refreshed.
* @return <tt>ClientBuilder</tt> configured from the provided <tt>KubeConfig</tt>
* @throws IOException if the files specified in the provided <tt>KubeConfig</tt> are not readable
*/
public static ClientBuilder kubeconfig(KubeConfig config, Duration tokenRefreshPeriod) throws
IOException {
final ClientBuilder builder = new ClientBuilder();
String server = config.getServer();
if (server == null) {
throw new IllegalArgumentException("No server in kubeconfig");
}
if (!server.contains("://")) {
if (server.contains(":443")) {
server = "https://" + server;
} else {
server = "http://" + server;
}
}
final byte[] caBytes =
config.getDataOrFileRelative(
config.getCertificateAuthorityData(), config.getCertificateAuthorityFile());
if (caBytes != null) {
builder.setCertificateAuthority(caBytes);
}
builder.setVerifyingSsl(config.verifySSL());
builder.setBasePath(server);
builder.setAuthentication(new KubeconfigAuthentication(config, tokenRefreshPeriod));
return builder;
}
/**
* Returns a new ApiClient instance reading from CertificateSigningRequest.
*
* <p>It will create a CertificateSigningRequest object to the cluster if it doesn't exist, and
* waits until the request is approved.
*
* @param bootstrapKubeConfig the bootstrap kube config
* @param privateKey the private key
* @param csr the csr
```

```
* @return the api client
* @throws IOException the io exception
* @throws CSRNotApprovedException the csr not approved exception
* @throws ApiException the api exception
*/
public static ApiClient fromCertificateSigningRequest(
KubeConfig bootstrapKubeConfig, PrivateKey privateKey, V1CertificateSigningRequest csr)
throws IOException, CSRNotApprovedException, ApiException {
// creates CSR or checks whether the existing one conflicts.
ApiClient bootstrapApiClient = ClientBuilder.kubeconfig(bootstrapKubeConfig).build();
return fromCertificateSigningRequest(bootstrapApiClient, privateKey, csr);
}
/**
* Returns a new ApiClient instance reading from CertificateSigningRequest.
*
* <p>It will create a CertificateSigningRequest object to the cluster if it doesn't exist, and
* waits until the request is approved.
*
* @param bootstrapApiClient the bootstrap api client
* @param privateKey the private key
* @param csr the csr
* @return the api client
* @throws IOException the io exception
* @throws CSRNotApprovedException the csr not approved exception
* @throws ApiException the api exception
*/
public static ApiClient fromCertificateSigningRequest(
ApiClient bootstrapApiClient, PrivateKey privateKey, V1CertificateSigningRequest csr)
throws IOException, CSRNotApprovedException, ApiException {
byte[] certificateData = CSRUtils.createAndWaitUntilCertificateSigned(bootstrapApiClient, csr);
InputStream is = bootstrapApiClient.getSslCaCert();
is.reset();
ClientBuilder newBuilder = new ClientBuilder();
newBuilder.setAuthentication(
new ClientCertificateAuthentication(certificateData, SSLUtils.dumpKey(privateKey)));
newBuilder.setBasePath(bootstrapApiClient.getBasePath());
newBuilder.setVerifyingSsl(bootstrapApiClient.isVerifyingSsl());
newBuilder.setCertificateAuthority(IOUtils.toByteArray(is));
return newBuilder.build();
}
public String getBasePath() {
return basePath;
}
public ClientBuilder setBasePath(String basePath) {
this.basePath = basePath;
return this;
}
public Authentication getAuthentication() {
return authentication;
}
public ClientBuilder setAuthentication(final Authentication authentication) {
this.authentication = authentication;
return this;
```

```
}
public ClientBuilder setCertificateAuthority(final byte[] caCertBytes) {
this.caCertBytes = caCertBytes;
return this;
}
public boolean isVerifyingSsl() {
return verifyingSsl;
}
public ClientBuilder setVerifyingSsl(boolean verifyingSsl) {
this.verifyingSsl = verifyingSsl;
return this;
}
public ClientBuilder setProtocols(List<Protocol> protocols) {
this.protocols = protocols;
return this;
}
public List<Protocol> getProtocols() {
return protocols;
}
public ClientBuilder setReadTimeout(Duration readTimeout) {
this.readTimeout = readTimeout;
return this;
}
public Duration getReadTimeout() {
return this.readTimeout;
}
public ClientBuilder setPingInterval(Duration pingInterval) {
this.pingInterval = pingInterval;
return this;
}
public Duration getPingInterval() {
return this.pingInterval;
}
public String getKeyStorePassphrase() {
return keyStorePassphrase;
}
public ClientBuilder setKeyStorePassphrase(String keyStorePassphrase) {
this.keyStorePassphrase = keyStorePassphrase;
return this;
}
public ApiClient build() {
final ApiClient client = new ApiClient();
client.setHttpClient(
client
.getHttpClient()
.newBuilder()
.protocols(protocols)
.readTimeout(this.readTimeout)
.pingInterval(pingInterval)
.build());
if (basePath != null) {
if (basePath.endsWith("/")) {
basePath = basePath.substring(0, basePath.length() - 1);
```

```
}
client.setBasePath(basePath);
}
client.setVerifyingSsl(verifyingSsl);
if (authentication != null) {
if (StringUtils.isNotEmpty(keyStorePassphrase)) {
if (authentication instanceof KubeconfigAuthentication) {
if (((KubeconfigAuthentication) authentication).getDelegateAuthentication()
instanceof ClientCertificateAuthentication) {
((ClientCertificateAuthentication)
(((KubeconfigAuthentication) authentication).getDelegateAuthentication()))
.setPassphrase(keyStorePassphrase);
}
}
}
authentication.provide(client);
}
// NOTE: this ordering is important. The API Client re-evaluates the CA certificate every
// time the SSL info changes, which means that if this comes after the following call
// you will try to load a certificate with an exhausted InputStream. So setting the CA
// certificate _has_ to be the last thing that you do related to SSL.
//
// TODO: this (imho) is broken in the generate Java Swagger Client code. We should fix it
// upstream and remove this dependency.
//
// TODO: Add a test to ensure that this works correctly...
if (caCertBytes != null) {
client.setSslCaCert(new ByteArrayInputStream(caCertBytes));
}
return client;
}
}
```

io/kubernetes/client/util/version/Version.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.version;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.Configuration;
```

```
import io.kubernetes.client.openapi.apis.VersionApi;
import io.kubernetes.client.openapi.models.VersionInfo;
import java.io.IOException;
import okhttp3.Call;
import okhttp3.Response;
public class Version {
private final VersionApi versionApi;
public Version() {
this(Configuration.getDefaultApiClient());
}
public Version(ApiClient apiClient) {
this.versionApi = new VersionApi(apiClient);
}
public VersionInfo getVersion() throws ApiException, IOException {
Call call = versionApi.getCode().buildCall(null);
Response response = null;
try {
response = call.execute();
} catch (IOException e) {
throw new ApiException(e);
}
if (!response.isSuccessful()) {
throw new ApiException(response.code(), "Version request failed: " + response.code());
}
return this.versionApi
.getApiClient()
.getJSON()
.deserialize(response.body().string(), VersionInfo.class);
}
}
```

io/kubernetes/client/util/authenticators/OpenIDConnectAuthenticator.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.authenticators;
import io.kubernetes.client.util.KubeConfig;
import java.io.ByteArrayInputStream;
import java.io.IOException;
```

```
import java.io.OutputStream;
import java.net.URL;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.security.KeyManagementException;
import java.security.KeyStore;
import java.security.KeyStoreException;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.security.cert.CertificateException;
import java.security.cert.CertificateFactory;
import java.util.Base64;
import java.util.Collection;
import java.util.Map;
import java.util.Scanner;
import javax.net.ssl.HttpsURLConnection;
import javax.net.ssl.SSLContext;
import javax.net.ssl.TrustManagerFactory;
import org.jose4j.json.internal.json_simple.JSONObject;
import org.jose4j.json.internal.json_simple.parser.JSONParser;
import org.jose4j.json.internal.json_simple.parser.ParseException;
import org.jose4j.jws.JsonWebSignature;
import org.jose4j.jwt.JwtClaims;
import org.jose4j.jwt.MalformedClaimException;
import org.jose4j.jwt.NumericDate;
import org.jose4j.jwt.consumer.InvalidJwtException;
import org.jose4j.lang.JoseException;
/** OpenIDConnectAuthenticator */
public class OpenIDConnectAuthenticator implements Authenticator {
/** The id_token */
public static final String OIDC_ID_TOKEN = "id-token";
/** The issuer */
public static final String OIDC_ISSUER = "idp-issuer-url";
/** The refresh_token */
public static final String OIDC_REFRESH_TOKEN = "refresh-token";
/** The client_id */
public static final String OIDC_CLIENT_ID = "client-id";
/** Optional client secret */
public static final String OIDC_CLIENT_SECRET = "client-secret";
/** Optional IdP TLS Certificate */
public static final String OIDC_IDP_CERT_DATA = "idp-certificate-authority-data";
static {
KubeConfig.registerAuthenticator(new OpenIDConnectAuthenticator());
}
@Override
public String getName() {
return "oidc";
}
@Override
public String getToken(Map<String, Object> config) {
return (String) config.get(OIDC_ID_TOKEN);
}
@Override
```

```
public boolean isExpired(Map<String, Object> config) {
String idToken = (String) config.get(OIDC_ID_TOKEN);
if (idToken == null) {
return true;
} else {
JsonWebSignature jws = new JsonWebSignature();
try {
jws.setCompactSerialization(idToken);
// we don't care if its valid or not cryptographicly as the only way to verify is to
// query
// the remote identity provider's configuration url which is the same chanel as the
// token
// request. If there is a malicious proxy there's no way for the client to know.
// Also,
// the client doesn't need to trust the, token, only bear it to the server which
// will verify
// it.
String jwt = jws.getUnverifiedPayload();
JwtClaims claims = JwtClaims.parse(jwt);
// expired now is >= expiration AND exp is present
return claims.getExpirationTime() == null
|| NumericDate.now().isOnOrAfter(claims.getExpirationTime());
} catch (JoseException | InvalidJwtException | MalformedClaimException e) {
throw new RuntimeException(e);
}
}
}
@Override
public Map<String, Object> refresh(Map<String, Object> config) {
String issuer = (String) config.get(OIDC_ISSUER);
String clientId = (String) config.get(OIDC_CLIENT_ID);
String refreshToken = (String) config.get(OIDC_REFRESH_TOKEN);
String clientSecret = (String) config.getOrDefault(OIDC_CLIENT_SECRET, "");
String idpCert = (String) config.get(OIDC_IDP_CERT_DATA);
SSLContext sslContext = null;
if (idpCert != null) {
// fist, lets get the pem
String pemCert = new String(Base64.getDecoder().decode(idpCert));
// next lets get a cert object
// need an alias name to store the certificate in a keystore. Also
// java keystores need passwords. this value is as good as any as
// there isn't anything actually secret being stored.
String alias = "doenotmatter";
KeyStore ks;
try {
ks = java.security.KeyStore.getInstance("PKCS12");
ks.load(null, alias.toCharArray());
ByteArrayInputStream bais =
new ByteArrayInputStream(pemCert.getBytes(StandardCharsets.UTF_8));
CertificateFactory cf = CertificateFactory.getInstance("X.509");
Collection<? extends java.security.cert.Certificate> c = cf.generateCertificates(bais);
int j = 0;
for (java.security.cert.Certificate certificate : c) {
```

```
ks.setCertificateEntry(alias + "-" + j, certificate);
j++;
}
TrustManagerFactory tmf = TrustManagerFactory.getInstance("PKIX");
tmf.init(ks);
// TODO would be good to make this more dyanamic. Doesn't seem like
// a good way to do this.
sslContext = SSLContext.getInstance("TLSv1.2");
sslContext.init(null, tmf.getTrustManagers(), new SecureRandom());
} catch (KeyStoreException
| NoSuchAlgorithmException
| CertificateException
| IOException
| KeyManagementException e) {
throw new RuntimeException("Could not import idp certificate", e);
}
}
// check the identity provider's configuration url for a token endpoint
String tokenURL = loadTokenURL(issuer, sslContext);
// get the refreshed tokens
JSONObject response =
refreshOidcToken(clientId, refreshToken, clientSecret, sslContext, tokenURL);
// reload the config
config.put(OIDC_ID_TOKEN, response.get("id_token"));
config.put(OIDC_REFRESH_TOKEN, response.get("refresh_token"));
return config;
}
/**
* Refreshes the OpenID Connect id_token
*
* @param clientId from client-id
* @param refreshToken from refresh-token
* @param clientSecret from client-secret
* @param sslContext to support TLS with a self signed certificate in
* idp-certificate-authority-data
* @param tokenURL the url for refreshing the token
* @return
*/
private JSONObject refreshOidcToken(
String clientId,
String refreshToken,
String clientSecret,
SSLContext sslContext,
String tokenURL) {
try {
URL tokenEndpoint = new URL(tokenURL);
HttpsURLConnection https = (HttpsURLConnection) tokenEndpoint.openConnection();
https.setRequestMethod("POST");
if (sslContext != null) {
https.setSSLSocketFactory(sslContext.getSocketFactory());
}
// per https://tools.ietf.org/html/rfc6749#section-2.3 the secret should be a header,
// not in
```

```
// the body
String credentials =
Base64.getEncoder()
.encodeToString(
new StringBuilder()
.append(clientId)
.append(':')
.append(clientSecret)
.toString()
.getBytes(StandardCharsets.UTF_8));
https.setRequestProperty(
"Authorization", new StringBuilder().append("Basic ").append(credentials).toString());
https.setRequestProperty("Content-Type", "application/x-www-form-urlencoded");
https.setDoOutput(true);
String urlData =
new StringBuilder()
.append("refresh_token=")
.append(URLEncoder.encode(refreshToken, "UTF-8"))
.append("&grant_type=refresh_token")
.toString();
OutputStream ou = https.getOutputStream();
ou.write(urlData.getBytes(StandardCharsets.UTF_8));
ou.flush();
ou.close();
int code = https.getResponseCode();
if (code != HttpsURLConnection.HTTP_OK) {
throw new RuntimeException(
new StringBuilder()
.append("Invalid response code for token retrieval - ")
.append(code)
.toString());
}
Scanner scanner = new Scanner(https.getInputStream(), StandardCharsets.UTF_8.name());
String json = scanner.useDelimiter("\\A").next();
return (JSONObject) new JSONParser().parse(json);
} catch (Throwable t) {
throw new RuntimeException("Could not refresh token", t);
}
}
/**
* Determines the token url
*
* @param issuer from the idp-issuer-url
* @param sslContext to support TLS with a self signed certificate in
* idp-certificate-authority-data
* @return
*/
private String loadTokenURL(String issuer, SSLContext sslContext) {
StringBuilder wellKnownUrl = new StringBuilder();
wellKnownUrl.append(issuer);
if (!issuer.endsWith("/")) {
wellKnownUrl.append("/");
}
```

```
wellKnownUrl.append(".well-known/openid-configuration");
try {
URL wellKnown = new URL(wellKnownUrl.toString());
HttpsURLConnection https = (HttpsURLConnection) wellKnown.openConnection();
https.setRequestMethod("GET");
if (sslContext != null) {
https.setSSLSocketFactory(sslContext.getSocketFactory());
}
https.setUseCaches(false);
int code = https.getResponseCode();
if (code != HttpsURLConnection.HTTP_OK) {
throw new RuntimeException(
new StringBuilder()
.append("Invalid response code for issuer - ")
.append(code)
.toString());
}
Scanner scanner = new Scanner(https.getInputStream(), StandardCharsets.UTF_8.name());
String json = scanner.useDelimiter("\\A").next();
JSONObject wellKnownJson = (JSONObject) new JSONParser().parse(json);
return (String) wellKnownJson.get("token_endpoint");
} catch (IOException | ParseException e) {
throw new RuntimeException("Could not refresh", e);
}
}
}
```

io/kubernetes/client/util/authenticators/GCPAuthenticator.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.authenticators;
import com.google.auth.oauth2.AccessToken;
import com.google.auth.oauth2.GoogleCredentials;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import io.kubernetes.client.util.KubeConfig;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
```

```
import java.time.Instant;
import java.util.Arrays;
import java.util.Date;
import java.util.List;
import java.util.Map;
import java.util.concurrent.TimeUnit;
import org.apache.commons.io.IOUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/**
* The Authenticator interface represents a plugin that can handle a specific type of authentication
* information (e.g. 'gcp')
*/
public class GCPAuthenticator implements Authenticator {
static {
KubeConfig.registerAuthenticator(new GCPAuthenticator());
}
static final String ACCESS_TOKEN = "access-token";
static final String EXPIRY = "expiry";
static final String CMD_ARGS = "cmd-args";
static final String CMD_PATH = "cmd-path";
static final String SCOPES = "scopes";
static final String[] DEFAULT_SCOPES =
new String[] {
"https://www.googleapis.com/auth/cloud-platform",
"https://www.googleapis.com/auth/userinfo.email"
};
private static final Logger log = LoggerFactory.getLogger(GCPAuthenticator.class);
private final ProcessBuilder pb;
private GoogleCredentials gc;
public GCPAuthenticator() {
this(new ProcessBuilder(), null);
}
public GCPAuthenticator(ProcessBuilder pb, GoogleCredentials gc) {
this.pb = pb;
this.gc = gc;
}
@Override
public String getName() {
return "gcp";
}
@Override
public String getToken(Map<String, Object> config) {
return (String) config.get(ACCESS_TOKEN);
}
@Override
public boolean isExpired(Map<String, Object> config) {
Object expiryObj = config.get(EXPIRY);
Instant expiry = null;
if (expiryObj instanceof Date) {
expiry = ((Date) expiryObj).toInstant();
} else if (expiryObj instanceof Instant) {
expiry = (Instant) expiryObj;
```

```
} else if (expiryObj instanceof String) {
expiry = Instant.parse((String) expiryObj);
} else {
throw new RuntimeException("Unexpected object type: " + expiryObj.getClass());
}
return (expiry != null && expiry.compareTo(Instant.now()) <= 0);
}
@Override
public Map<String, Object> refresh(Map<String, Object> config) {
if (isCmd(config)) {
return refreshCmd(config);
}
// Google Application Credentials-based refresh
// https://cloud.google.com/kubernetes-engine/docs/how-to/api-server-authentication#environments-wit
hout-gcloud
String[] scopes = parseScopes(config);
try {
if (this.gc == null) this.gc = GoogleCredentials.getApplicationDefault().createScoped(scopes);
gc.refreshIfExpired();
AccessToken accessToken = gc.getAccessToken();
config.put(ACCESS_TOKEN, accessToken.getTokenValue());
config.put(EXPIRY, accessToken.getExpirationTime());
return config;
} catch (IOException e) {
throw new RuntimeException("The Application Default Credentials are not available.", e);
}
}
public String[] parseScopes(Map<String, Object> config) {
String scopes = (String) config.get(SCOPES);
if (scopes == null) {
return DEFAULT_SCOPES;
}
if (scopes.isEmpty()) {
return new String[] {};
}
return scopes.split(",");
}
private boolean isCmd(Map<String, Object> config) {
return config.containsKey(CMD_ARGS) && config.containsKey(CMD_PATH);
}
private Map<String, Object> refreshCmd(Map<String, Object> config) {
String cmdPath = (String) config.get(CMD_PATH);
String cmdArgs = (String) config.get(CMD_ARGS);
List<String> fullCmd =
Arrays.asList(String.join(" ", cmdPath.trim(), cmdArgs.trim()).split(" "));
try {
Process process = this.pb.command(fullCmd).start();
process.waitFor(10, TimeUnit.SECONDS);
if (process.exitValue() != 0) {
String stdErr = IOUtils.toString(process.getErrorStream(), StandardCharsets.UTF_8);
throw new IllegalStateException(
"Failed to executing access token command "
+ fullCmd
```

```
+ ": exitValue: "
+ process.exitValue()
+ ", stdErr: "
+ stdErr);
}
String output = IOUtils.toString(process.getInputStream(), StandardCharsets.UTF_8);
String credentialJson = output.substring(output.indexOf("{"), output.lastIndexOf("}") + 1);
JsonObject root = JsonParser.parseString(credentialJson).getAsJsonObject();
if (root.has("credential")) {
JsonObject credJson = root.getAsJsonObject("credential");
if (credJson.has("access_token") && credJson.has("token_expiry")) {
config.put(ACCESS_TOKEN, credJson.get("access_token").getAsString());
config.put(EXPIRY, credJson.get("token_expiry").getAsString());
return config;
}
}
throw new IllegalStateException("Failed to parsing access token output: " + output);
} catch (IOException | InterruptedException e) {
throw new RuntimeException("Could not refresh token", e);
}
}
}
```

io/kubernetes/client/util/authenticators/Authenticator.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.authenticators;
import java.util.Map;
/**
* The Authenticator interface represents a plugin that can handle a specific type of authentication
* information (e.g. 'gcp')
*/
public interface Authenticator {
/**
* Return the name of this authenticator, this should be the value that is also in a kubeconfig
* file.
*/
String getName();
```

```
/**
* Get a token from this authenticator.
*
* @param config The configuration information for this authenticator
* @return The new token, null of no such token can be found/generated
*/
String getToken(Map<String, Object> config);
/** Determine if this config is expired */
boolean isExpired(Map<String, Object> config);
/** Refresh an expired token with a new fresh one. */
Map<String, Object> refresh(Map<String, Object> config);
}
```

io/kubernetes/client/util/WebSocketStreamHandler.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import java.io.ByteArrayInputStream;
import java.io.Closeable;
import java.io.IOException;
import java.io.InputStream;
import java.io.InterruptedIOException;
import java.io.OutputStream;
import java.io.PipedInputStream;
import java.io.PipedOutputStream;
import java.io.Reader;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.HashMap;
import java.util.Map;
import okhttp3.WebSocket;
import okio.ByteString;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/**
* WebSocketStreamHandler understands the Kubernetes streaming protocol and separates a single
* WebSockets stream into a number of different streams using that protocol.
```

```
*/
public class WebSocketStreamHandler implements WebSockets.SocketListener, Closeable {
private static final Logger log = LoggerFactory.getLogger(WebSocketStreamHandler.class);
private final Map<Integer, PipedInputStream> input = new HashMap<>();
private final Map<Integer, PipedOutputStream> pipedOutput = new HashMap<>();
private final Map<Integer, OutputStream> output = new HashMap<>();
private WebSocket socket;
private Throwable error;
@SuppressWarnings("unused")
private String protocol;
private State state = State.UNINITIALIZED;
private enum State {
UNINITIALIZED,
OPEN,
CLOSED
}
public synchronized void waitForInitialized() throws InterruptedException {
if (state != State.UNINITIALIZED) {
return;
}
this.wait();
}
@Override
public synchronized void open(String protocol, WebSocket socket) {
if (state != State.UNINITIALIZED) throw new IllegalStateException();
this.protocol = protocol;
this.socket = socket;
state = State.OPEN;
notifyAll();
}
@Override
public void bytesMessage(InputStream in) {
try {
handleMessage(in.read(), in);
} catch (IOException ex) {
log.error("Error reading message channel", ex);
}
}
@Override
public void textMessage(Reader in) {
try {
handleMessage(
in.read(),
new ByteArrayInputStream(Streams.toString(in).getBytes(StandardCharsets.UTF_8)));
} catch (IOException ex) {
log.error("Error writing message", ex);
}
}
protected void handleMessage(int stream, InputStream inStream) throws IOException {
try {
OutputStream out = getSocketInputOutputStream(stream);
Streams.copy(inStream, out);
out.flush();
```

```
} finally {
inStream.close();
}
}
@Override
public void failure(Throwable t) {
this.error = t;
}
public Throwable getError() {
return this.error;
}
@Override
public synchronized void close() {
if (state != State.CLOSED) {
// Close all output streams. Caller of getInputStream(int) is responsible
// for closing returned input streams
for (PipedOutputStream out : pipedOutput.values()) {
try {
out.flush();
} catch (IOException ex) {
log.error("Error on flush", ex);
}
try {
out.close();
} catch (IOException ex) {
log.error("Error on close", ex);
}
}
for (OutputStream out : output.values()) {
try {
out.flush();
} catch (IOException ex) {
log.error("Error on flush", ex);
}
try {
out.close();
} catch (IOException ex) {
log.error("Error on close", ex);
}
}
state = State.CLOSED;
if (null != socket) {
// code 1000 means "Normal Closure"
socket.close(1000, "Triggered client-side terminate");
log.debug("Successfully closed socket.");
}
}
notifyAll();
}
/**
* Get a specific input stream using its identifier. Caller is responsible for closing these
* streams.
*
```

```
* @param stream The stream to return
* @return The specified stream.
*/
public synchronized InputStream getInputStream(int stream) {
if (state == State.CLOSED) throw new IllegalStateException();
if (!input.containsKey(stream)) {
try {
PipedInputStream pipeIn = new PipedInputStream();
PipedOutputStream pipeOut = new PipedOutputStream(pipeIn);
pipedOutput.put(stream, pipeOut);
input.put(stream, pipeIn);
} catch (IOException ex) {
// This is _very_ unlikely, as it requires the above constructor to fail.
// don't force callers to catch, but still throw
throw new IllegalStateException(ex);
}
}
return input.get(stream);
}
/**
* Gets a specific output stream using it's identified
*
* @param stream The stream to return
* @return The specified stream.
*/
public synchronized OutputStream getOutputStream(int stream) {
if (!output.containsKey(stream)) {
output.put(stream, new WebSocketOutputStream(stream));
}
return output.get(stream);
}
/**
* Get the pipe to write data to a specific InputStream. This is called when new data is read from
* the web socket, to send the data on to the right stream.
*
* @param stream The stream to return
* @return The specified stream.
*/
private synchronized OutputStream getSocketInputOutputStream(int stream) {
if (!pipedOutput.containsKey(stream)) {
try {
PipedInputStream pipeIn = new PipedInputStream();
PipedOutputStream pipeOut = new PipedOutputStream(pipeIn);
pipedOutput.put(stream, pipeOut);
input.put(stream, pipeIn);
} catch (IOException ex) {
// This is _very_ unlikely, as it requires the above constructor to fail.
// don't force callers to catch, but still throw
throw new IllegalStateException(ex);
}
}
return pipedOutput.get(stream);
}
```

```
private class WebSocketOutputStream extends OutputStream {
private static final long MAX_QUEUE_SIZE = 16L * 1024 * 1024;
private static final int MAX_WAIT_MILLIS = 10000;
private static final int WAIT_MILLIS = 10;
private final byte stream;
public WebSocketOutputStream(int stream) {
this.stream = (byte) stream;
}
@Override
public void flush() throws IOException {
if (state == State.CLOSED) {
throw new IOException("Socket is closed!");
}
int i = 0;
while (WebSocketStreamHandler.this.socket.queueSize() > 0) {
try {
Thread.sleep(100);
} catch (InterruptedException ex) {
}
// Wait a maximum of 10 seconds.
if (i++ > 100) {
throw new IOException("Timed out waiting for web-socket to flush.");
}
}
}
@Override
public void write(int b) throws IOException {
write(new byte[] {(byte) b});
}
@Override
public void write(byte[] b) throws IOException {
write(b, 0, b.length);
}
@Override
public void write(byte[] b, int offset, int length) throws IOException {
if (WebSocketStreamHandler.this.socket == null) {
synchronized (WebSocketStreamHandler.this) {
if (state == State.CLOSED) throw new IllegalStateException();
if (WebSocketStreamHandler.this.socket == null) {
// wait for the websocket to be opened
try {
WebSocketStreamHandler.this.wait();
} catch (InterruptedException e) {
throw new InterruptedIOException();
}
}
}
}
int bytesWritten = 0;
int remaining = length;
while (bytesWritten < length) {
// OkHTTP3 Web Sockets limits buffer size to 16MiB, so cap buffer at 15MiB
int bufferSize = Math.min(remaining, 15 * 1024 * 1024);
```

```
byte[] buffer = new byte[bufferSize + 1];
buffer[0] = stream;
System.arraycopy(b, offset + bytesWritten, buffer, 1, bufferSize);
ByteString byteString = ByteString.of(buffer);
final Instant start = Instant.now();
synchronized (WebSocketOutputStream.this) {
while (WebSocketStreamHandler.this.socket.queueSize() + byteString.size() > MAX_QUEUE_SIZE
&& Instant.now().isBefore(start.plus(MAX_WAIT_MILLIS, ChronoUnit.MILLIS))) {
try {
wait(WAIT_MILLIS);
} catch (InterruptedException e) {
throw new IOException("Error waiting web socket queue", e);
}
}
if (!WebSocketStreamHandler.this.socket.send(byteString)) {
throw new IOException("WebSocket has closed.");
}
notifyAll();
}
bytesWritten += bufferSize;
remaining -= bufferSize;
}
}
}
}
```

io/kubernetes/client/util/generic/options/CreateOptions.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.options;
public class CreateOptions {
private String dryRun;
private String fieldManager;
public CreateOptions dryRun(String dryRun) {
this.dryRun = dryRun;
return this;
}
public String getDryRun() {
```

```
return dryRun;
}
public void setDryRun(String dryRun) {
this.dryRun = dryRun;
}
public CreateOptions fieldManager(String fieldManager) {
this.fieldManager = fieldManager;
return this;
}
public String getFieldManager() {
return fieldManager;
}
public void setFieldManager(String fieldManager) {
this.fieldManager = fieldManager;
}
}
```

io/kubernetes/client/util/generic/options/GetOptions.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.options;
public class GetOptions {
private Boolean isPartialObjectMetadataRequest;
public GetOptions isPartialObjectMetadataRequest(Boolean isPartialObjectMetadataRequest) {
this.isPartialObjectMetadataRequest = isPartialObjectMetadataRequest;
return this;
}
public Boolean isPartialObjectMetadataRequest() {
return isPartialObjectMetadataRequest;
}
public void setPartialObjectMetadataRequest(Boolean isPartialObjectMetadataRequest) {
this.isPartialObjectMetadataRequest = isPartialObjectMetadataRequest;
}
}
```

io/kubernetes/client/util/generic/options/DeleteOptions.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.options;
import io.kubernetes.client.openapi.models.V1DeleteOptions;
public class DeleteOptions extends V1DeleteOptions {}
```

io/kubernetes/client/util/generic/options/PatchOptions.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.options;
public class PatchOptions {
private String dryRun;
private String fieldManager;
private Boolean force;
public PatchOptions dryRun(String dryRun) {
this.dryRun = dryRun;
return this;
}
public String getDryRun() {
return dryRun;
}
public void setDryRun(String dryRun) {
this.dryRun = dryRun;
}
public PatchOptions force(Boolean force) {
this.force = force;
return this;
```

```
}
public Boolean getForce() {
return force;
}
public void setForce(Boolean force) {
this.force = force;
}
public PatchOptions fieldManager(String fieldManager) {
this.fieldManager = fieldManager;
return this;
}
public String getFieldManager() {
return fieldManager;
}
public void setFieldManager(String fieldManager) {
this.fieldManager = fieldManager;
}
}
```

io/kubernetes/client/util/generic/options/ListOptions.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.options;
import com.google.gson.annotations.SerializedName;
public class ListOptions {
@SerializedName("fieldSelector")
private String fieldSelector;
@SerializedName("labelSelector")
private String labelSelector;
@SerializedName("resourceVersion")
private String resourceVersion;
@SerializedName("timeoutSeconds")
private Integer timeoutSeconds;
@SerializedName("limit")
private Integer limit;
@SerializedName("continue")
private String _continue;
@SerializedName("isPartialObjectMetadataListRequest")
```

```
private Boolean isPartialObjectMetadataListRequest;
public ListOptions fieldSelector(String fieldSelector) {
this.fieldSelector = fieldSelector;
return this;
}
public String getFieldSelector() {
return fieldSelector;
}
public void setFieldSelector(String fieldSelector) {
this.fieldSelector = fieldSelector;
}
public ListOptions labelSelector(String labelSelector) {
this.labelSelector = labelSelector;
return this;
}
public String getLabelSelector() {
return labelSelector;
}
public void setLabelSelector(String labelSelector) {
this.labelSelector = labelSelector;
}
public ListOptions resourceVersion(String resourceVersion) {
this.resourceVersion = resourceVersion;
return this;
}
public String getResourceVersion() {
return resourceVersion;
}
public void setResourceVersion(String resourceVersion) {
this.resourceVersion = resourceVersion;
}
public ListOptions limit(Integer limit) {
this.limit = limit;
return this;
}
public Integer getLimit() {
return limit;
}
public void setLimit(Integer limit) {
this.limit = limit;
}
public ListOptions _continue(String _continue) {
this._continue = _continue;
return this;
}
public String getContinue() {
return _continue;
}
public void setContinue(String _continue) {
this._continue = _continue;
}
public ListOptions timeoutSeconds(Integer timeoutSeconds) {
this.timeoutSeconds = timeoutSeconds;
```

```
return this;
}
public Integer getTimeoutSeconds() {
return timeoutSeconds;
}
public void setTimeoutSeconds(Integer timeoutSeconds) {
this.timeoutSeconds = timeoutSeconds;
}
public ListOptions isPartialObjectMetadataListRequest(
Boolean isPartialObjectMetadataListRequest) {
this.isPartialObjectMetadataListRequest = isPartialObjectMetadataListRequest;
return this;
}
public Boolean isPartialObjectMetadataListRequest() {
return isPartialObjectMetadataListRequest;
}
public void setPartialObjectMetadataListRequest(Boolean isPartialObjectMetadataListRequest) {
this.isPartialObjectMetadataListRequest = isPartialObjectMetadataListRequest;
}
}
```

io/kubernetes/client/util/generic/options/UpdateOptions.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.options;
public class UpdateOptions {
private String dryRun;
private String fieldManager;
public UpdateOptions dryRun(String dryRun) {
this.dryRun = dryRun;
return this;
}
public String getDryRun() {
return dryRun;
}
public void setDryRun(String dryRun) {
this.dryRun = dryRun;
}
```

```
public UpdateOptions fieldManager(String fieldManager) {
this.fieldManager = fieldManager;
return this;
}
public String getFieldManager() {
return fieldManager;
}
public void setFieldManager(String fieldManager) {
this.fieldManager = fieldManager;
}
}
```

io/kubernetes/client/util/generic/GenericKubernetesApi.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic;
import com.google.gson.Gson;
import com.google.gson.JsonElement;
import com.google.gson.JsonSyntaxException;
import com.google.gson.reflect.TypeToken;
import io.kubernetes.client.common.KubernetesListObject;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.common.KubernetesType;
import io.kubernetes.client.custom.V1Patch;
import io.kubernetes.client.openapi.ApiCallback;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.ApiResponse;
import io.kubernetes.client.openapi.Configuration;
import io.kubernetes.client.openapi.apis.CustomObjectsApi;
import io.kubernetes.client.openapi.models.V1ObjectMeta;
import io.kubernetes.client.openapi.models.V1Status;
import io.kubernetes.client.util.ProxyContentTypeRequestBody;
import io.kubernetes.client.util.Strings;
import io.kubernetes.client.util.Watch;
import io.kubernetes.client.util.Watchable;
import io.kubernetes.client.util.generic.options.CreateOptions;
import io.kubernetes.client.util.generic.options.DeleteOptions;
```

```
import io.kubernetes.client.util.generic.options.GetOptions;
import io.kubernetes.client.util.generic.options.ListOptions;
import io.kubernetes.client.util.generic.options.PatchOptions;
import io.kubernetes.client.util.generic.options.UpdateOptions;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.CancellationException;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicReference;
import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.UnaryOperator;
import javax.annotation.Nonnull;
import okhttp3.Call;
import okhttp3.HttpUrl;
import okhttp3.Request;
/**
* The Generic kubernetes api provides a unified client interface for not only the non-core-group
* built-in resources from kubernetes but also the custom-resources models meet the following
* requirements:
*
* <p>1. there's a `V1ObjectMeta` field in the model along with its getter/setter. 2. there's a
* `V1ListMeta` field in the list model along with its getter/setter. - supports Gson
* serialization/deserialization. 3. the generic kubernetes api covers all the basic operations over
* the custom resources including {get, list, watch, create, update, patch, delete}.
*
* <p>The Generic kubernetes api is expected to have the following behaviors upon failures where
* {@link KubernetesApiResponse#isSuccess()} returns false:
*
* <p>- For kubernetes-defined failures, the server will return a {@link V1Status} with 4xx/5xx
* code. The status object will be nested in {@link KubernetesApiResponse#getStatus()} - For the
* other unknown reason (including network, JVM..), throws an unchecked exception.
*
* @param <ApiType> the api type parameter
* @param <ApiListType> the api list type parameter
*/
public class GenericKubernetesApi<
ApiType extends KubernetesObject, ApiListType extends KubernetesListObject> {
private static final String PARTIAL_OBJECT_METADATA_HEADER =
"application/json;as=PartialObjectMetadata;g=meta.k8s.io;v=v1,application/json";
private static final String PARTIAL_OBJECT_METADATA_LIST_HEADER =
"application/json;as=PartialObjectMetadataList;g=meta.k8s.io;v=v1,application/json";
// TODO(yue9944882): supports status operations..
// TODO(yue9944882): supports generic sub-resource operations..
// TODO(yue9944882): supports delete-collections..
private final Class<ApiType> apiTypeClass;
```

```
private final Class<ApiListType> apiListTypeClass;
private final String apiGroup;
private final String apiVersion;
private final String resourcePlural;
private final CustomObjectsApi customObjectsApi;
/**
* Instantiates a new Generic kubernetes api.
*
* @param apiTypeClass the api type class
* @param apiListTypeClass the api list type class
* @param apiGroup the api group
* @param apiVersion the api version
* @param resourcePlural the resource plural
*/
public GenericKubernetesApi(
Class<ApiType> apiTypeClass,
Class<ApiListType> apiListTypeClass,
String apiGroup,
String apiVersion,
String resourcePlural) {
this(
apiTypeClass,
apiListTypeClass,
apiGroup,
apiVersion,
resourcePlural,
new CustomObjectsApi(Configuration.getDefaultApiClient()));
}
/**
* Instantiates a new Generic kubernetes api.
*
* @param apiTypeClass the api type class, e.g. V1Job.class
* @param apiListTypeClass the api list type class e.g. V1JobList.class
* @param apiGroup the api group
* @param apiVersion the api version
* @param resourcePlural the resource plural, e.g. "jobs"
* @param apiClient the api client
*/
public GenericKubernetesApi(
Class<ApiType> apiTypeClass,
Class<ApiListType> apiListTypeClass,
String apiGroup,
String apiVersion,
String resourcePlural,
ApiClient apiClient) {
this(
apiTypeClass,
apiListTypeClass,
apiGroup,
apiVersion,
resourcePlural,
new CustomObjectsApi(apiClient));
}
```

```
/**
* Instantiates a new Generic kubernetes api with the ApiClient specified.
*
* @param apiTypeClass the api type class, e.g. V1Job.class
* @param apiListTypeClass the api list type class e.g. V1JobList.class
* @param apiGroup the api group
* @param apiVersion the api version
* @param resourcePlural the resource plural, e.g. "jobs"
* @param customObjectsApi the custom objects api
*/
public GenericKubernetesApi(
Class<ApiType> apiTypeClass,
Class<ApiListType> apiListTypeClass,
String apiGroup,
String apiVersion,
String resourcePlural,
CustomObjectsApi customObjectsApi) {
this.apiGroup = apiGroup;
this.apiVersion = apiVersion;
this.resourcePlural = resourcePlural;
this.apiTypeClass = apiTypeClass;
this.apiListTypeClass = apiListTypeClass;
this.customObjectsApi = customObjectsApi;
}
// For unit-test
ApiClient getApiClient() {
return customObjectsApi.getApiClient();
}
/**
* Get kubernetes api response.
*
* @param name the name
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> get(String name) {
return get(name, new GetOptions());
}
/**
* Get kubernetes api response under the namespace.
*
* @param namespace the namespace
* @param name the name
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> get(String namespace, String name) {
return get(namespace, name, new GetOptions());
}
/**
* Get kubernetes api response.
*
* @param name the name
* @param callback the callback
* @return future for the kubernetes api response
```

```
*/
public Future<KubernetesApiResponse<ApiType>> getAsync(
String name, @Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
return getAsync(name, new GetOptions(), callback);
}
/**
* Get kubernetes api response under the namespace.
*
* @param namespace the namespace
* @param name the name
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> getAsync(
String namespace, String name, @Nonnull Consumer<KubernetesApiResponse<ApiType>> callback)
{
return getAsync(namespace, name, new GetOptions(), callback);
}
/**
* List kubernetes api response cluster-scoped.
*
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiListType> list() {
return list(new ListOptions());
}
/**
* List kubernetes api response under the namespace.
*
* @param namespace the namespace
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiListType> list(String namespace) {
return list(namespace, new ListOptions());
}
/**
* List kubernetes api response cluster-scoped.
*
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiListType>> listAsync(
@Nonnull Consumer<KubernetesApiResponse<ApiListType>> callback) {
return listAsync(new ListOptions(), callback);
}
/**
* List kubernetes api response under the namespace.
*
* @param namespace the namespace
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiListType>> listAsync(
```

```
String namespace, @Nonnull Consumer<KubernetesApiResponse<ApiListType>> callback) {
return listAsync(namespace, new ListOptions(), callback);
}
/**
* Create kubernetes api response, if the namespace in the object is present, it will send a
* namespace-scoped requests, vice versa.
*
* @param object the object
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> create(ApiType object) {
return create(object, new CreateOptions());
}
/**
* Create kubernetes api response, if the namespace in the object is present, it will send a
* namespace-scoped requests, vice versa.
*
* @param object the object
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> createAsync(
ApiType object, @Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
return createAsync(object, new CreateOptions(), callback);
}
/**
* Create kubernetes api response, if the namespace in the object is present, it will send a
* namespace-scoped requests, vice versa.
*
* @param object the object
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> update(ApiType object) {
return update(object, new UpdateOptions());
}
/**
* Create kubernetes api response, if the namespace in the object is present, it will send a
* namespace-scoped requests, vice versa.
*
* @param object the object
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> updateAsync(
ApiType object, @Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
return updateAsync(object, new UpdateOptions(), callback);
}
/**
* Patch kubernetes api response.
*
* @param name the name
* @param patchType the patch type, supported values defined in V1Patch
* @param patch the string patch content
```

```
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> patch(String name, String patchType, V1Patch patch) {
return patch(name, patchType, patch, new PatchOptions());
}
/**
* Patch kubernetes api response under the namespace.
*
* @param namespace the namespace
* @param name the name
* @param patchType the patch type, supported values defined in V1Patch
* @param patch the string patch content
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> patch(
String namespace, String name, String patchType, V1Patch patch) {
return patch(namespace, name, patchType, patch, new PatchOptions());
}
/**
* Patch kubernetes api response.
*
* @param name the name
* @param patchType the patch type, supported values defined in V1Patch
* @param patch the string patch content
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> patchAsync(
String name,
String patchType,
V1Patch patch,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
return patchAsync(name, patchType, patch, new PatchOptions(), callback);
}
/**
* Patch kubernetes api response under the namespace.
*
* @param namespace the namespace
* @param name the name
* @param patchType the patch type, supported values defined in V1Patch
* @param patch the string patch content
* @return the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> patchAsync(
String namespace,
String name,
String patchType,
V1Patch patch,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
return patchAsync(namespace, name, patchType, patch, new PatchOptions(), callback);
}
/**
* Delete kubernetes api response.
```

```
*
* @param name the name
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> delete(String name) {
return delete(name, new DeleteOptions());
}
/**
* Delete kubernetes api response under the namespace.
*
* @param namespace the namespace
* @param name the name
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> delete(String namespace, String name) {
return delete(namespace, name, new DeleteOptions());
}
/**
* Delete kubernetes api response.
*
* @param name the name
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> deleteAsync(
String name, @Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
return deleteAsync(name, new DeleteOptions(), callback);
}
/**
* Delete kubernetes api response under the namespace.
*
* @param namespace the namespace
* @param name the name
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> deleteAsync(
String namespace, String name, @Nonnull Consumer<KubernetesApiResponse<ApiType>> callback)
{
return deleteAsync(namespace, name, new DeleteOptions(), callback);
}
/**
* Creates a cluster-scoped Watch on the resource.
*
* @return the watchable
* @throws ApiException the api exception
*/
public Watchable<ApiType> watch() throws ApiException {
return watch(new ListOptions());
}
/**
* Creates a namespace-scoped Watch on the resource.
*
```

```
* @param namespace the namespace
* @return the watchable
* @throws ApiException the api exception
*/
public Watchable<ApiType> watch(String namespace) throws ApiException {
return watch(namespace, new ListOptions());
}
// TODO(yue9944882): watch one resource?
/**
* Get kubernetes api response.
*
* @param name the name
* @param getOptions the get options
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> get(String name, final GetOptions getOptions) {
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
return executeCall(
customObjectsApi.getApiClient(), apiTypeClass, makeClusterGetCallBuilder(name, getOptions));
}
private CallBuilder makeClusterGetCallBuilder(String name, final GetOptions getOptions) {
return () ->
adaptGetCall(
customObjectsApi.getApiClient(),
customObjectsApi.getClusterCustomObject(
this.apiGroup, this.apiVersion, this.resourcePlural, name)
.buildCall(null),
getOptions);
}
private Call adaptGetCall(ApiClient apiClient, Call call, GetOptions getOptions) {
if (Optional.ofNullable(getOptions)
.map(GetOptions::isPartialObjectMetadataRequest)
.orElse(Boolean.FALSE)) {
Request request = call.request();
return apiClient
.getHttpClient()
.newCall(request.newBuilder().header("Accept", PARTIAL_OBJECT_METADATA_HEADER).build());
}
return call;
}
/**
* Get kubernetes api response.
*
* @param name the name
* @param getOptions the get options
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> getAsync(
String name,
final GetOptions getOptions,
```

```
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
return executeCallAsync(
customObjectsApi.getApiClient(),
apiTypeClass,
makeClusterGetCallBuilder(name, getOptions),
callback);
}
/**
* Get kubernetes api response.
*
* @param namespace the namespace
* @param name the name
* @param getOptions the get options
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> get(
String namespace, String name, final GetOptions getOptions) {
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
if (Strings.isNullOrEmpty(namespace)) {
throw new IllegalArgumentException("invalid namespace");
}
return executeCall(
customObjectsApi.getApiClient(),
apiTypeClass,
makeNamespacedGetCallBuilder(namespace, name, getOptions));
}
private CallBuilder makeNamespacedGetCallBuilder(
String namespace, String name, final GetOptions getOptions) {
return () ->
adaptGetCall(
customObjectsApi.getApiClient(),
customObjectsApi.getNamespacedCustomObject(
this.apiGroup, this.apiVersion, namespace, this.resourcePlural, name)
.buildCall(null),
getOptions);
}
/**
* Get kubernetes api response.
*
* @param namespace the namespace
* @param name the name
* @param getOptions the get options
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> getAsync(
String namespace,
String name,
```

```
final GetOptions getOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
if (Strings.isNullOrEmpty(namespace)) {
throw new IllegalArgumentException("invalid namespace");
}
return executeCallAsync(
customObjectsApi.getApiClient(),
apiTypeClass,
makeNamespacedGetCallBuilder(namespace, name, getOptions),
callback);
}
/**
* List kubernetes api response.
*
* @param listOptions the list options
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiListType> list(final ListOptions listOptions) {
return executeCall(
customObjectsApi.getApiClient(), apiListTypeClass, makeClusterListCallBuilder(listOptions));
}
private CallBuilder makeClusterListCallBuilder(final ListOptions listOptions) {
return () ->
adaptListCall(
customObjectsApi.getApiClient(),
customObjectsApi.listClusterCustomObject( this.apiGroup, this.apiVersion, this.resourcePlural)
.fieldSelector(listOptions.getFieldSelector())
.labelSelector(listOptions.getLabelSelector())
.limit(listOptions.getLimit())
.resourceVersion(listOptions.getResourceVersion())
.timeoutSeconds(listOptions.getTimeoutSeconds())
.watch(false)
.buildCall(null),
listOptions);
}
private Call adaptListCall(ApiClient apiClient, Call call, ListOptions listOptions) {
if (Optional.ofNullable(listOptions)
.map(ListOptions::isPartialObjectMetadataListRequest)
.orElse(Boolean.FALSE)) {
Request request = call.request();
return apiClient
.getHttpClient()
.newCall(
request.newBuilder().header("Accept", PARTIAL_OBJECT_METADATA_LIST_HEADER).build());
}
return call;
}
/**
* List kubernetes api response.
*
```

```
* @param namespace the namespace
* @param listOptions the list options
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiListType> list(String namespace, final ListOptions listOptions) {
if (Strings.isNullOrEmpty(namespace)) {
throw new IllegalArgumentException("invalid namespace");
}
return executeCall(
customObjectsApi.getApiClient(),
apiListTypeClass,
makeNamespacedListCallBuilder(namespace, listOptions));
}
private CallBuilder makeNamespacedListCallBuilder(
String namespace, final ListOptions listOptions) {
return () ->
adaptListCall(
customObjectsApi.getApiClient(),
customObjectsApi.listNamespacedCustomObject(this.apiGroup, this.apiVersion, namespace,
this.resourcePlural)
._continue(listOptions.getContinue())
.fieldSelector(listOptions.getFieldSelector())
.labelSelector(listOptions.getLabelSelector())
.limit(listOptions.getLimit())
.resourceVersion(listOptions.getResourceVersion())
.timeoutSeconds(listOptions.getTimeoutSeconds())
.watch(false)
.buildCall(null),
listOptions);
}
/**
* List kubernetes api response.
*
* @param listOptions the list options
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiListType>> listAsync(
final ListOptions listOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiListType>> callback) {
return executeCallAsync(
customObjectsApi.getApiClient(),
apiListTypeClass,
makeClusterListCallBuilder(listOptions),
callback);
}
/**
* List kubernetes api response.
*
* @param namespace the namespace
* @param listOptions the list options
* @param callback the callback
* @return future for the kubernetes api response
```

```
*/
public Future<KubernetesApiResponse<ApiListType>> listAsync(
String namespace,
final ListOptions listOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiListType>> callback) {
if (Strings.isNullOrEmpty(namespace)) {
throw new IllegalArgumentException("invalid namespace");
}
return executeCallAsync(
customObjectsApi.getApiClient(),
apiListTypeClass,
makeNamespacedListCallBuilder(namespace, listOptions),
callback);
}
/**
* Create kubernetes api response.
*
* @param object the object
* @param createOptions the create options
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> create(ApiType object, final CreateOptions createOptions) {
V1ObjectMeta objectMeta = object.getMetadata();
boolean isNamespaced = !Strings.isNullOrEmpty(objectMeta.getNamespace());
if (isNamespaced) {
return create(objectMeta.getNamespace(), object, createOptions);
}
return executeCall(
customObjectsApi.getApiClient(),
apiTypeClass,
makeClusterCreateCallBuilder(object, createOptions));
}
private CallBuilder makeClusterCreateCallBuilder(
ApiType object, final CreateOptions createOptions) {
// TODO(yue9944882): judge namespaced object via api discovery
return () ->
customObjectsApi.createClusterCustomObject(
this.apiGroup,
this.apiVersion,
this.resourcePlural,
object)
.dryRun(createOptions.getDryRun())
.dryRun(createOptions.getFieldManager())
.buildCall(null);
}
/**
* Create kubernetes api response.
*
* @param namespace the namespace
* @param object the object
* @param createOptions the create options
* @return the kubernetes api response
*/
```

```
public KubernetesApiResponse<ApiType> create(
String namespace, ApiType object, final CreateOptions createOptions) {
return executeCall(
customObjectsApi.getApiClient(),
apiTypeClass,
makeNamespacedCreateCallBuilder(namespace, object, createOptions));
}
private CallBuilder makeNamespacedCreateCallBuilder(
String namespace, ApiType object, final CreateOptions createOptions) {
// TODO(yue9944882): judge namespaced object via api discovery
return () ->
customObjectsApi.createNamespacedCustomObject(
this.apiGroup,
this.apiVersion,
namespace,
this.resourcePlural,
object)
.dryRun(createOptions.getDryRun())
.fieldManager(createOptions.getFieldManager())
.buildCall(null);
}
/**
* Create kubernetes api response.
*
* @param object the object
* @param createOptions the create options
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> createAsync(
ApiType object,
final CreateOptions createOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
V1ObjectMeta objectMeta = object.getMetadata();
boolean isNamespaced = !Strings.isNullOrEmpty(objectMeta.getNamespace());
if (isNamespaced) {
return createAsync(objectMeta.getNamespace(), object, createOptions, callback);
}
return executeCallAsync(
customObjectsApi.getApiClient(),
apiTypeClass,
makeClusterCreateCallBuilder(object, createOptions),
callback);
}
/**
* Create kubernetes api response.
*
* @param namespace the namespace
* @param object the object
* @param createOptions the create options
* @param callback the callback
* @return future for the kubernetes api response
*/
```

```
public Future<KubernetesApiResponse<ApiType>> createAsync(
String namespace,
ApiType object,
final CreateOptions createOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
return executeCallAsync(
customObjectsApi.getApiClient(),
apiTypeClass,
makeNamespacedCreateCallBuilder(namespace, object, createOptions),
callback);
}
/**
* Update kubernetes api response.
*
* @param object the object
* @param updateOptions the update options
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> update(ApiType object, final UpdateOptions updateOptions)
{
return executeCall(
customObjectsApi.getApiClient(),
apiTypeClass,
makeUpdateCallBuilder(object, updateOptions));
}
private CallBuilder makeUpdateCallBuilder(ApiType object, final UpdateOptions updateOptions) {
V1ObjectMeta objectMeta = object.getMetadata();
boolean isNamespaced = !Strings.isNullOrEmpty(objectMeta.getNamespace());
return () ->
//// TODO(yue9944882): judge namespaced object via api discovery
isNamespaced
? customObjectsApi.replaceNamespacedCustomObject(
this.apiGroup,
this.apiVersion,
objectMeta.getNamespace(),
this.resourcePlural,
objectMeta.getName(),
object)
.dryRun(updateOptions.getDryRun())
.fieldManager(updateOptions.getFieldManager())
.buildCall(null)
: customObjectsApi.replaceClusterCustomObject(
this.apiGroup,
this.apiVersion,
this.resourcePlural,
objectMeta.getName(),
object)
.dryRun(updateOptions.getDryRun())
.fieldManager(updateOptions.getFieldManager())
.buildCall(null);
}
/**
* Update kubernetes api response.
```

```
*
* @param object the object
* @param updateOptions the update options
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> updateAsync(
ApiType object,
final UpdateOptions updateOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
return executeCallAsync(
customObjectsApi.getApiClient(),
apiTypeClass,
makeUpdateCallBuilder(object, updateOptions),
callback);
}
/**
* Create kubernetes api response, if the namespace in the object is present, it will send a
* namespace-scoped requests, vice versa.
*
* @param object the object
* @param status function to extract the status from the object
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> updateStatus(
ApiType object, Function<ApiType, Object> status) {
return updateStatus(object, status, new UpdateOptions());
}
/**
* Update status of kubernetes api response.
*
* @param object the object
* @param status function to extract the status from the object
* @param updateOptions the update options
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> updateStatus(
ApiType object, Function<ApiType, Object> status, final UpdateOptions updateOptions) {
return executeCall(
customObjectsApi.getApiClient(),
apiTypeClass,
makeUpdateStatusCallBuilder(object, status, updateOptions));
}
private CallBuilder makeUpdateStatusCallBuilder(
ApiType object, Function<ApiType, Object> status, final UpdateOptions updateOptions) {
V1ObjectMeta objectMeta = object.getMetadata();
boolean isNamespaced = !Strings.isNullOrEmpty(objectMeta.getNamespace());
return () ->
//// TODO(yue9944882): judge namespaced object via api discovery
isNamespaced
? customObjectsApi.patchNamespacedCustomObjectStatus(
this.apiGroup,
this.apiVersion,
```

```
objectMeta.getNamespace(),
this.resourcePlural,
objectMeta.getName(),
Arrays.asList(new StatusPatch(status.apply(object))))
.dryRun(updateOptions.getDryRun())
.fieldManager(updateOptions.getFieldManager())
.buildCall(null)
: customObjectsApi.patchClusterCustomObjectStatus(
this.apiGroup,
this.apiVersion,
this.resourcePlural,
objectMeta.getName(),
Arrays.asList(new StatusPatch(status.apply(object))))
.dryRun(updateOptions.getDryRun())
.fieldManager(updateOptions.getFieldManager())
.buildCall(null);
}
/**
* Create kubernetes api response, if the namespace in the object is present, it will send a
* namespace-scoped requests, vice versa.
*
* @param object the object
* @param status function to extract the status from the object
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> updateStatusAsync(
ApiType object,
Function<ApiType, Object> status,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
return updateStatusAsync(object, status, new UpdateOptions(), callback);
}
/**
* Update status of kubernetes api response.
*
* @param object the object
* @param status function to extract the status from the object
* @param updateOptions the update options
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> updateStatusAsync(
ApiType object,
Function<ApiType, Object> status,
final UpdateOptions updateOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
return executeCallAsync(
customObjectsApi.getApiClient(),
apiTypeClass,
makeUpdateStatusCallBuilder(object, status, updateOptions),
callback);
}
private Call adaptPatchCall(ApiClient apiClient, Call call, String patchType) {
```

```
Request request = call.request();
return apiClient
.getHttpClient()
.newCall(
request
.newBuilder()
.patch(new ProxyContentTypeRequestBody(request.body(), patchType))
.build());
}
/**
* Patch kubernetes api response.
*
* @param name the name
* @param patchType the patch type
* @param patch the patch
* @param patchOptions the patch options
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> patch(
String name, String patchType, V1Patch patch, final PatchOptions patchOptions) {
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
return executeCall(
customObjectsApi.getApiClient(),
apiTypeClass,
makeClusterPatchCallBuilder(name, patchType, patch, patchOptions));
}
private CallBuilder makeClusterPatchCallBuilder(
String name, String patchType, V1Patch patch, final PatchOptions patchOptions) {
return () ->
adaptPatchCall(
customObjectsApi.getApiClient(),
customObjectsApi.patchClusterCustomObject(
this.apiGroup,
this.apiVersion,
this.resourcePlural,
name,
patch)
.dryRun(patchOptions.getDryRun())
.fieldManager(patchOptions.getFieldManager())
.force(patchOptions.getForce())
.buildCall(null),
patchType);
}
/**
* Patch kubernetes api response.
*
* @param namespace the namespace
* @param name the name
* @param patchType the patch type
* @param patch the patch
* @param patchOptions the patch options
```

```
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> patch(
String namespace,
String name,
String patchType,
V1Patch patch,
final PatchOptions patchOptions) {
if (Strings.isNullOrEmpty(namespace)) {
throw new IllegalArgumentException("invalid namespace");
}
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
return executeCall(
customObjectsApi.getApiClient(),
apiTypeClass,
makeNamespacedPatchCallBuilder(namespace, name, patchType, patch, patchOptions));
}
private CallBuilder makeNamespacedPatchCallBuilder(
String namespace,
String name,
String patchType,
V1Patch patch,
final PatchOptions patchOptions) {
return () ->
adaptPatchCall(
customObjectsApi.getApiClient(),
customObjectsApi.patchNamespacedCustomObject(
this.apiGroup,
this.apiVersion,
namespace,
this.resourcePlural,
name,
patch)
.dryRun(patchOptions.getDryRun())
.fieldManager(patchOptions.getFieldManager())
.force(patchOptions.getForce())
.buildCall(null),
patchType);
}
/**
* Patch kubernetes api response.
*
* @param name the name
* @param patchType the patch type
* @param patch the patch
* @param patchOptions the patch options
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> patchAsync(
String name,
```

```
String patchType,
V1Patch patch,
final PatchOptions patchOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
return executeCallAsync(
customObjectsApi.getApiClient(),
apiTypeClass,
makeClusterPatchCallBuilder(name, patchType, patch, patchOptions),
callback);
}
/**
* Patch kubernetes api response.
*
* @param namespace the namespace
* @param name the name
* @param patchType the patch type
* @param patch the patch
* @param patchOptions the patch options
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> patchAsync(
String namespace,
String name,
String patchType,
V1Patch patch,
final PatchOptions patchOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
if (Strings.isNullOrEmpty(namespace)) {
throw new IllegalArgumentException("invalid namespace");
}
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
return executeCallAsync(
customObjectsApi.getApiClient(),
apiTypeClass,
makeNamespacedPatchCallBuilder(namespace, name, patchType, patch, patchOptions),
callback);
}
/**
* Delete kubernetes api response.
*
* @param name the name
* @param deleteOptions the delete options
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> delete(String name, final DeleteOptions deleteOptions) {
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
```

```
}
return executeCall(
customObjectsApi.getApiClient(),
apiTypeClass,
makeClusterDeleteCallBuilder(name, deleteOptions));
}
private CallBuilder makeClusterDeleteCallBuilder(String name, final DeleteOptions deleteOptions) {
return () ->
customObjectsApi.deleteClusterCustomObject(
this.apiGroup,
this.apiVersion,
this.resourcePlural,
name)
// TODO: fill/convert the option
.buildCall(null);
}
/**
* Delete kubernetes api response.
*
* @param namespace the namespace
* @param name the name
* @param deleteOptions the delete options
* @return the kubernetes api response
*/
public KubernetesApiResponse<ApiType> delete(
String namespace, String name, final DeleteOptions deleteOptions) {
if (Strings.isNullOrEmpty(namespace)) {
throw new IllegalArgumentException("invalid namespace");
}
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
return executeCall(
customObjectsApi.getApiClient(),
apiTypeClass,
makeNamespacedDeleteCallBuilder(namespace, name, deleteOptions));
}
private CallBuilder makeNamespacedDeleteCallBuilder(
String namespace, String name, final DeleteOptions deleteOptions) {
return () ->
customObjectsApi.deleteNamespacedCustomObject(
this.apiGroup,
this.apiVersion,
namespace,
this.resourcePlural,
name)
.body(deleteOptions)
.buildCall(null);
}
/**
* Delete kubernetes api response.
*
* @param name the name
```

```
* @param deleteOptions the delete options
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> deleteAsync(
String name,
final DeleteOptions deleteOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
return executeCallAsync(
customObjectsApi.getApiClient(),
apiTypeClass,
makeClusterDeleteCallBuilder(name, deleteOptions),
callback);
}
/**
* Delete kubernetes api response.
*
* @param namespace the namespace
* @param name the name
* @param deleteOptions the delete options
* @param callback the callback
* @return future for the kubernetes api response
*/
public Future<KubernetesApiResponse<ApiType>> deleteAsync(
String namespace,
String name,
final DeleteOptions deleteOptions,
@Nonnull Consumer<KubernetesApiResponse<ApiType>> callback) {
if (Strings.isNullOrEmpty(namespace)) {
throw new IllegalArgumentException("invalid namespace");
}
if (Strings.isNullOrEmpty(name)) {
throw new IllegalArgumentException("invalid name");
}
return executeCallAsync(
customObjectsApi.getApiClient(),
apiTypeClass,
makeNamespacedDeleteCallBuilder(namespace, name, deleteOptions),
callback);
}
/**
* Watch watchable.
*
* @param listOptions the list options
* @return the watchable
* @throws ApiException the api exception
*/
public Watchable<ApiType> watch(final ListOptions listOptions) throws ApiException {
Call call =
customObjectsApi.listClusterCustomObject(
```

```
this.apiGroup,
this.apiVersion,
this.resourcePlural)
._continue(listOptions.getContinue())
.fieldSelector(listOptions.getFieldSelector())
.labelSelector(listOptions.getLabelSelector())
.limit(listOptions.getLimit())
.resourceVersion(listOptions.getResourceVersion())
.timeoutSeconds(listOptions.getTimeoutSeconds())
.watch(true)
.buildCall(null);
call = tweakCallForCoreV1Group(call);
return Watch.createWatch(
customObjectsApi.getApiClient(),
call,
TypeToken.getParameterized(Watch.Response.class, apiTypeClass).getType());
}
/**
* Watch watchable.
*
* @param namespace the namespace
* @param listOptions the list options
* @return the watchable
* @throws ApiException the api exception
*/
public Watchable<ApiType> watch(String namespace, final ListOptions listOptions)
throws ApiException {
if (Strings.isNullOrEmpty(namespace)) {
throw new IllegalArgumentException("invalid namespace");
}
Call call =
customObjectsApi.listNamespacedCustomObject(
this.apiGroup,
this.apiVersion,
namespace,
this.resourcePlural)
._continue(listOptions.getContinue())
.fieldSelector(listOptions.getFieldSelector())
.labelSelector(listOptions.getLabelSelector())
.limit(listOptions.getLimit())
.resourceVersion(listOptions.getResourceVersion())
.timeoutSeconds(listOptions.getTimeoutSeconds())
.watch(true)
.buildCall(null);
call = tweakCallForCoreV1Group(call);
return Watch.createWatch(
customObjectsApi.getApiClient(),
call,
TypeToken.getParameterized(Watch.Response.class, apiTypeClass).getType());
}
private static <DataType extends KubernetesType>
KubernetesApiResponse<DataType> getKubernetesApiResponse(
Class<DataType> dataClass, JsonElement element, Gson gson) {
```

```
return getKubernetesApiResponse(dataClass, element, gson, 200);
}
private static <DataType extends KubernetesType>
KubernetesApiResponse<DataType> getKubernetesApiResponse(
Class<DataType> dataClass, JsonElement element, Gson gson, int httpStatusCode) {
JsonElement kindElement = element.getAsJsonObject().get("kind");
boolean isStatus = kindElement != null && "Status".equals(kindElement.getAsString());
if (isStatus) {
return new KubernetesApiResponse<>(gson.fromJson(element, V1Status.class), httpStatusCode);
}
return new KubernetesApiResponse<>(gson.fromJson(element, dataClass));
}
private Call prepareCall(CallBuilder callBuilder) throws ApiException {
Call call = callBuilder.build();
return tweakCallForCoreV1Group(call);
}
private void checkForIOException(ApiException e) {
if (e.getCause() instanceof IOException) {
throw new IllegalStateException(e.getCause()); // make this a checked exception?
}
}
private <DataType extends KubernetesType>
KubernetesApiResponse<DataType> responseFromApiException(
ApiClient apiClient, ApiException e) {
checkForIOException(e);
final V1Status status;
try {
status = apiClient.getJSON().deserialize(e.getResponseBody(), V1Status.class);
} catch (JsonSyntaxException jsonEx) {
// craft a status object
return new KubernetesApiResponse<>(
new V1Status().code(e.getCode()).message(e.getResponseBody()), e.getCode());
}
if (null == status) { // the response body can be something unexpected sometimes...
// this line should never reach?
throw new RuntimeException(e);
}
return new KubernetesApiResponse<>(status, e.getCode());
}
private <DataType extends KubernetesType> KubernetesApiResponse<DataType> executeCall(
ApiClient apiClient, Class<DataType> dataClass, CallBuilder callBuilder) {
try {
Call call = prepareCall(callBuilder);
ApiResponse<JsonElement> response = apiClient.execute(call, JsonElement.class);
JsonElement element = response.getData();
if (element == null) {
throw new ApiException(null, response.getStatusCode(), null, "Unexpected response body");
}
return getKubernetesApiResponse(dataClass, element, apiClient.getJSON().getGson());
} catch (ApiException e) {
return responseFromApiException(apiClient, e);
}
}
```

```
// For unit-test
interface CallAdapter extends UnaryOperator<Call> {}
private <DataType extends KubernetesType> Call prepareCall(
CallBuilder callBuilder, @Nonnull Consumer<KubernetesApiResponse<DataType>> callback)
throws ApiException {
Call call = prepareCall(callBuilder);
// For unit-test
if (callback instanceof CallAdapter) {
call = ((CallAdapter) callback).apply(call);
}
return call;
}
private <DataType extends KubernetesType>
Future<KubernetesApiResponse<DataType>> executeCallAsync(
ApiClient apiClient,
Class<DataType> dataClass,
CallBuilder callBuilder,
@Nonnull Consumer<KubernetesApiResponse<DataType>> callback) {
try {
Call call = prepareCall(callBuilder, callback);
CountDownLatch latch = new CountDownLatch(1);
AtomicReference<KubernetesApiResponse<DataType>> result = new AtomicReference<>();
apiClient.executeAsync(
call,
JsonElement.class,
new ApiCallback<JsonElement>() {
@Override
public void onFailure(
ApiException e, int statusCode, Map<String, List<String>> responseHeaders) {
if (latch.getCount() > 0) {
KubernetesApiResponse<DataType> r = responseFromApiException(apiClient, e);
result.set(r);
latch.countDown();
callback.accept(r);
}
}
@Override
public void onSuccess(
JsonElement element, int statusCode, Map<String, List<String>> responseHeaders) {
KubernetesApiResponse<DataType> r =
getKubernetesApiResponse(dataClass, element, apiClient.getJSON().getGson());
if (latch.getCount() > 0) {
result.set(r);
latch.countDown();
callback.accept(r);
}
}
@Override
public void onUploadProgress(long bytesWritten, long contentLength, boolean done) {
// no-op
}
@Override
public void onDownloadProgress(long bytesRead, long contentLength, boolean done) {
```

```
// no-op
}
});
return new Future<KubernetesApiResponse<DataType>>() {
private final AtomicBoolean isCanceled = new AtomicBoolean(false);
@Override
public boolean cancel(boolean mayInterruptIfRunning) {
if (isDone()) {
return false;
}
call.cancel();
isCanceled.set(true);
latch.countDown();
return true;
}
@Override
public boolean isCancelled() {
return isCanceled.get();
}
@Override
public boolean isDone() {
return latch.getCount() == 0;
}
@Override
public KubernetesApiResponse<DataType> get() throws InterruptedException {
latch.await();
if (isCancelled()) {
throw new CancellationException();
}
return result.get();
}
@Override
public KubernetesApiResponse<DataType> get(long timeout, @Nonnull TimeUnit unit)
throws InterruptedException, TimeoutException {
if (latch.await(timeout, unit)) {
if (isCancelled()) {
throw new CancellationException();
}
return result.get();
} else {
throw new TimeoutException();
}
}
};
} catch (ApiException e) {
checkForIOException(e);
throw new RuntimeException(e);
}
}
// CallBuilder builds a call and throws ApiException otherwise.
private interface CallBuilder {
/**
* Build call.
```

```
*
* @return the call
* @throws ApiException the api exception
*/
Call build() throws ApiException;
}
private Call tweakCallForCoreV1Group(Call call) {
if (!apiGroup.equals("")) {
return call;
}
HttpUrl url = call.request().url();
String basePath = customObjectsApi.getApiClient().getBasePath();
// count segments in the basePath
int offset = 0;
for (int i = basePath.indexOf('://') + 3; i < basePath.length(); ++i) {
if (basePath.charAt(i) == '/') {
++offset;
}
}
HttpUrl tweakedUrl =
url.newBuilder().removePathSegment(offset + 1).setPathSegment(offset, "api").build();
return this.customObjectsApi
.getApiClient()
.getHttpClient()
.newCall(call.request().newBuilder().url(tweakedUrl).build());
}
public static class StatusPatch {
private String op = "replace";
private String path = "/status";
private Object value;
public StatusPatch(Object value) {
this.value = value;
}
public String getOp() {
return op;
}
public void setOp(String op) {
this.op = op;
}
public String getPath() {
return path;
}
public void setPath(String path) {
this.path = path;
}
public Object getValue() {
return value;
}
public void setValue(Object value) {
this.value = value;
}
}
}
```

io/kubernetes/client/util/generic/dynamic/DynamicKubernetesApi.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.dynamic;
import com.google.gson.Gson;
import com.google.gson.TypeAdapter;
import io.kubernetes.client.common.KubernetesListObject;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.util.generic.GenericKubernetesApi;
/**
* DynamicKubernetesApi can be used for reading and writing arbitrary resources without knowing it's
* java definitions.
*/
public class DynamicKubernetesApi
extends GenericKubernetesApi<DynamicKubernetesObject, DynamicKubernetesListObject> {
private final Gson gson;
/**
* Instantiates a new Dynamic kubernetes api.
*
* @param apiGroup the api group
* @param apiVersion the api version
* @param resourcePlural the resource plural
* @param apiClient the api client
*/
public DynamicKubernetesApi(
String apiGroup, String apiVersion, String resourcePlural, ApiClient apiClient) {
super(
DynamicKubernetesObject.class,
DynamicKubernetesListObject.class,
apiGroup,
apiVersion,
resourcePlural,
apiClient);
TypeAdapter<KubernetesObject> objAdapter =
apiClient.getJSON().getGson().getAdapter(KubernetesObject.class);
TypeAdapter<KubernetesListObject> objListAdapter =
```

```
apiClient.getJSON().getGson().getAdapter(KubernetesListObject.class);
if (!DynamicKubernetesTypeAdaptorFactory.GenericListObjectCreator.class.equals(
objListAdapter.getClass())
|| !DynamicKubernetesTypeAdaptorFactory.GenericObjectCreator.class.equals(
objAdapter.getClass())) {
apiClient
.getJSON()
.setGson(
apiClient
.getJSON()
.getGson()
.newBuilder()
.registerTypeAdapterFactory(new DynamicKubernetesTypeAdaptorFactory())
.create());
}
gson = apiClient.getJSON().getGson();
}
/**
* Gets gson instance.
*
* @return the gson
*/
public Gson getGson() {
return gson;
}
}
```

io/kubernetes/client/util/generic/dynamic/DynamicKubernetesTypeAdaptorFactory.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.dynamic;
import com.google.gson.Gson;
import com.google.gson.InstanceCreator;
import com.google.gson.JsonObject;
import com.google.gson.TypeAdapter;
import com.google.gson.TypeAdapterFactory;
import com.google.gson.reflect.TypeToken;
import com.google.gson.stream.JsonReader;
```

```
import com.google.gson.stream.JsonWriter;
import io.kubernetes.client.common.KubernetesListObject;
import io.kubernetes.client.common.KubernetesObject;
import java.io.IOException;
import java.lang.reflect.Type;
public class DynamicKubernetesTypeAdaptorFactory implements TypeAdapterFactory {
@Override
public <T> TypeAdapter<T> create(Gson gson, TypeToken<T> typeToken) {
if (shouldHandleAsSingleObject(typeToken)) {
return (TypeAdapter<T>) (new GenericObjectCreator(gson));
}
if (shouldHandleAsListObject(typeToken)) {
return (TypeAdapter<T>) (new GenericListObjectCreator(gson));
}
return null;
}
private boolean shouldHandleAsSingleObject(TypeToken typeToken) {
if (TypeToken.get(KubernetesObject.class).equals(typeToken)) {
return true;
}
return TypeToken.get(DynamicKubernetesObject.class).equals(typeToken);
}
private boolean shouldHandleAsListObject(TypeToken typeToken) {
if (TypeToken.get(KubernetesListObject.class).equals(typeToken)) {
return true;
}
return TypeToken.get(DynamicKubernetesListObject.class).equals(typeToken);
}
class GenericListObjectCreator extends TypeAdapter<DynamicKubernetesListObject>
implements InstanceCreator<DynamicKubernetesListObject> {
GenericListObjectCreator(Gson delegate) {
this.delegate = delegate;
}
private final Gson delegate;
@Override
public DynamicKubernetesListObject createInstance(Type type) {
return new DynamicKubernetesListObject();
}
@Override
public void write(JsonWriter jsonWriter, DynamicKubernetesListObject t) throws IOException {
jsonWriter.jsonValue(delegate.toJson(t.getRaw()));
}
@Override
public DynamicKubernetesListObject read(JsonReader jsonReader) throws IOException {
return new DynamicKubernetesListObject(delegate.fromJson(jsonReader, JsonObject.class));
}
}
class GenericObjectCreator extends TypeAdapter<DynamicKubernetesObject>
implements InstanceCreator<DynamicKubernetesObject> {
GenericObjectCreator(Gson delegate) {
this.delegate = delegate;
}
private final Gson delegate;
```

```
@Override
public DynamicKubernetesObject createInstance(Type type) {
return new DynamicKubernetesObject();
}
@Override
public void write(JsonWriter jsonWriter, DynamicKubernetesObject t) throws IOException {
jsonWriter.jsonValue(delegate.toJson(t.getRaw()));
}
@Override
public DynamicKubernetesObject read(JsonReader jsonReader) throws IOException {
return new DynamicKubernetesObject(delegate.fromJson(jsonReader, JsonObject.class));
}
}
}
```

io/kubernetes/client/util/generic/dynamic/DynamicKubernetesObject.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.dynamic;
import com.google.gson.JsonObject;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.openapi.models.V1ObjectMeta;
import java.util.Objects;
/**
* DynamicKubernetesObject is a wrapper for {@link JsonObject} that fits the common kubernetes type
* interface {@link KubernetesObject}.
*/
public class DynamicKubernetesObject implements KubernetesObject {
public DynamicKubernetesObject() {
this(new JsonObject());
}
public DynamicKubernetesObject(JsonObject obj) {
this.raw = obj;
}
private final JsonObject raw;
@Override
public V1ObjectMeta getMetadata() {
return Dynamics.internalJSONCodec
```

```
.getGson()
.fromJson(this.raw.get("metadata"), V1ObjectMeta.class);
}
@Override
public String getApiVersion() {
return this.raw.get("apiVersion").getAsString();
}
@Override
public String getKind() {
return this.raw.get("kind").getAsString();
}
public JsonObject getRaw() {
return raw;
}
public void setApiVersion(String apiVersion) {
this.raw.addProperty("apiVersion", apiVersion);
}
public void setKind(String kind) {
this.raw.addProperty("kind", kind);
}
public void setMetadata(V1ObjectMeta objectMeta) {
this.raw.add("metadata", Dynamics.internalJSONCodec.getGson().toJsonTree(objectMeta));
}
@Override
public boolean equals(Object o) {
if (this == o) return true;
if (o == null || getClass() != o.getClass()) return false;
DynamicKubernetesObject that = (DynamicKubernetesObject) o;
return Objects.equals(raw, that.raw);
}
@Override
public int hashCode() {
return Objects.hash(raw);
}
}
```

io/kubernetes/client/util/generic/dynamic/LegacyJSON.java


```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
```

```
*/
package io.kubernetes.client.util.generic.dynamic;
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.JsonElement;
import com.google.gson.JsonParseException;
import com.google.gson.TypeAdapter;
import com.google.gson.stream.JsonReader;
import com.google.gson.stream.JsonWriter;
import io.gsonfire.GsonFireBuilder;
import io.kubernetes.client.gson.V1StatusPreProcessor;
import io.kubernetes.client.openapi.models.V1Status;
import java.io.IOException;
import java.io.StringReader;
import java.lang.reflect.Type;
import java.text.DateFormat;
import java.text.ParseException;
import java.time.Instant;
import java.time.LocalDate;
import java.time.OffsetDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeFormatterBuilder;
import java.time.format.DateTimeParseException;
import java.time.temporal.ChronoField;
import java.util.Date;
import java.util.Map;
import okio.ByteString;
@Deprecated
public class LegacyJSON {
private Gson gson;
private boolean isLenientOnJson = false;
private static final DateTimeFormatter RFC3339MICRO_FORMATTER =
new DateTimeFormatterBuilder()
.parseDefaulting(ChronoField.OFFSET_SECONDS, 0)
.append(DateTimeFormatter.ofPattern("yyyy-MM-dd'T'HH:mm:ss"))
.optionalStart()
.appendFraction(ChronoField.NANO_OF_SECOND, 6, 6, true)
.optionalEnd()
.appendOffsetId()
.toFormatter();
private DateTypeAdapter dateTypeAdapter = new DateTypeAdapter();
private SqlDateTypeAdapter sqlDateTypeAdapter = new SqlDateTypeAdapter();
private OffsetDateTimeTypeAdapter offsetDateTimeTypeAdapter =
new OffsetDateTimeTypeAdapter(RFC3339MICRO_FORMATTER);
private LocalDateTypeAdapter localDateTypeAdapter = new LocalDateTypeAdapter();
private ByteArrayAdapter byteArrayAdapter = new ByteArrayAdapter();
public static GsonBuilder createGson() {
GsonFireBuilder fireBuilder = new GsonFireBuilder();
GsonBuilder builder =
fireBuilder
.registerPreProcessor(V1Status.class, new V1StatusPreProcessor())
.createGsonBuilder();
return builder;
```

```
}
private static String getDiscriminatorValue(JsonElement readElement, String discriminatorField) {
JsonElement element = readElement.getAsJsonObject().get(discriminatorField);
if (null == element) {
throw new IllegalArgumentException(
"missing discriminator field: <" + discriminatorField + ">");
}
return element.getAsString();
}
/**
* Returns the Java class that implements the OpenAPI schema for the specified discriminator
* value.
*
* @param classByDiscriminatorValue The map of discriminator values to Java classes.
* @param discriminatorValue The value of the OpenAPI discriminator in the input data.
* @return The Java class that implements the OpenAPI schema
*/
private static Class getClassByDiscriminator(
Map classByDiscriminatorValue, String discriminatorValue) {
Class clazz = (Class) classByDiscriminatorValue.get(discriminatorValue);
if (null == clazz) {
throw new IllegalArgumentException(
"cannot determine model class of name: <" + discriminatorValue + ">");
}
return clazz;
}
public LegacyJSON() {
gson =
createGson()
.registerTypeAdapter(Date.class, dateTypeAdapter)
.registerTypeAdapter(java.sql.Date.class, sqlDateTypeAdapter)
.registerTypeAdapter(OffsetDateTime.class, offsetDateTimeTypeAdapter)
.registerTypeAdapter(LocalDate.class, localDateTypeAdapter)
.registerTypeAdapter(byte[].class, byteArrayAdapter)
.create();
}
/**
* Get Gson.
*
* @return Gson
*/
public Gson getGson() {
return gson;
}
/**
* Set Gson.
*
* @param gson Gson
* @return JSON
*/
public LegacyJSON setGson(Gson gson) {
this.gson = gson;
return this;
```

```
}
public LegacyJSON setLenientOnJson(boolean lenientOnJson) {
isLenientOnJson = lenientOnJson;
return this;
}
/**
* Serialize the given Java object into JSON string.
*
* @param obj Object
* @return String representation of the JSON
*/
public String serialize(Object obj) {
return gson.toJson(obj);
}
/**
* Deserialize the given JSON string to Java object.
*
* @param <T> Type
* @param body The JSON string
* @param returnType The type to deserialize into
* @return The deserialized Java object
*/
@SuppressWarnings("unchecked")
public <T> T deserialize(String body, Type returnType) {
try {
if (isLenientOnJson) {
JsonReader jsonReader = new JsonReader(new StringReader(body));
// see
// https://google-gson.googlecode.com/svn/trunk/gson/docs/javadocs/com/google/gson/stream/JsonRe
ader.html#setLenient(boolean)
jsonReader.setLenient(true);
return gson.fromJson(jsonReader, returnType);
} else {
return gson.fromJson(body, returnType);
}
} catch (JsonParseException e) {
// Fallback processing when failed to parse JSON form response body:
// return the response body string directly for the String return type;
if (returnType.equals(String.class)) {
return (T) body;
} else {
throw (e);
}
}
}
/**
* Gson TypeAdapter for Byte Array type
*/
public class ByteArrayAdapter extends TypeAdapter<byte[]> {
@Override
public void write(JsonWriter out, byte[] value) throws IOException {
boolean oldHtmlSafe = out.isHtmlSafe();
out.setHtmlSafe(false);
```

```
if (value == null) {
out.nullValue();
} else {
out.value(ByteString.of(value).base64());
}
out.setHtmlSafe(oldHtmlSafe);
}
@Override
public byte[] read(JsonReader in) throws IOException {
switch (in.peek()) {
case NULL:
in.nextNull();
return null;
default:
String bytesAsBase64 = in.nextString();
ByteString byteString = ByteString.decodeBase64(bytesAsBase64);
return byteString.toByteArray();
}
}
}
/**
* Gson TypeAdapter for JSR310 OffsetDateTime type
*/
public static class OffsetDateTimeTypeAdapter extends TypeAdapter<OffsetDateTime> {
private DateTimeFormatter formatter;
public OffsetDateTimeTypeAdapter() {
this(DateTimeFormatter.ISO_OFFSET_DATE_TIME);
}
public OffsetDateTimeTypeAdapter(DateTimeFormatter formatter) {
this.formatter = formatter;
}
public void setFormat(DateTimeFormatter dateFormat) {
this.formatter = dateFormat;
}
@Override
public void write(JsonWriter out, OffsetDateTime date) throws IOException {
if (date == null) {
out.nullValue();
} else {
out.value(formatter.format(date));
}
}
@Override
public OffsetDateTime read(JsonReader in) throws IOException {
switch (in.peek()) {
case NULL:
in.nextNull();
return null;
default:
String date = in.nextString();
if (date.endsWith("+0000")) {
date = date.substring(0, date.length() - 5) + "Z";
}
```

```
try {
return OffsetDateTime.parse(date, formatter);
} catch (DateTimeParseException e) {
// backward-compatibility for ISO8601 timestamp format
return OffsetDateTime.parse(date, DateTimeFormatter.ISO_OFFSET_DATE_TIME);
}
}
}
}
/**
* Gson TypeAdapter for JSR310 LocalDate type
*/
public class LocalDateTypeAdapter extends TypeAdapter<LocalDate> {
private DateTimeFormatter formatter;
public LocalDateTypeAdapter() {
this(DateTimeFormatter.ISO_LOCAL_DATE);
}
public LocalDateTypeAdapter(DateTimeFormatter formatter) {
this.formatter = formatter;
}
public void setFormat(DateTimeFormatter dateFormat) {
this.formatter = dateFormat;
}
@Override
public void write(JsonWriter out, LocalDate date) throws IOException {
if (date == null) {
out.nullValue();
} else {
out.value(formatter.format(date));
}
}
@Override
public LocalDate read(JsonReader in) throws IOException {
switch (in.peek()) {
case NULL:
in.nextNull();
return null;
default:
String date = in.nextString();
return LocalDate.parse(date, formatter);
}
}
}
public LegacyJSON setOffsetDateTimeFormat(DateTimeFormatter dateFormat) {
offsetDateTimeTypeAdapter.setFormat(dateFormat);
return this;
}
public LegacyJSON setLocalDateFormat(DateTimeFormatter dateFormat) {
localDateTypeAdapter.setFormat(dateFormat);
return this;
}
/**
* Gson TypeAdapter for java.sql.Date type If the dateFormat is null, a simple "yyyy-MM-dd" format
```

```
* will be used (more efficient than SimpleDateFormat).
*/
public static class SqlDateTypeAdapter extends TypeAdapter<java.sql.Date> {
private DateFormat dateFormat;
public SqlDateTypeAdapter() {
}
public SqlDateTypeAdapter(DateFormat dateFormat) {
this.dateFormat = dateFormat;
}
public void setFormat(DateFormat dateFormat) {
this.dateFormat = dateFormat;
}
@Override
public void write(JsonWriter out, java.sql.Date date) throws IOException {
if (date == null) {
out.nullValue();
} else {
String value;
if (dateFormat != null) {
value = dateFormat.format(date);
} else {
value = date.toString();
}
out.value(value);
}
}
@Override
public java.sql.Date read(JsonReader in) throws IOException {
switch (in.peek()) {
case NULL:
in.nextNull();
return null;
default:
String date = in.nextString();
try {
if (dateFormat != null) {
return new java.sql.Date(dateFormat.parse(date).getTime());
}
return new java.sql.Date(
Instant.from(DateTimeFormatter.ISO_INSTANT.parse(date)).toEpochMilli());
} catch (ParseException e) {
throw new JsonParseException(e);
}
}
}
}
/**
* Gson TypeAdapter for java.util.Date type If the dateFormat is null, ISO8601Utils will be used.
*/
public static class DateTypeAdapter extends TypeAdapter<Date> {
private DateFormat dateFormat;
public DateTypeAdapter() {
}
```

```
public DateTypeAdapter(DateFormat dateFormat) {
this.dateFormat = dateFormat;
}
public void setFormat(DateFormat dateFormat) {
this.dateFormat = dateFormat;
}
@Override
public void write(JsonWriter out, Date date) throws IOException {
if (date == null) {
out.nullValue();
} else {
String value;
if (dateFormat != null) {
value = dateFormat.format(date);
} else {
value = DateTimeFormatter.ISO_INSTANT.format(date.toInstant());
}
out.value(value);
}
}
@Override
public Date read(JsonReader in) throws IOException {
try {
switch (in.peek()) {
case NULL:
in.nextNull();
return null;
default:
String date = in.nextString();
try {
if (dateFormat != null) {
return dateFormat.parse(date);
}
return Date.from(Instant.from(DateTimeFormatter.ISO_INSTANT.parse(date)));
} catch (ParseException e) {
throw new JsonParseException(e);
}
}
} catch (IllegalArgumentException e) {
throw new JsonParseException(e);
}
}
}
public LegacyJSON setDateFormat(DateFormat dateFormat) {
dateTypeAdapter.setFormat(dateFormat);
return this;
}
public LegacyJSON setSqlDateFormat(DateFormat dateFormat) {
sqlDateTypeAdapter.setFormat(dateFormat);
return this;
}
}
```

io/kubernetes/client/util/generic/dynamic/Dynamics.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.dynamic;
import com.google.gson.Gson;
import com.google.gson.JsonElement;
import java.util.Map;
import org.yaml.snakeyaml.LoaderOptions;
import org.yaml.snakeyaml.Yaml;
import org.yaml.snakeyaml.constructor.SafeConstructor;
public class Dynamics {
static final LegacyJSON internalJSONCodec = new LegacyJSON();
static final Yaml internalYamlCodec = new Yaml(new SafeConstructor(new LoaderOptions()));
public static DynamicKubernetesObject newFromJson(String jsonContent) {
return newFromJson(internalJSONCodec.getGson(), jsonContent);
}
public static DynamicKubernetesObject newFromJson(Gson gson, String jsonContent) {
JsonElement raw = gson.fromJson(jsonContent, JsonElement.class);
return new DynamicKubernetesObject(raw.getAsJsonObject());
}
public static DynamicKubernetesListObject newListFromJson(String jsonContent) {
return newListFromJson(internalJSONCodec.getGson(), jsonContent);
}
public static DynamicKubernetesListObject newListFromJson(Gson gson, String jsonContent) {
JsonElement raw = gson.fromJson(jsonContent, JsonElement.class);
return new DynamicKubernetesListObject(raw.getAsJsonObject());
}
public static DynamicKubernetesObject newFromYaml(String yamlContent) {
return newFromYaml(internalYamlCodec, yamlContent);
}
public static DynamicKubernetesObject newFromYaml(Yaml yamlCodec, String yamlContent) {
return newFromJson(
internalJSONCodec.getGson(),
fromYamlToJson(yamlCodec, internalJSONCodec.getGson(), yamlContent));
}
public static String fromYamlToJson(String yamlContent) {
return fromYamlToJson(internalYamlCodec, internalJSONCodec.getGson(), yamlContent);
}
```

```
public static String fromJsonToYaml(String jsonContent) {
return fromJsonToYaml(internalYamlCodec, internalJSONCodec.getGson(), jsonContent);
}
public static String fromYamlToJson(Yaml yamlCodec, Gson gson, String yamlContent) {
Map<String, Object> rawYamlObj = yamlCodec.load(yamlContent);
return gson.toJson(rawYamlObj);
}
public static String fromJsonToYaml(Yaml yamlCodec, Gson gson, String jsonContent) {
Map<String, Object> rawJsonObj = gson.fromJson(jsonContent, Map.class);
return yamlCodec.dump(rawJsonObj);
}
}
```

io/kubernetes/client/util/generic/dynamic/DynamicKubernetesListObject.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic.dynamic;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import io.kubernetes.client.common.KubernetesListObject;
import io.kubernetes.client.openapi.Configuration;
import io.kubernetes.client.openapi.models.V1ListMeta;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
/**
* DynamicKubernetesListObject is a wrapper for {@link JsonObject} that fits the common kubernetes
* list type interface {@link KubernetesListObject}.
*/
public class DynamicKubernetesListObject implements KubernetesListObject {
public DynamicKubernetesListObject() {
this(new JsonObject());
}
public DynamicKubernetesListObject(JsonObject obj) {
this.raw = obj;
}
private final JsonObject raw;
@Override
```

```
public V1ListMeta getMetadata() {
return Configuration.getDefaultApiClient()
.getJSON()
.getGson()
.fromJson(this.raw.get("metadata"), V1ListMeta.class);
}
@Override
public String getApiVersion() {
return this.raw.get("apiVersion").getAsString();
}
@Override
public String getKind() {
return this.raw.get("kind").getAsString();
}
@Override
public List<DynamicKubernetesObject> getItems() {
List<DynamicKubernetesObject> list = new ArrayList<>();
for (JsonElement e : this.raw.get("items").getAsJsonArray()) {
list.add(new DynamicKubernetesObject(e.getAsJsonObject()));
}
return list;
}
public JsonObject getRaw() {
return raw;
}
public void setApiVersion(String apiVersion) {
this.raw.addProperty("apiVersion", apiVersion);
}
public void setKind(String kind) {
this.raw.addProperty("kind", kind);
}
public void setMetadata(V1ListMeta objectMeta) {
this.raw.add(
"metadata", Configuration.getDefaultApiClient().getJSON().getGson().toJsonTree(objectMeta));
}
@Override
public boolean equals(Object o) {
if (this == o) return true;
if (o == null || getClass() != o.getClass()) return false;
DynamicKubernetesListObject that = (DynamicKubernetesListObject) o;
return Objects.equals(raw, that.raw);
}
@Override
public int hashCode() {
return Objects.hash(raw);
}
}
```

io/kubernetes/client/util/generic/KubernetesApiResponse.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.generic;
import io.kubernetes.client.common.KubernetesType;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.models.V1Status;
import java.net.HttpURLConnection;
public class KubernetesApiResponse<DataType extends KubernetesType> {
private final DataType object;
private final V1Status status;
private final int httpStatusCode;
public KubernetesApiResponse(DataType object) {
this.object = object;
this.status = null;
this.httpStatusCode = HttpURLConnection.HTTP_OK; // 200
}
public KubernetesApiResponse(V1Status status, int httpStatusCode) {
this.object = null;
this.status = status;
this.httpStatusCode = httpStatusCode;
}
public DataType getObject() {
return object;
}
public V1Status getStatus() {
return status;
}
public int getHttpStatusCode() {
return httpStatusCode;
}
public boolean isSuccess() {
return this.httpStatusCode < HttpURLConnection.HTTP_BAD_REQUEST; // 400
}
/**
* Throws api exception kubernetes api response on failure. This is the recommended approach to
* deal with errors returned from server.
*
* @return the kubernetes api response
* @throws ApiException the api exception
*/
public KubernetesApiResponse<DataType> throwsApiException() throws ApiException {
return onFailure(
```

```
(code, errorStatus) -> {
throw new ApiException(code, errorStatus.toString());
});
}
/**
* Calling errorStatusHandler upon errors from server..
*
* @param errorStatusHandler the error status handler
* @return the kubernetes api response
* @throws ApiException the api exception
*/
public KubernetesApiResponse<DataType> onFailure(ErrorStatusHandler errorStatusHandler)
throws ApiException {
if (!isSuccess()) {
errorStatusHandler.handle(this.getHttpStatusCode(), this.status);
}
return this;
}
public interface ErrorStatusHandler {
void handle(int code, V1Status errorStatus) throws ApiException;
}
}
```

io/kubernetes/client/util/ProxyContentTypeRequestBody.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import java.io.IOException;
import okhttp3.MediaType;
import okhttp3.RequestBody;
import okio.BufferedSink;
public class ProxyContentTypeRequestBody extends RequestBody {
private String overridePatchFormat;
private RequestBody delegateRequestBody;
public ProxyContentTypeRequestBody(RequestBody delegateRequestBody) {
this.delegateRequestBody = delegateRequestBody;
}
public ProxyContentTypeRequestBody(RequestBody delegateRequestBody, String patchFormat) {
```

```
this(delegateRequestBody);
this.overridePatchFormat = patchFormat;
}
@Override
public MediaType contentType() {
if (!Strings.isNullOrEmpty(overridePatchFormat)) {
return MediaType.parse(overridePatchFormat);
}
return delegateRequestBody.contentType();
}
@Override
public void writeTo(BufferedSink bufferedSink) throws IOException {
delegateRequestBody.writeTo(bufferedSink);
}
}
```

io/kubernetes/client/util/Namespaces.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import java.io.File;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
/** Namespaces provides a set of helpers for operating namespaces. */
public class Namespaces {
public static final String NAMESPACE_ALL = "";
public static final String NAMESPACE_DEFAULT = "default";
public static final String NAMESPACE_KUBESYSTEM = "kube-system";
public static String getPodNamespace() throws IOException {
return new String(
Files.readAllBytes(new File(Config.SERVICEACCOUNT_NAMESPACE_PATH).toPath()),
StandardCharsets.UTF_8);
}
}
```

io/kubernetes/client/util/SSLUtils.java

```
/*
```

Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
```
package io.kubernetes.client.util;
import java.io.ByteArrayInputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.StringWriter;
import java.security.KeyPair;
import java.security.KeyStore;
import java.security.KeyStoreException;
import java.security.NoSuchAlgorithmException;
import java.security.PrivateKey;
import java.security.Provider;
import java.security.Security;
import java.security.UnrecoverableKeyException;
import java.security.cert.Certificate;
import java.security.cert.CertificateException;
import java.security.cert.CertificateFactory;
import java.security.cert.X509Certificate;
import java.security.spec.InvalidKeySpecException;
import java.util.Collection;
import java.util.ServiceLoader;
import javax.net.ssl.KeyManager;
import javax.net.ssl.KeyManagerFactory;
import org.bouncycastle.asn1.pkcs.PrivateKeyInfo;
import org.bouncycastle.openssl.PEMKeyPair;
import org.bouncycastle.openssl.PEMParser;
import org.bouncycastle.openssl.jcajce.JcaMiscPEMGenerator;
import org.bouncycastle.openssl.jcajce.JcaPEMKeyConverter;
import org.bouncycastle.util.io.pem.PemWriter;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class SSLUtils {
private static final Logger log = LoggerFactory.getLogger(SSLUtils.class);
static {
ServiceLoader<Provider> services = ServiceLoader.load(java.security.Provider.class);
for (Provider service : services) {
log.debug("Found security provider: " + service.getName());
Security.addProvider(service);
}
}
```

```
public static boolean isNotNullOrEmpty(String val) {
return val != null && val.length() > 0;
}
public static KeyManager[] keyManagers(
byte[] certData,
byte[] keyData,
String algo,
String passphrase,
String keyStoreFile,
String keyStorePassphrase)
throws NoSuchAlgorithmException, UnrecoverableKeyException, KeyStoreException,
CertificateException, InvalidKeySpecException, IOException {
KeyManager[] keyManagers = null;
if (certData != null && keyData != null) {
KeyStore keyStore =
createKeyStore(certData, keyData, algo, passphrase, keyStoreFile, keyStorePassphrase);
KeyManagerFactory kmf =
KeyManagerFactory.getInstance(KeyManagerFactory.getDefaultAlgorithm());
kmf.init(keyStore, passphrase.toCharArray());
keyManagers = kmf.getKeyManagers();
}
return keyManagers;
}
public static KeyStore createKeyStore(
byte[] clientCertData,
byte[] clientKeyData,
String clientKeyAlgo,
String clientKeyPassphrase,
String keyStoreFile,
String keyStorePassphrase)
throws IOException, CertificateException, NoSuchAlgorithmException, InvalidKeySpecException,
KeyStoreException {
try (InputStream certInputStream = new ByteArrayInputStream(clientCertData);
InputStream keyInputStream = new ByteArrayInputStream(clientKeyData)) {
return createKeyStore(
certInputStream,
keyInputStream,
clientKeyAlgo,
clientKeyPassphrase != null ? clientKeyPassphrase.toCharArray() : null,
keyStoreFile,
getKeyStorePassphrase(keyStorePassphrase));
}
}
public static byte[] dumpKey(PrivateKey privateKey) throws IOException {
StringWriter writer = new StringWriter();
PemWriter pemWriter = new PemWriter(writer);
pemWriter.writeObject(new JcaMiscPEMGenerator(privateKey));
pemWriter.flush();
return writer.toString().getBytes();
}
public static String recognizePrivateKeyAlgo(byte[] privateKeyBytes) {
String dataString = new String(privateKeyBytes);
String algo = ""; // PKCS#8
```

```
if (dataString.contains("BEGIN EC PRIVATE KEY")) {
algo = "EC"; // PKCS#1 - EC
}
if (dataString.contains("BEGIN RSA PRIVATE KEY")) {
algo = "RSA"; // PKCS#1 - RSA
}
return algo;
}
public static PrivateKey loadKey(byte[] privateKeyBytes)
throws IOException, InvalidKeySpecException {
return loadKey(
new ByteArrayInputStream(privateKeyBytes), recognizePrivateKeyAlgo(privateKeyBytes));
}
public static PrivateKey loadKey(byte[] pemPrivateKeyBytes, String algo)
throws IOException, InvalidKeySpecException {
return loadKey(new ByteArrayInputStream(pemPrivateKeyBytes), algo);
}
public static PrivateKey loadKey(InputStream keyInputStream, String clientKeyAlgo)
throws IOException, InvalidKeySpecException {
final PrivateKey privateKey;
try (final PEMParser pemParser = new PEMParser(new InputStreamReader(keyInputStream))) {
final Object pemObject = pemParser.readObject();
if (pemObject == null) {
final String message =
String.format("PEM Private Key Algorithm [%s] not parsed", clientKeyAlgo);
throw new InvalidKeySpecException(message);
}
final JcaPEMKeyConverter converter = new JcaPEMKeyConverter();
if (pemObject instanceof PEMKeyPair) {
final PEMKeyPair pemKeyPair = (PEMKeyPair) pemObject;
final KeyPair keyPair = converter.getKeyPair(pemKeyPair);
privateKey = keyPair.getPrivate();
} else if (pemObject instanceof PrivateKeyInfo) {
final PrivateKeyInfo privateKeyInfo = (PrivateKeyInfo) pemObject;
privateKey = converter.getPrivateKey(privateKeyInfo);
} else {
final String pemObjectType = pemObject.getClass().getSimpleName();
final String message =
String.format(
"PEM Private Key Algorithm [%s] Type [%s] not supported",
clientKeyAlgo, pemObjectType);
throw new InvalidKeySpecException(message);
}
}
return privateKey;
}
public static KeyStore createKeyStore(
InputStream certInputStream,
InputStream keyInputStream,
String clientKeyAlgo,
char[] clientKeyPassphrase,
String keyStoreFile,
char[] keyStorePassphrase)
```

```
throws IOException, CertificateException, NoSuchAlgorithmException, InvalidKeySpecException,
KeyStoreException {
CertificateFactory certFactory = CertificateFactory.getInstance("X509");
Collection<? extends Certificate> certs = certFactory.generateCertificates(certInputStream);
PrivateKey privateKey = loadKey(keyInputStream, clientKeyAlgo);
KeyStore keyStore;
try {
keyStore = KeyStore.getInstance("JKS");
} catch (KeyStoreException e) {
// Not having an instance of JKS happens on Android, for example.
// Since we rely on BouncyCastle anyway, let's try BKS instead
// (which is BouncyCastle's JKS compatible provider).
keyStore = KeyStore.getInstance("BKS");
}
if (keyStoreFile != null && keyStoreFile.length() > 0) {
keyStore.load(new FileInputStream(keyStoreFile), keyStorePassphrase);
} else {
loadDefaultKeyStoreFile(keyStore, keyStorePassphrase);
}
String alias =
((X509Certificate) certs.stream().findFirst().get()).getSubjectX500Principal().getName();
keyStore.setKeyEntry(
alias, privateKey, clientKeyPassphrase, certs.toArray(new X509Certificate[certs.size()]));
return keyStore;
}
private static void loadDefaultKeyStoreFile(KeyStore keyStore, char[] keyStorePassphrase)
throws CertificateException, NoSuchAlgorithmException, IOException {
String keyStorePath = System.getProperty("javax.net.ssl.keyStore");
if (keyStorePath != null && keyStorePath.length() > 0) {
File keyStoreFile = new File(keyStorePath);
if (loadDefaultStoreFile(keyStore, keyStoreFile, keyStorePassphrase)) {
return;
}
}
keyStore.load(null);
}
private static boolean loadDefaultStoreFile(KeyStore keyStore, File fileToLoad, char[] passphrase)
throws CertificateException, NoSuchAlgorithmException, IOException {
if (fileToLoad.exists() && fileToLoad.isFile() && fileToLoad.length() > 0) {
try (FileInputStream inputStream = new FileInputStream(fileToLoad)) {
keyStore.load(inputStream, passphrase);
}
return true;
}
return false;
}
private static char[] getKeyStorePassphrase(String keyStorePassphrase) {
if (keyStorePassphrase == null || keyStorePassphrase.length() == 0) {
return System.getProperty("javax.net.ssl.keyStorePassword", "changeit").toCharArray();
}
return keyStorePassphrase.toCharArray();
}
}
```

io/kubernetes/client/util/CallGenerator.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.openapi.ApiException;
import okhttp3.Call;
/**
* The interface Call generator. It's for homogenizing client interface so that we can invoke a
* generized adaptor interface elsewhere.
*
* <p>For example, we can adapt list-node interface from CoreV1Api package like:
*
* <p>(CallGeneratorParams params) -> { return coreV1Api.listNodeCall( null, null, null, null, null,
* params.resourceVersion, params.timeoutSeconds, params.watch, null, null); },
*/
@FunctionalInterface
public interface CallGenerator {
/**
* Generate call.
*
* @param params the params
* @return the call
* @throws ApiException the api exception
*/
Call generate(CallGeneratorParams params) throws ApiException;
}
```

io/kubernetes/client/util/taints/Taints.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
```

```
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.taints;
import io.kubernetes.client.openapi.models.V1Node;
import io.kubernetes.client.openapi.models.V1NodeSpec;
import io.kubernetes.client.openapi.models.V1Taint;
import java.util.Iterator;
public class Taints {
public enum Effect {
NO_SCHEDULE {
public String toString() {
return "NoSchedule";
}
},
PREFER_NO_SCHEDULE {
public String toString() {
return "PreferNoSchedule";
}
},
NO_EXECUTE {
public String toString() {
return "NoExecute";
}
}
}
public static V1Taint findTaint(V1Node node, String key, Effect effect) {
for (V1Taint taint : node.getSpec().getTaints()) {
if (taint.getKey().equals(key) && taint.getEffect().equals(effect.toString())) {
return taint;
}
}
return null;
}
public static TaintsBuilder taints(V1Node node) {
return new TaintsBuilder(node);
}
public static class TaintsBuilder {
private V1Node node;
TaintsBuilder(V1Node node) {
this.node = node;
}
/**
* Add a taint to a node
*
* @param key The key for the taint
* @param effect The effect
*/
public TaintsBuilder addTaint(String key, Effect effect) {
return addTaint(key, null, effect);
}
/**
* Add a taint to a node
```

```
*
* @param key The key for the taint
* @param effect The effect
*/
public TaintsBuilder addTaint(String key, String value, Effect effect) {
V1Taint taint = new V1Taint();
taint.setKey(key);
taint.setValue(value);
taint.setEffect(effect.toString());
if (node.getSpec() == null) {
node.setSpec(new V1NodeSpec());
}
node.getSpec().addTaintsItem(taint);
return this;
}
/**
* Add a taint to a node
*
* @param key The key for the taint
* @param effect The effect
*/
public TaintsBuilder updateTaint(String key, String value, Effect effect) {
V1Taint taint = findTaint(node, key, effect);
taint.setValue(value);
return this;
}
/**
* Remove all taints matching a specified key
*
* @param key: The key for the taint(s) to remove
*/
public TaintsBuilder removeTaint(String key) {
if (node.getSpec() == null) {
return this;
}
Iterator<V1Taint> taints = node.getSpec().getTaints().iterator();
while (taints.hasNext()) {
V1Taint taint = taints.next();
if (taint.getKey().equals(key)) {
taints.remove();
}
}
return this;
}
/**
* Remove a taint matching a key and an effect
*
* @param key They key to match
* @param effect The effect to match
*/
public TaintsBuilder removeTaint(String key, Effect effect) {
if (node.getSpec() == null) {
return this;
```

```
}
Iterator<V1Taint> taints = node.getSpec().getTaints().iterator();
while (taints.hasNext()) {
V1Taint taint = taints.next();
if (taint.getKey().equals(key) && taint.getEffect().equals(effect.toString())) {
taints.remove();
}
}
return this;
}
}
}
```

io/kubernetes/client/util/Strings.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import javax.annotation.Nullable;
public final class Strings {
private Strings() {
throw new AssertionError("not supported");
}
public static boolean isNullOrEmpty(String value) {
return value == null || value.isEmpty();
}
public static String nullToEmpty(@Nullable String string) {
return (string == null) ? "" : string;
}
}
```

io/kubernetes/client/util/PatchUtils.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
```

```
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.Configuration;
import okhttp3.Call;
import okhttp3.Request;
/** Provides utility helper functions to send PATCH requests against kubernetes cluster. */
public class PatchUtils {
/**
* Patches the resource with the patch format specified.
*
* @param <ApiType> the type parameter
* @param apiTypeClass the api type class
* @param callFunc the call func returns a okhttp call object that wraps the http request
* @param patchFormat the patch format
* @return the api type
* @throws ApiException the api exception
*/
public static <ApiType> ApiType patch(
Class<ApiType> apiTypeClass, PatchCallFunc callFunc, String patchFormat) throws ApiException {
return patch(apiTypeClass, callFunc, patchFormat, Configuration.getDefaultApiClient());
}
/**
* Patches the resource with the patch format and {@link io.kubernetes.client.openapi.ApiClient}
* specified.
*
* @param <ApiType> the type parameter
* @param apiTypeClass the api type class
* @param callFunc the call func returns a okhttp call object that wraps the http request
* @param patchFormat the patch format
* @param apiClient the api client
* @return the api type
* @throws ApiException the api exception
*/
public static <ApiType> ApiType patch(
Class<ApiType> apiTypeClass, PatchCallFunc callFunc, String patchFormat, ApiClient apiClient)
throws ApiException {
Request request = callFunc.getCall().request();
return apiClient
.<ApiType>execute(
apiClient
.getHttpClient()
.newCall(
callFunc
.getCall()
.request()
.newBuilder()
.patch(new ProxyContentTypeRequestBody(request.body(), patchFormat))
```

```
.build()),
apiTypeClass)
.getData();
}
/**
* The Patch call func returns an okhttp call object. This interface is only used for adapting
* generated openapi client interfaces. e.g. {@link
* io.kubernetes.client.openapi.apis.CoreV1Api#patchNamespacedPodCall}
*/
public interface PatchCallFunc {
/**
* Gets okhttp call object.
*
* @return the call
* @throws ApiException the api exception
*/
Call getCall() throws ApiException;
}
}
```

io/kubernetes/client/util/annotations/Annotations.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.annotations;
import io.kubernetes.client.common.KubernetesObject;
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
public class Annotations {
/**
* Adds one annotation to the object.
*
* @param kubernetesObject the kubernetes object
* @param annotation the annotation key
* @param value the annotations value
*/
public static void addAnnotations(
KubernetesObject kubernetesObject, String annotation, String value) {
```

```
addAnnotations(kubernetesObject, Collections.singletonMap(annotation, value));
}
/**
* Adds a set of annotations to the object.
*
* @param kubernetesObject the kubernetes object
* @param mergingAnnotations the merging annotations
*/
public static void addAnnotations(
KubernetesObject kubernetesObject, Map<String, String> mergingAnnotations) {
if (null == mergingAnnotations) {
return;
}
final Map<String, String> currentAnnotations =
null != kubernetesObject.getMetadata().getAnnotations()
? kubernetesObject.getMetadata().getAnnotations()
: new HashMap<>();
mergingAnnotations.entrySet().stream()
.forEach(e -> currentAnnotations.put(e.getKey(), e.getValue()));
kubernetesObject.getMetadata().setAnnotations(currentAnnotations);
}
}
```

io/kubernetes/client/util/TypeAccessor.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.util.exception.TypeMetaReflectException;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
public class TypeAccessor {
private static final String METHOD_NAME_GET_APIVERSION = "getApiVersion";
private static final String METHOD_NAME_GET_KIND = "getKind";
public static String apiVersion(Object obj) throws TypeMetaReflectException {
try {
Method mdField = obj.getClass().getMethod(METHOD_NAME_GET_APIVERSION);
return (String) mdField.invoke(obj);
} catch (NoSuchMethodException | IllegalAccessException | InvocationTargetException e) {
throw new TypeMetaReflectException(e);
}
```

```
}
public static String kind(Object obj) throws TypeMetaReflectException {
try {
Method mdField = obj.getClass().getMethod(METHOD_NAME_GET_KIND);
return (String) mdField.invoke(obj);
} catch (NoSuchMethodException | IllegalAccessException | InvocationTargetException e) {
throw new TypeMetaReflectException(e);
}
}
}
```

io/kubernetes/client/util/exception/CopyNotSupportedException.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.exception;
public class CopyNotSupportedException extends Exception {
public CopyNotSupportedException(String errorMessage) {
super(errorMessage);
}
}
```

io/kubernetes/client/util/exception/TypeMetaReflectException.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
```

```
package io.kubernetes.client.util.exception;
public class TypeMetaReflectException extends Exception {
public TypeMetaReflectException(ReflectiveOperationException e) {
super(e);
}
}
```

io/kubernetes/client/util/exception/CSRSigningException.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.exception;
public class CSRSigningException extends Exception {
public CSRSigningException(Exception e) {
super(e);
}
}
```

io/kubernetes/client/util/exception/ObjectMetaReflectException.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.exception;
public class ObjectMetaReflectException extends ReflectiveOperationException {
public ObjectMetaReflectException(ReflectiveOperationException e) {
super(e);
```

}
}

io/kubernetes/client/util/exception/CSRNotApprovedException.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.exception;
public class CSRNotApprovedException extends Exception {
public CSRNotApprovedException(String msg) {
super(msg);
}
public CSRNotApprovedException(Exception e) {
super(e);
}
}
```

io/kubernetes/client/util/exception/IncompleteDiscoveryException.java

```
/*
Copyright 2023 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.exception;
import io.kubernetes.client.Discovery.APIResource;
import io.kubernetes.client.openapi.ApiException;
import java.util.Set;
public class IncompleteDiscoveryException extends ApiException {
```

```
private static final long serialVersionUID = 1L;
private final transient Set<APIResource> discoveredResources;
public IncompleteDiscoveryException(String message, ApiException cause, Set<APIResource>
discoveredResources) {
super(message, cause, cause.getCode(), cause.getResponseHeaders(), cause.getResponseBody());
this.discoveredResources = discoveredResources;
}
public Set<APIResource> getDiscoveredResources() {
return discoveredResources;
}
}
```

io/kubernetes/client/util/labels/LabelMatcher.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.labels;
/**
* The interface LabelValueMatcher is a boolean test checking whether a label value meets
* requirements.
*/
public interface LabelMatcher {
String getKey();
/**
* Returns true if a label value matches.
*
* @param s the label value
* @return the boolean
*/
boolean test(String s);
}
```

io/kubernetes/client/util/labels/LabelSelector.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
```

```
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.labels;
import io.kubernetes.client.openapi.models.V1LabelSelector;
import io.kubernetes.client.openapi.models.V1LabelSelectorRequirement;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.function.Predicate;
import java.util.stream.Collectors;
/**
* The type Label selector matches labels by requirements.
* https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/
*/
public class LabelSelector implements Predicate<Map<String, String>> {
public static final String LABEL_SELECTOR_OP_IN = "In";
public static final String LABEL_SELECTOR_OP_NOT_IN = "NotIn";
public static final String LABEL_SELECTOR_OP_EXISTS = "Exists";
public static final String LABEL_SELECTOR_OP_DOES_NOT_EXIST = "DoesNotExist";
LabelSelector(LabelMatcher... matchers) {
this.labelMatchers = matchers;
}
/**
* Constructs a new label selector from a conjunction of label matcher.
*
* @param matchers the label matchers
* @return the label selector
*/
public static LabelSelector and(LabelMatcher... matchers) {
return new LabelSelector(matchers);
}
/**
* Constructs a new label selector that matches everything.
*
* @return the empty label selector
*/
public static LabelSelector empty() {
return and();
}
/**
* Transform a V1LabelSelector to LabelSelector
*
* @param ps V1LabelSelector from spec
* @return the transformed LabelSelector
*/
public static LabelSelector parse(V1LabelSelector ps) throws IllegalArgumentException {
int matcherNum = 0;
if (ps.getMatchLabels() != null) matcherNum = ps.getMatchLabels().size();
```

```
if (ps.getMatchExpressions() != null) matcherNum = ps.getMatchExpressions().size();
List<LabelMatcher> matchers = new ArrayList<>(matcherNum);
if (ps.getMatchLabels() != null) {
for (Map.Entry<String, String> entry : ps.getMatchLabels().entrySet()) {
LabelMatcher m = EqualityMatcher.equal(entry.getKey(), entry.getValue());
matchers.add(m);
}
}
if (ps.getMatchExpressions() != null) {
for (V1LabelSelectorRequirement expr : ps.getMatchExpressions()) {
switch (expr.getOperator()) {
case LABEL_SELECTOR_OP_IN:
matchers.add(SetMatcher.in(expr.getKey(), expr.getValues().toArray(new String[0])));
break;
case LABEL_SELECTOR_OP_NOT_IN:
matchers.add(SetMatcher.notIn(expr.getKey(), expr.getValues().toArray(new String[0])));
break;
case LABEL_SELECTOR_OP_EXISTS:
matchers.add(SetMatcher.exists(expr.getKey()));
break;
case LABEL_SELECTOR_OP_DOES_NOT_EXIST:
matchers.add(SetMatcher.notExists(expr.getKey()));
break;
default:
throw new IllegalArgumentException(
expr.getOperator() + " is not a valid pod selector operator");
}
}
}
return LabelSelector.and(matchers.toArray(matchers.toArray(new LabelMatcher[0])));
}
private LabelMatcher[] labelMatchers;
@Override
public boolean test(Map<String, String> labels) {
return Arrays.stream(labelMatchers)
.allMatch(matcher -> matcher.test(labels.get(matcher.getKey())));
}
@Override
public String toString() {
return Arrays.stream(labelMatchers).map(m -> m.toString()).collect(Collectors.joining(","));
}
}
```

io/kubernetes/client/util/labels/EqualityMatcher.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
```

```
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.labels;
import java.util.function.Function;
import java.util.function.Predicate;
/**
* Equality-based matcher for label-selection.
*
https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/#equality-based-requirement
*/
public class EqualityMatcher implements LabelMatcher {
EqualityMatcher(String key, Operator operator, String value) {
this.key = key;
this.operator = operator;
this.value = value;
}
private Operator operator;
private String key;
private String value;
/**
* The "equal" matcher. Matches a label iff the label is present and equal.
*
* @param key the matching label key
* @param value the matching label value
* @return the equality matcher
*/
public static EqualityMatcher equal(String key, String value) {
return new EqualityMatcher(key, Operator.EQUAL, value);
}
/**
* The "not equal" matcher. Matches a label iff the label is not present or not equal.
*
* @param key the matching label key
* @param value the matching label value
* @return the equality matcher
*/
public static EqualityMatcher notEqual(String key, String value) {
return new EqualityMatcher(key, Operator.NOT_EQUAL, value);
}
@Override
public String toString() {
return new StringBuilder()
.append(key)
.append(" ")
.append(operator.name)
.append(" ")
.append(value)
.toString();
}
```

```
@Override
public boolean test(String s) {
return operator.with(value).test(s);
}
@Override
public String getKey() {
return key;
}
private enum Operator {
EQUAL(
"=",
arg -> {
return v -> arg.equals(v);
}),
DOUBLE_EQUAL(
"==",
arg -> {
return v -> arg.equals(v);
}),
NOT_EQUAL(
"!=",
arg -> {
return v -> !arg.equals(v);
});
Operator(String name, Function<String, Predicate<String>> matcherFunc) {
this.name = name;
this.matcherFunc = matcherFunc;
}
private final String name;
private final Function<String, Predicate<String>> matcherFunc;
Predicate<String> with(String value) {
return matcherFunc.apply(value);
}
}
}
```

io/kubernetes/client/util/labels/Labels.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.labels;
```

```
import io.kubernetes.client.common.KubernetesObject;
import java.util.HashMap;
import java.util.Map;
/** Helper utilities for manipulating kubernetes labels. */
public class Labels {
/**
* Adds one label to the object.
*
* @param kubernetesObject the kubernetes object
* @param label the label key
* @param value the label value
*/
public static void addLabels(KubernetesObject kubernetesObject, String label, String value) {
Map<String, String> mergingLabels = new HashMap<>();
mergingLabels.put(label, value);
addLabels(kubernetesObject, mergingLabels);
}
/**
* Adds a set of labels to the object.
*
* @param kubernetesObject the kubernetes object
* @param mergingLabels the merging labels
*/
public static void addLabels(
KubernetesObject kubernetesObject, Map<String, String> mergingLabels) {
if (null == mergingLabels) {
return;
}
final Map<String, String> currentLabels =
null != kubernetesObject.getMetadata().getLabels()
? kubernetesObject.getMetadata().getLabels()
: new HashMap<>();
mergingLabels.entrySet().stream().forEach(e -> currentLabels.put(e.getKey(), e.getValue()));
kubernetesObject.getMetadata().setLabels(currentLabels);
}
}
```

io/kubernetes/client/util/labels/SetMatcher.java


```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
```

```
package io.kubernetes.client.util.labels;
import java.util.Arrays;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.stream.Collectors;
/**
* Set-based matcher for label-selection.
* https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/#set-based-requirement
*/
public class SetMatcher implements LabelMatcher {
SetMatcher(String key, SetMatcher.Operator operator) {
this(key, operator, new String[] {});
}
/**
* Instantiates a new Set matcher.
*
* @param key the matching label key
* @param operator the matching label operator
* @param values the values
*/
SetMatcher(String key, SetMatcher.Operator operator, String[] values) {
this.key = key;
this.operator = operator;
this.values = values;
}
private SetMatcher.Operator operator;
private String key;
private String[] values;
/**
* The "in" set matcher. Matches if the label exists in the set.
*
* @param key the key
* @param values the values
* @return the set matcher
*/
public static SetMatcher in(String key, String... values) {
return new SetMatcher(key, Operator.IN, values);
}
/**
* The "not in" set matcher. Matches if the label doesn't exist in the set.
*
* @param key the key
* @param values the values
* @return the set matcher
*/
public static SetMatcher notIn(String key, String... values) {
return new SetMatcher(key, Operator.NOT_IN, values);
}
/**
* The "exists" set matcher. Matches if the label is present in the set.
*
* @param key the key
* @return the set matcher
```

```
*/
public static SetMatcher exists(String key) {
return new SetMatcher(key, Operator.EXISTS);
}
/**
* The "not exists" set matcher. Matches if the label is not present in the set.
*
* @param key the key
* @return the set matcher
*/
public static SetMatcher notExists(String key) {
return new SetMatcher(key, Operator.NOT_EXISTS);
}
@Override
public String toString() {
switch (operator) {
case IN:
case NOT_IN:
return new StringBuilder()
.append(key)
.append(" ")
.append(operator.name)
.append(" ")
.append("(")
.append(Arrays.stream(this.values).collect(Collectors.joining(",")))
.append(")")
.toString();
case EXISTS:
case NOT_EXISTS:
return operator.name + key;
default:
throw new IllegalStateException("unknown operator " + operator.name);
}
}
@Override
public String getKey() {
return key;
}
@Override
public boolean test(String s) {
return operator.with(values).test(s);
}
private enum Operator {
IN(
"in",
args -> {
return v -> contains(args, v);
}),
NOT_IN(
"notin",
args -> {
return v -> !contains(args, v);
}),
```

```
EXISTS(
"",
args -> {
return v -> v != null;
}),
NOT_EXISTS(
"!",
args -> {
return v -> v == null;
});
Operator(String name, Function<String[], Predicate<String>> matcherFunc) {
this.name = name;
this.matcherFunc = matcherFunc;
}
private final String name;
private final Function<String[], Predicate<String>> matcherFunc;
/**
* Gets name.
*
* @return the name
*/
public String getName() {
return name;
}
Predicate<String> with(String... values) {
return matcherFunc.apply(values);
}
private static boolean contains(String[] strs, String s) {
return Arrays.stream(strs).anyMatch(arg -> arg.equals(s));
}
}
}
```

io/kubernetes/client/util/ListAccessor.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.openapi.models.V1ListMeta;
import io.kubernetes.client.util.exception.ObjectMetaReflectException;
import java.lang.reflect.InvocationTargetException;
```

```
import java.lang.reflect.Method;
import java.util.List;
/** ListAccessor is a set of utils which accesses common fields from list objects dynamically */
public class ListAccessor {
private static final String METHOD_NAME_GET_ITEMS = "getItems";
private static final String METHOD_NAME_GET_LIST_METADATA = "getMetadata";
@Deprecated // use KubernetesListObject instead
public static V1ListMeta listMetadata(Object listObj) throws ObjectMetaReflectException {
try {
Method mdField = listObj.getClass().getMethod(METHOD_NAME_GET_LIST_METADATA);
return (V1ListMeta) mdField.invoke(listObj);
} catch (NoSuchMethodException | IllegalAccessException | InvocationTargetException e) {
throw new ObjectMetaReflectException(e);
}
}
@Deprecated // use KubernetesListObject instead
public static <ApiType> List<ApiType> getItems(Object listObj) throws ObjectMetaReflectException {
try {
Method getItemsMethod = listObj.getClass().getMethod(METHOD_NAME_GET_ITEMS);
return (List<ApiType>) getItemsMethod.invoke(listObj);
} catch (NoSuchMethodException | IllegalAccessException | InvocationTargetException e) {
throw new ObjectMetaReflectException(e);
}
}
}
```

io/kubernetes/client/util/credentials/ClientCertificateAuthentication.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.credentials;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.util.SSLUtils;
import java.io.IOException;
import java.security.KeyStoreException;
import java.security.NoSuchAlgorithmException;
import java.security.UnrecoverableKeyException;
import java.security.cert.CertificateException;
import java.security.spec.InvalidKeySpecException;
```

```
import javax.net.ssl.KeyManager;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/** Uses Client Certificates to configure {@link ApiClient} authentication to the Kubernetes API. */
public class ClientCertificateAuthentication implements Authentication {
private static final Logger log = LoggerFactory.getLogger(ClientCertificateAuthentication.class);
private final byte[] certificate;
private final byte[] key;
private String passphrase = "";
public ClientCertificateAuthentication(final byte[] certificate, final byte[] key) {
this.certificate = certificate;
this.key = key;
this.passphrase = "";
}
@Override
public void provide(ApiClient client) {
String algo = SSLUtils.recognizePrivateKeyAlgo(key);
try {
final KeyManager[] keyManagers =
SSLUtils.keyManagers(certificate, key, algo, getPassphrase(), null, null);
client.setKeyManagers(keyManagers);
} catch (NoSuchAlgorithmException
| UnrecoverableKeyException
| CertificateException
| KeyStoreException
| InvalidKeySpecException
| IOException e) {
log.warn("Could not create key manager for Client Certificate authentication.", e);
throw new RuntimeException(e);
}
}
public String getPassphrase() {
return passphrase;
}
public void setPassphrase(String passphrase) {
this.passphrase = passphrase;
}
}
```

io/kubernetes/client/util/credentials/RefreshAuthentication.java

```
/*
Copyright 2024 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
```

```
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.credentials;
import io.kubernetes.client.openapi.ApiClient;
import java.io.IOException;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.util.function.Supplier;
import okhttp3.Interceptor;
import okhttp3.OkHttpClient;
import okhttp3.Request;
import okhttp3.Response;
// TODO: prefer OpenAPI backed Auentication once it is available. see details in
// https://github.com/OpenAPITools/openapi-generator/pull/6036. currently, the
// workaround is to hijack the http request.
// TODO: Merge this with TokenFileAuthentication.
public class RefreshAuthentication implements Authentication, Interceptor {
private Instant expiry;
private Duration refreshPeriod;
private String token;
private Supplier<String> tokenSupplier;
private Clock clock;
public RefreshAuthentication(Supplier<String> tokenSupplier, Duration refreshPeriod) {
this(tokenSupplier, refreshPeriod, Clock.systemUTC());
}
public RefreshAuthentication(Supplier<String> tokenSupplier, Duration refreshPeriod, Clock clock) {
this.expiry = Instant.MIN;
this.refreshPeriod = refreshPeriod;
this.token = tokenSupplier.get();
this.tokenSupplier = tokenSupplier;
this.clock = clock;
}
private String getToken() {
if (Instant.now(this.clock).isAfter(this.expiry)) {
this.token = tokenSupplier.get();
expiry = Instant.now(this.clock).plusSeconds(refreshPeriod.toSeconds());
}
return this.token;
}
public Duration getRefreshPeriod() {
return this.refreshPeriod;
}
public void setExpiry(Instant expiry) {
this.expiry = expiry;
}
@Override
public void provide(ApiClient client) {
OkHttpClient withInterceptor = client.getHttpClient().newBuilder().addInterceptor(this).build();
client.setHttpClient(withInterceptor);
}
```

```
@Override
public Response intercept(Interceptor.Chain chain) throws IOException {
Request request = chain.request();
Request newRequest;
newRequest = request.newBuilder().header("Authorization", "Bearer " + getToken()).build();
return chain.proceed(newRequest);
}
}
```

io/kubernetes/client/util/credentials/DummyAuthentication.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.credentials;
import io.kubernetes.client.openapi.ApiClient;
class DummyAuthentication implements Authentication {
@Override
public void provide(ApiClient client) {}
}
```

io/kubernetes/client/util/credentials/EKSAuthentication.java

```
/*
Copyright 2023 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.credentials;
import com.amazonaws.DefaultRequest;
```

```
import com.amazonaws.auth.AWS4Signer;
import com.amazonaws.auth.AWSSessionCredentialsProvider;
import com.amazonaws.http.HttpMethodName;
import com.amazonaws.services.securitytoken.model.GetCallerIdentityRequest;
import com.amazonaws.util.RuntimeHttpUtils;
import io.kubernetes.client.openapi.ApiClient;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.net.URI;
import java.nio.charset.StandardCharsets;
import java.time.Clock;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.Base64;
import java.util.Date;
/**
* EKS cluster authentication which generates a bearer token from AWS AK/SK. It doesn't require an
"aws"
* command line tool in the $PATH.
*/
public class EKSAuthentication implements Authentication {
private static final Logger log = LoggerFactory.getLogger(EKSAuthentication.class);
/**
* Instantiates a new Eks authentication.
*
* @param provider the AWS credential provider
* @param region the region where EKS cluster at
* @param clusterName the EKS cluster name
*/
public EKSAuthentication(AWSSessionCredentialsProvider provider, String region, String clusterName)
{
this(provider, region, clusterName, MAX_EXPIRY_SECONDS);
}
public EKSAuthentication(AWSSessionCredentialsProvider provider, String region, String clusterName,
int expirySeconds) {
this.provider = provider;
this.region = region;
this.clusterName = clusterName;
if (expirySeconds > MAX_EXPIRY_SECONDS) {
expirySeconds = MAX_EXPIRY_SECONDS;
}
this.expirySeconds = expirySeconds;
this.stsEndpoint = URI.create("https://sts." + this.region + ".amazonaws.com");
}
private static final int MAX_EXPIRY_SECONDS = 60 * 15;
private final AWSSessionCredentialsProvider provider;
private final String region;
private final String clusterName;
private final URI stsEndpoint;
private final int expirySeconds;
@Override
public void provide(ApiClient client) {
DefaultRequest<GetCallerIdentityRequest> defaultRequest =
```

```
new DefaultRequest<>(new GetCallerIdentityRequest(), "sts");
defaultRequest.setResourcePath("/");
defaultRequest.setEndpoint(stsEndpoint);
defaultRequest.setHttpMethod(HttpMethodName.GET);
defaultRequest.addParameter("Action", "GetCallerIdentity");
defaultRequest.addParameter("Version", "2011-06-15");
defaultRequest.addHeader("x-k8s-aws-id", clusterName);
AWS4Signer signer = new AWS4Signer();
Date expirationTime = new Date(Clock.systemDefaultZone().millis() + 60 * 1000);
signer.setServiceName("sts");
signer.presignRequest(
defaultRequest,
this.provider.getCredentials(),
expirationTime);
String encodedUrl =
Base64.getUrlEncoder()
.withoutPadding()
.encodeToString( RuntimeHttpUtils.convertRequestToUrl(
defaultRequest, true, false).toString()
.getBytes(StandardCharsets.UTF_8));
String token = "k8s-aws-v1." + encodedUrl;
client.setApiKeyPrefix("Bearer");
client.setApiKey(token);
log.info("Generated BEARER token for ApiClient, expiring at {}", Instant.now().plus(expirySeconds,
ChronoUnit.SECONDS));
}
}
```

io/kubernetes/client/util/credentials/TokenFileAuthentication.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.credentials;
import io.kubernetes.client.openapi.ApiClient;
import java.io.IOException;
import java.nio.charset.Charset;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.time.Instant;
```

```
import okhttp3.Interceptor;
import okhttp3.OkHttpClient;
import okhttp3.Request;
import okhttp3.Response;
// TODO: prefer OpenAPI backed Auentication once it is available. see details in
// https://github.com/OpenAPITools/openapi-generator/pull/6036. currently, the
// workaround is to hijack the http request.
public class TokenFileAuthentication implements Authentication, Interceptor {
private String file;
private String token;
private Instant expiry;
public TokenFileAuthentication(String file) {
this.expiry = Instant.MIN;
this.file = file;
}
private String getToken() {
if (Instant.now().isAfter(this.expiry)) {
try {
this.token =
new String(Files.readAllBytes(Paths.get(this.file)), Charset.defaultCharset()).trim();
expiry = Instant.now().plusSeconds(60);
} catch (IOException ie) {
throw new RuntimeException("Cannot read file: " + this.file);
}
}
return this.token;
}
public void setExpiry(Instant expiry) {
this.expiry = expiry;
}
public void setFile(String file) {
this.file = file;
}
@Override
public void provide(ApiClient client) {
OkHttpClient withInterceptor = client.getHttpClient().newBuilder().addInterceptor(this).build();
client.setHttpClient(withInterceptor);
}
@Override
public Response intercept(Interceptor.Chain chain) throws IOException {
Request request = chain.request();
Request newRequest;
newRequest = request.newBuilder().header("Authorization", "Bearer " + getToken()).build();
return chain.proceed(newRequest);
}
}
```

io/kubernetes/client/util/credentials/AccessTokenAuthentication.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.credentials;
import io.kubernetes.client.openapi.ApiClient;
import java.util.Objects;
/** Uses a Bearer Token to configure {@link ApiClient} authentication to the Kubernetes API. */
public class AccessTokenAuthentication implements Authentication {
private String token;
public AccessTokenAuthentication(final String token) {
Objects.requireNonNull(token, "Access Token cannot be null");
this.token = token;
}
@Override
public void provide(ApiClient client) {
client.setApiKeyPrefix("Bearer");
client.setApiKey(token);
}
}
```

io/kubernetes/client/util/credentials/Authentication.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.credentials;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.util.ClientBuilder;
/**
* Allows the implementation of different authentication mechanisms for the Kubernetes API.
*
```

```
* @see ClientBuilder#setAuthentication(Authentication)
*/
public interface Authentication {
void provide(ApiClient client);
}
```

io/kubernetes/client/util/credentials/UsernamePasswordAuthentication.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util.credentials;
import io.kubernetes.client.openapi.ApiClient;
import java.nio.charset.StandardCharsets;
import okio.ByteString;
/**
* Uses Username and Password to configure {@link ApiClient} authentication to the Kubernetes API.
*/
public class UsernamePasswordAuthentication implements Authentication {
private final String username;
private final String password;
public UsernamePasswordAuthentication(final String username, final String password) {
this.username = username;
this.password = password;
}
@Override
public void provide(ApiClient client) {
final String usernameAndPassword = username + ":" + password;
client.setApiKeyPrefix("Basic");
client.setApiKey(
ByteString.of(usernameAndPassword.getBytes(StandardCharsets.ISO_8859_1)).base64());
}
}
```

io/kubernetes/client/util/credentials/KubeconfigAuthentication.java

```
/*
```

Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
```
*/
package io.kubernetes.client.util.credentials;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.util.KubeConfig;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.Map;
import org.apache.commons.lang3.StringUtils;
/**
* Uses a {@link KubeConfig} to configure {@link ApiClient} authentication to the Kubernetes API.
*
* <p>Tries to configure the following authentication mechanisms in this order.
*
* <ul>
* <li>{@link ClientCertificateAuthentication} (using client certificate files or data)
* <li>{@link UsernamePasswordAuthentication}
* <li>{@link AccessTokenAuthentication}
* </ul>
*/
public class KubeconfigAuthentication implements Authentication {
private final Authentication delegateAuthentication;
private Duration tokenRefreshPeriod;
public KubeconfigAuthentication(final KubeConfig config, Duration tokenRefreshPeriod) throws
IOException {
byte[] clientCert =
config.getDataOrFileRelative(
config.getClientCertificateData(), config.getClientCertificateFile());
byte[] clientKey =
config.getDataOrFileRelative(config.getClientKeyData(), config.getClientKeyFile());
// 1. honors x509 key-pairs
if (clientCert != null && clientKey != null) {
delegateAuthentication = new ClientCertificateAuthentication(clientCert, clientKey);
return;
}
// 2. honors username/password
String userName = config.getUsername();
String userPassword = config.getPassword();
if (StringUtils.isNotEmpty(userName) && StringUtils.isNotEmpty(userPassword)) {
delegateAuthentication = new UsernamePasswordAuthentication(userName, userPassword);
return;
}
// 3. honors bearer token or client certificate generated by exec command
```

```
Map<String, String> credentials = config.getCredentials();
if (credentials != null) {
if (StringUtils.isNotEmpty(credentials.get(KubeConfig.CRED_TOKEN_KEY))) {
delegateAuthentication =
tokenRefreshPeriod == null ?
new AccessTokenAuthentication(credentials.get(KubeConfig.CRED_TOKEN_KEY)) :
new RefreshAuthentication(() -> config.getCredentials().get(KubeConfig.CRED_TOKEN_KEY),
tokenRefreshPeriod);
return;
} else if (StringUtils.isNotEmpty(
credentials.get(KubeConfig.CRED_CLIENT_CERTIFICATE_DATA_KEY))
&& StringUtils.isNotEmpty(credentials.get(KubeConfig.CRED_CLIENT_KEY_DATA_KEY))) {
delegateAuthentication =
new ClientCertificateAuthentication(
credentials
.get(KubeConfig.CRED_CLIENT_CERTIFICATE_DATA_KEY)
.getBytes(StandardCharsets.UTF_8),
credentials
.get(KubeConfig.CRED_CLIENT_KEY_DATA_KEY)
.getBytes(StandardCharsets.UTF_8));
return;
}
}
// 4. falling back to dummy authentication
delegateAuthentication = new DummyAuthentication();
}
@Override
public void provide(ApiClient client) {
delegateAuthentication.provide(client);
}
public Authentication getDelegateAuthentication() {
return delegateAuthentication;
}
}
```

io/kubernetes/client/util/ObjectAccessor.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.openapi.models.V1ObjectMeta;
```

```
import io.kubernetes.client.util.exception.ObjectMetaReflectException;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
/** ObjectAccessor is a set of utils which accesses common fields from objects dynamically */
public class ObjectAccessor {
private static final String METHOD_NAME_GET_METADATA = "getMetadata";
@Deprecated // use KubernetesObject instead
public static V1ObjectMeta objectMetadata(Object obj) throws ObjectMetaReflectException {
try {
Method mdField = obj.getClass().getMethod(METHOD_NAME_GET_METADATA);
return (V1ObjectMeta) mdField.invoke(obj);
} catch (NoSuchMethodException | IllegalAccessException | InvocationTargetException e) {
throw new ObjectMetaReflectException(e);
}
}
@Deprecated // use KubernetesObject instead
public static String namespace(Object obj) throws ObjectMetaReflectException {
return objectMetadata(obj).getNamespace();
}
@Deprecated // use KubernetesObject instead
public static String name(Object obj) throws ObjectMetaReflectException {
return objectMetadata(obj).getName();
}
}
```

io/kubernetes/client/util/ModelMapper.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.util;
import io.kubernetes.client.Discovery;
import io.kubernetes.client.apimachinery.GroupVersionKind;
import io.kubernetes.client.apimachinery.GroupVersionResource;
import io.kubernetes.client.common.KubernetesListObject;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.util.exception.IncompleteDiscoveryException;
import io.kubernetes.client.util.generic.dynamic.DynamicKubernetesObject;
import java.io.File;
import java.io.IOException;
import java.net.JarURLConnection;
```

```
import java.net.URI;
import java.net.URL;
import java.net.URLDecoder;
import java.time.Duration;
import java.util.ArrayList;
import java.util.Enumeration;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.jar.JarEntry;
import java.util.jar.JarFile;
import org.apache.commons.lang3.tuple.ImmutablePair;
import org.apache.commons.lang3.tuple.Pair;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/**
* Model mapper loads pre-built kubernetes models from classpath and manages their mapping
between
* their apiVersion and kind.
*/
public class ModelMapper {
private static final Logger logger = LoggerFactory.getLogger(ModelMapper.class);
public static final Duration DEFAULT_DISCOVERY_REFRESH_INTERVAL = Duration.ofMinutes(30);
// initially loaded from classpath, can be incorrect
private static Map<GroupVersionKind, Class<?>> preBuiltClassesByGVK = new HashMap<>();
// Model's api-group prefix to kubernetes api-group
private static Map<String, String> preBuiltApiGroups = new HashMap<>();
// Model's api-version midfix to kubernetes api-version
private static List<String> preBuiltApiVersions = new ArrayList<>();
private static BiDirectionalMap<GroupVersionKind, Class<?>> classesByGVK =
new BiDirectionalMap<>();
private static BiDirectionalMap<GroupVersionResource, Class<?>> classesByGVR =
new BiDirectionalMap<>();
private static Map<Class<?>, Boolean> isNamespacedByClasses = new ConcurrentHashMap<>();
private static Set<Discovery.APIResource> lastAPIDiscovery = new HashSet<>();
// zero means api-discovery never done
private static volatile long nextDiscoveryRefreshTimeMillis = 0;
static {
try {
initModelMap();
} catch (Exception ex) {
logger.error("Unexpected exception while loading classes", ex);
}
}
/**
* Registering concrete model classes by its apiGroupVersion (e.g. "apps/v1") and kind (e.g.
* "Deployment").
*
* <p>Note that the the so-called apiGroupVersion equals to the "apiVersion" in the kubenretes
```

```
* resource yamls.
*
* @deprecated This method is deprecated. Please use the alternative method with the correct
* parameters. For example, use {@link #addModelMap(String, String, String, Class)}
* instead.
*/
@Deprecated
public static void addModelMap(String apiGroupVersion, String kind, Class<?> clazz) {
String[] parts = apiGroupVersion.split("/");
if (parts.length <= 1) { // legacy api group
addModelMap("", apiGroupVersion, kind, clazz);
} else {
addModelMap(parts[0], parts[1], kind, clazz);
}
}
/**
* Registering concrete model classes by its group, version and kind (e.g. "apps", "v1",
* "Deployment")
*
* @param group the group
* @param version the version
* @param kind the kind
* @param clazz the clazz
* @deprecated This method is deprecated and will be removed in a future version.
* Use the non-deprecated method(s) instead.
*/
@Deprecated
public static void addModelMap(String group, String version, String kind, Class<?> clazz) {
preBuiltAddModelMap(group, version, kind, clazz);
}
/**
* Registering concrete model classes by its group, version and kind (e.g. "apps", "v1",
* "Deployment")
*
* @param group the group
* @param version the version
* @param kind the kind
* @param resourceNamePlural the resource name plural
* @param objClass the clazz
*/
public static void addModelMap(
String group,
String version,
String kind,
String resourceNamePlural,
Class<? extends KubernetesObject> objClass,
Class<? extends KubernetesListObject> objListClass) {
// TODO(yue9944882): consistency between bi-directional maps
classesByGVK.add(new GroupVersionKind(group, version, kind), objClass);
classesByGVR.add(new GroupVersionResource(group, version, resourceNamePlural), objClass);
if (objListClass != null) {
classesByGVK.add(new GroupVersionKind(group, version, kind + "List"), objListClass);
}
```

```
}
/**
* Registering concrete model classes by its group, version, kind and isNamespaced (e.g. "apps",
* "v1", "Deployment", true).
*
* @param group the group
* @param version the version
* @param kind the kind
* @param resourceNamePlural the resource name plural
* @param isNamespacedResource the is namespaced resource
* @param objClass the clazz
*/
public static void addModelMap(
String group,
String version,
String kind,
String resourceNamePlural,
Boolean isNamespacedResource,
Class<? extends KubernetesObject> objClass) {
addModelMap(group, version, kind, resourceNamePlural, objClass, null);
isNamespacedByClasses.put(objClass, isNamespacedResource);
}
public static void addModelMap(
String group,
String version,
String kind,
String resourceNamePlural,
Boolean isNamespacedResource,
Class<? extends KubernetesObject> objClass,
Class<? extends KubernetesListObject> objListClass) {
addModelMap(group, version, kind, resourceNamePlural, objClass, objListClass);
isNamespacedByClasses.put(objClass, isNamespacedResource);
}
/**
* Gets the model classes by given apiGroupVersion (e.g. "apps/v1") and kind (e.g. "Deployment").
*
* @param apiGroupVersion the api version
* @param kind the kind
* @return the api type class
*/
public static Class<?> getApiTypeClass(String apiGroupVersion, String kind) {
String[] parts = apiGroupVersion.split("/");
String apiGroup;
String apiVersion;
if (parts.length == 1) {
// legacy group
apiGroup = "";
apiVersion = apiGroupVersion;
} else {
apiGroup = parts[0];
apiVersion = parts[1];
}
return getApiTypeClass(apiGroup, apiVersion, kind);
```

```
}
/**
* Gets the model classes by given group, version and kind (e.g. "apps", ""v1", "Deployment").
*
* @param group the group
* @param version the version
* @param kind the kind
* @return the api type class
*/
public static Class<?> getApiTypeClass(String group, String version, String kind) {
Class<?> clazz = classesByGVK.getByK(new GroupVersionKind(group, version, kind));
if (clazz != null) {
return clazz;
}
clazz = preBuiltGetApiTypeClass(group, version, kind);
if (clazz != null) {
return clazz;
}
return DynamicKubernetesObject.class;
}
/**
* Gets the GVK by the given model class.
*
* @param clazz the clazz
* @return the group version kind by class
*/
public static GroupVersionKind getGroupVersionKindByClass(Class<?> clazz) {
return classesByGVK.getByV(clazz);
}
/**
* Gets the GVK by the given model class.
*
* @param clazz the clazz
* @return the group version kind by class
*/
public static GroupVersionResource getGroupVersionResourceByClass(Class<?> clazz) {
return classesByGVR.getByV(clazz);
}
/**
* Refreshes the model mapping by syncing up w/the api discovery info from the kubernetes
* apiserver. These mapping will be cached for {@link
* ModelMapper#DEFAULT_DISCOVERY_REFRESH_INTERVAL}.
*
* @param discovery the discovery
* @return the set
* @throws ApiException the api exception
*/
public static Set<Discovery.APIResource> refresh(Discovery discovery) throws ApiException {
return refresh(discovery, DEFAULT_DISCOVERY_REFRESH_INTERVAL);
}
/**
* Refreshes the model mapping by syncing up w/the api discovery info from the kubernetes
* apiserver.
```

```
*
* <p>Note: if model mappings can be incomplete if this method is never called.
*
* @param discovery the discovery
* @return the api-discovery set
* @throws ApiException the api exception
*/
public static Set<Discovery.APIResource> refresh(Discovery discovery, Duration refreshInterval)
throws ApiException {
long nowMillis = System.currentTimeMillis();
if (nowMillis < nextDiscoveryRefreshTimeMillis) {
return lastAPIDiscovery;
}
Set<Discovery.APIResource> apiResources = null;
try {
apiResources = discovery.findAll();
} catch (IncompleteDiscoveryException e) {
logger.warn("Error while getting all api resources, some api resources will not be refreshed", e);
apiResources = e.getDiscoveredResources();
}
for (Discovery.APIResource apiResource : apiResources) {
for (String version : apiResource.getVersions()) {
Class<? extends KubernetesObject> objClass =
(Class<? extends KubernetesObject>)
getApiTypeClass(apiResource.getGroup(), version, apiResource.getKind());
// no such classes registered in the ModelMapper, ignoring
if (objClass == null) {
continue;
}
Class<? extends KubernetesListObject> objListClass =
(Class<? extends KubernetesListObject>)
getApiTypeClass(apiResource.getGroup(), version, apiResource.getKind() + "List");
// sync up w/ the latest api discovery
addModelMap(
apiResource.getGroup(),
version,
apiResource.getKind(),
apiResource.getResourcePlural(),
apiResource.getNamespaced(),
objClass,
objListClass);
}
}
lastAPIDiscovery = apiResources;
nextDiscoveryRefreshTimeMillis = refreshInterval.toMillis() + nowMillis;
return lastAPIDiscovery;
}
public static boolean isApiDiscoveryRefreshed() {
return nextDiscoveryRefreshTimeMillis != 0;
}
static void preBuiltAddModelMap(String group, String version, String kind, Class<?> clazz) {
preBuiltClassesByGVK.put(new GroupVersionKind(group, version, kind), clazz);
}
```

```
public static Class<?> preBuiltGetApiTypeClass(String group, String version, String kind) {
Class<?> clazz = preBuiltClassesByGVK.get(new GroupVersionKind(group, version, kind));
if (clazz != null) {
return clazz;
}
return preBuiltClassesByGVK.get(new GroupVersionKind("", version, kind));
}
public static Optional<GroupVersionKind> preBuiltGetGroupVersionKindByClass(Class<?> clazz) {
return preBuiltClassesByGVK.entrySet().stream()
.filter(e -> clazz.equals(e.getValue()))
.map(Map.Entry::getKey)
.findFirst();
}
/**
* Checks whether the class is connected with a namespaced kubernetes resource.
*
* @param clazz the clazz
* @return the boolean
*/
public static Boolean isNamespaced(Class<?> clazz) {
return isNamespacedByClasses.get(clazz);
}
/**
* Find GroupVersionKind from fields in a Kubernetes Resource
*
* @param apiVersion The apiVersion field from the object
* @param kind The kind for the object
* @return the GroupVersionKind
*/
public static GroupVersionKind groupVersionKindFromApiVersionAndKind(
String apiVersion, String kind) {
int ix = apiVersion.indexOf("/");
String group = ix == -1 ? "" : apiVersion.substring(0, ix);
String version = apiVersion.substring(ix + 1);
return new GroupVersionKind(group, version, kind);
}
public static Discovery.APIResource findApiResourceByGroupVersionKind(GroupVersionKind gvk) {
Map<String, Discovery.APIResource> apiResourcesByKey = new HashMap<>();
for (Discovery.APIResource apiResource : lastAPIDiscovery) {
for (String version : apiResource.getVersions()) {
String key = apiResource.getKind() + "/" + apiResource.getGroup() + "/" + version;
apiResourcesByKey.put(key, apiResource);
}
}
String key = gvk.getKind() + "/" + gvk.getGroup() + "/" + gvk.getVersion();
return apiResourcesByKey.get(key);
}
private static void initApiGroupMap() {
preBuiltApiGroups.put("Admissionregistration", "admissionregistration.k8s.io");
preBuiltApiGroups.put("Apiextensions", "apiextensions.k8s.io");
preBuiltApiGroups.put("Apiregistration", "apiregistration.k8s.io");
preBuiltApiGroups.put("Apps", "apps");
preBuiltApiGroups.put("Authentication", "authentication.k8s.io");
```

```
preBuiltApiGroups.put("Authorization", "authorization.k8s.io");
preBuiltApiGroups.put("Autoscaling", "autoscaling");
preBuiltApiGroups.put("Batch", "batch");
preBuiltApiGroups.put("Certificates", "certificates.k8s.io");
preBuiltApiGroups.put("Core", "");
preBuiltApiGroups.put("Extensions", "extensions");
preBuiltApiGroups.put("Events", "events.k8s.io");
preBuiltApiGroups.put("FlowControl", "flowcontrol.apiserver.k8s.io");
preBuiltApiGroups.put("Networking", "networking.k8s.io");
preBuiltApiGroups.put("Policy", "policy");
preBuiltApiGroups.put("RbacAuthorization", "rbac.authorization.k8s.io");
preBuiltApiGroups.put("Scheduling", "scheduling.k8s.io");
preBuiltApiGroups.put("Settings", "settings.k8s.io");
preBuiltApiGroups.put("Storage", "storage.k8s.io");
}
private static void initApiVersionList() {
// Order important
preBuiltApiVersions.add("V2beta1");
preBuiltApiVersions.add("V2beta2");
preBuiltApiVersions.add("V2alpha1");
preBuiltApiVersions.add("V1beta2");
preBuiltApiVersions.add("V1beta1");
preBuiltApiVersions.add("V1alpha1");
preBuiltApiVersions.add("V1");
preBuiltApiVersions.add("V2");
}
private static void initModelMap() throws IOException {
initApiGroupMap();
initApiVersionList();
for (String classInfo :
getClassNamesFromPackage(
Yaml.class.getClassLoader(), "io.kubernetes.client.openapi.models")) {
Class<?> clazz = loadClass(classInfo, Yaml.class.getClassLoader());
if (!KubernetesObject.class.isAssignableFrom(clazz)
&& !KubernetesListObject.class.isAssignableFrom(clazz)) {
continue;
}
Pair<String, String> groupAndOther = getApiGroup(clazz.getSimpleName());
Pair<String, String> versionAndOther = getApiVersion(groupAndOther.getRight());
String group = Strings.nullToEmpty(groupAndOther.getLeft());
String version = versionAndOther.getLeft();
String kind = versionAndOther.getRight();
preBuiltClassesByGVK.put(new GroupVersionKind(group, version, kind), clazz);
}
if (preBuiltClassesByGVK.size() == 0) {
logger.warn("No kubernetes api model classes found from classloader, " + "this may break automatic
api discovery");
}
}
private static Class<?> loadClass(String name, ClassLoader classLoader) {
try {
return Class.forName(name, false, classLoader);
} catch (ClassNotFoundException e) {
```

```
return null;
}
}
private static Pair<String, String> getApiGroup(String name) {
return preBuiltApiGroups.entrySet().stream()
.filter(e -> name.startsWith(e.getKey()))
.map(e -> new ImmutablePair<>(e.getValue(), name.substring(e.getKey().length())))
.findFirst()
.orElse(new ImmutablePair<>(null, name));
}
private static Pair<String, String> getApiVersion(String name) {
return preBuiltApiVersions.stream()
.filter(v -> name.startsWith(v))
.map(v -> new ImmutablePair<>(v.toLowerCase(), name.substring(v.length())))
.findFirst()
.orElse(new ImmutablePair<>(null, name));
}
static class BiDirectionalMap<K, V> {
private Map<K, V> kvMap = new ConcurrentHashMap<>();
private Map<V, K> vkMap = new ConcurrentHashMap<>();
void add(K k, V v) {
kvMap.put(k, v);
vkMap.put(v, k);
}
V getByK(K k) {
return kvMap.get(k);
}
K getByV(V v) {
return vkMap.get(v);
}
}
private static List<String> getClassNamesFromPackage(ClassLoader classLoader, String pkg) throws
IOException {
ArrayList<String> names = new ArrayList<>();
String packageName = pkg.replace(".", "/");
URL packageURL = classLoader.getResource(packageName);
if (packageURL.getProtocol().equals("jar")) {
processJarPackage(packageURL, packageName, pkg, names);
} else {
processFilePackage(packageURL, pkg, names);
}
return names;
}
private static void processJarPackage(URL packageURL, String packageName, String pkg,
ArrayList<String> names) throws IOException {
String jarFileName = URLDecoder.decode(packageURL.getFile(), "UTF-8");
JarFile jf = null;
// jar: client in repository; nested: client in a fat jar
if (jarFileName.startsWith("jar:") || jarFileName.startsWith("nested:")) {
jf = ((JarURLConnection) packageURL.openConnection()).getJarFile();
}
// file: client is a file in target (unit test)
if (jarFileName.startsWith("file:") ) {
```

```
jarFileName = jarFileName.substring(5, jarFileName.indexOf("!"));
jf = new JarFile(jarFileName);
}
if (jf == null) {
logger.error("Loading classes from jar with error packageURL: {}", jarFileName);
return;
}
logger.info("Loading classes from jar {}", jarFileName);
Enumeration<JarEntry> jarEntries = jf.entries();
while (jarEntries.hasMoreElements()) {
processJarEntry(jarEntries.nextElement(), packageName, pkg, names);
}
jf.close();
}
private static void processJarEntry(JarEntry jarEntry, String packageName, String pkg,
ArrayList<String> names) {
String entryName = jarEntry.getName();
if (entryName.startsWith(packageName) && entryName.length() > packageName.length() + 5) {
entryName = entryName.substring(packageName.length() + 1, entryName.lastIndexOf('.'));
names.add(pkg + "." + entryName);
}
}
private static void processFilePackage(URL packageURL, String pkg, ArrayList<String> names) {
URI uri = URI.create(packageURL.toString());
File folder = new File(uri.getPath());
File[] contenuti = folder.listFiles();
if (contenuti == null) {
logger.warn("No files to load found in {}", folder.getPath());
return;
}
for (File actual : contenuti) {
processFileEntry(actual, pkg, names);
}
}
private static void processFileEntry(File actual, String pkg, ArrayList<String> names) {
String entryName = actual.getName();
entryName = entryName.substring(0, entryName.lastIndexOf('.'));
names.add(pkg + "." + entryName);
}
}
```

io/kubernetes/client/Attach.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
```

```
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.Configuration;
import io.kubernetes.client.openapi.models.V1Pod;
import io.kubernetes.client.util.WebSocketStreamHandler;
import io.kubernetes.client.util.WebSockets;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
public class Attach {
private ApiClient apiClient;
/** Simple Attach API constructor, uses default configuration */
public Attach() {
this(Configuration.getDefaultApiClient());
}
/**
* Attach API Constructor
*
* @param apiClient The api client to use.
*/
public Attach(ApiClient apiClient) {
this.apiClient = apiClient;
}
/**
* Get the API client for these Attach operations.
*
* @return The API client that will be used.
*/
public ApiClient getApiClient() {
return apiClient;
}
/**
* Set the API client for subsequent Attach operations.
*
* @param apiClient The new API client to use.
*/
public void setApiClient(ApiClient apiClient) {
this.apiClient = apiClient;
}
/**
* Setup a Builder for the given namespace and name
*
* @param namespace The namespace of the Pod
* @param name The name of the Pod
*/
public ConnectionBuilder newConnectionBuilder(String namespace, String name) {
return new ConnectionBuilder(namespace, name);
}
```

```
/**
* Attach to a running AttachResult in a container. If there are multiple containers in the pod,
* uses the first container in the Pod.
*
* @param namespace The namespace of the Pod
* @param name The name of the Pod
* @param stdin If true, pass a stdin stream into the container
*/
public AttachResult attach(String namespace, String name, boolean stdin)
throws ApiException, IOException {
return attach(namespace, name, null, stdin, false);
}
/**
* Attach to a running AttachResult in a container. If there are multiple containers in the pod,
* uses the first container in the Pod.
*
* @param pod The pod where the command is run.
* @param stdin If true, pass a stdin stream into the container
*/
public AttachResult attach(V1Pod pod, boolean stdin) throws ApiException, IOException {
return attach(pod, stdin, false);
}
/**
* Attach to a running AttachResult in a container. If there are multiple containers in the pod,
* uses the first container in the Pod.
*
* @param pod The pod where the command is run.
* @param stdin If true, pass a stdin stream into the container
* @param tty If true, stdin is a tty.
*/
public AttachResult attach(V1Pod pod, boolean stdin, boolean tty)
throws ApiException, IOException {
return attach(pod, null, stdin, tty);
}
/**
* Attach to a running AttachResult in a container. If there are multiple containers in the pod,
* uses the first container in the Pod.
*
* @param pod The pod where the command is run.
* @param container The container in the Pod where the command is run.
* @param stdin If true, pass a stdin stream into the container.
* @param tty If true, stdin is a TTY (only applies if stdin is true)
*/
public AttachResult attach(V1Pod pod, String container, boolean stdin, boolean tty)
throws ApiException, IOException {
return attach(
pod.getMetadata().getNamespace(), pod.getMetadata().getName(), container, stdin, tty);
}
/**
* Attach to a running AttachResult in a container. If there are multiple containers in the pod,
* uses the first container in the Pod.
*
* @param namespace The namespace of the Pod
```

```
* @param name The name of the Pod
* @param container The container in the Pod where the command is run.
* @param stdin If true, pass a stdin stream into the container.
* @param tty If true, stdin is a TTY (only applies if stdin is true)
*/
public AttachResult attach(
String namespace, String name, String container, boolean stdin, boolean tty)
throws ApiException, IOException {
return newConnectionBuilder(namespace, name)
.setContainer(container)
.setStdin(stdin)
.setTty(tty)
.connect();
}
public final class ConnectionBuilder {
private final String namespace;
private final String name;
private String container;
private boolean stdin;
private boolean stdout;
private boolean stderr;
private boolean tty;
private ConnectionBuilder(String namespace, String name) {
this.namespace = namespace;
this.name = name;
this.stdin = true;
}
public String getName() {
return name;
}
public String getNamespace() {
return namespace;
}
public String getContainer() {
return container;
}
public ConnectionBuilder setContainer(String container) {
this.container = container;
return this;
}
public boolean getStdin() {
return stdin;
}
public ConnectionBuilder setStdin(boolean stdin) {
this.stdin = stdin;
return this;
}
public boolean getStdout() {
return stdout;
}
public ConnectionBuilder setStdout(boolean stdout) {
this.stdout = stdout;
return this;
```

```
}
public boolean getStderr() {
return stderr;
}
public ConnectionBuilder setStderr(boolean stderr) {
this.stderr = stderr;
return this;
}
public boolean getTty() {
return tty;
}
public ConnectionBuilder setTty(boolean tty) {
this.tty = tty;
return this;
}
private String makePath() {
return "/api/v1/namespaces/"
+ namespace
+ "/pods/"
+ name
+ "/attach?"
+ "stdin="
+ stdin
+ "&stdout="
+ stdout
+ "&stderr="
+ stderr
+ "&tty="
+ tty
+ (container != null ? "&container=" + container : "");
}
public AttachResult connect() throws ApiException, IOException {
WebSocketStreamHandler handler = new WebSocketStreamHandler();
AttachResult result = new AttachResult(handler);
WebSockets.stream(makePath(), "GET", apiClient, handler);
return result;
}
}
/**
* AttachResult contains the result of an Attach call, it includes streams for stdout stderr and
* stdin.
*/
public static class AttachResult implements java.io.Closeable {
private WebSocketStreamHandler handler;
public AttachResult(WebSocketStreamHandler handler) throws IOException {
this.handler = handler;
}
public OutputStream getStandardInputStream() {
return handler.getOutputStream(0);
}
public InputStream getStandardOutputStream() throws IOException {
return handler.getInputStream(1);
}
```

```
public InputStream getErrorStream() {
return handler.getInputStream(2);
}
public InputStream getConnectionErrorStream() {
return handler.getInputStream(3);
}
public OutputStream getResizeStream() {
return handler.getOutputStream(4);
}
public void close() {
handler.close();
}
}
}
```

io/kubernetes/client/PortForward.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.Configuration;
import io.kubernetes.client.openapi.Pair;
import io.kubernetes.client.openapi.models.V1Pod;
import io.kubernetes.client.util.WebSocketStreamHandler;
import io.kubernetes.client.util.WebSockets;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
/**
* Utility class for setting up port-forwarding connections. Uses the WebSockets API, not the SPDY
* API (which the Go client uses)
*
* <p>The protocol is undocumented as far as I can tell, but the PR that added it is here:
* github.com/kubernetes/kubernetes/pull/33684
*
* <p>And the protocol is:
```

```
*
* <p>web sockets on server/api/v1/namespaces/&lt;namespace&gt;/pods/&lt;pod&gt;/portforward with
* args ports=80 and ports=8080
*
* <p>I/O for first port (80) is on Channel 0 Err for first port (80) is on Channel 1 I/O for second
* port (8080) is on Channel 2 Err for second port (8080) is on Channel 3 < and so on for remaining
* ports>
*
* <p>The first two bytes of each output stream is the port that is being forwarded in little-endian
* format.
*/
public class PortForward {
private ApiClient apiClient;
/** Simple PortForward API constructor, uses default configuration */
public PortForward() {
this(Configuration.getDefaultApiClient());
}
/**
* PortForward API Constructor
*
* @param apiClient The api client to use.
*/
public PortForward(ApiClient apiClient) {
this.apiClient = apiClient;
}
/**
* Get the API client for these PortForward operations.
*
* @return The API client that will be used.
*/
public ApiClient getApiClient() {
return apiClient;
}
/**
* Set the API client for subsequent PortForward operations.
*
* @param apiClient The new API client to use.
*/
public void setApiClient(ApiClient apiClient) {
this.apiClient = apiClient;
}
private String makePath(String namespace, String name) {
return "/api/v1/namespaces/" + namespace + "/pods/" + name + "/portforward";
}
/**
* PortForward to a container
*
* @param pod The pod where the port forward is run.
* @param ports The ports to forward
* @return The result of the Port Forward request.
*/
public PortForwardResult forward(V1Pod pod, List<Integer> ports)
throws ApiException, IOException {
```

```
return forward(pod.getMetadata().getNamespace(), pod.getMetadata().getName(), ports);
}
/**
* PortForward to a container.
*
* @param namespace The namespace of the Pod
* @param name The name of the Pod
* @param ports The ports to forward
* @return The result of the Port Forward request.
*/
public PortForwardResult forward(String namespace, String name, List<Integer> ports)
throws ApiException, IOException {
String path = makePath(namespace, name);
WebSocketStreamHandler handler = new WebSocketStreamHandler();
PortForwardResult result = new PortForwardResult(handler, ports);
List<Pair> queryParams = new ArrayList<>(ports.size());
for (Integer port : ports) {
queryParams.add(new Pair("ports", port.toString()));
}
WebSockets.stream(path, "GET", queryParams, apiClient, handler);
try {
handler.waitForInitialized();
} catch (InterruptedException ex) {
throw new ApiException(ex);
}
Throwable err = handler.getError();
if (err != null) {
throw new ApiException(err);
}
// Wait for streams to start.
result.init();
return result;
}
/**
* PortForwardResult contains the result of an Attach call, it includes streams for stdout stderr
* and stdin.
*/
public static class PortForwardResult {
private WebSocketStreamHandler handler;
private HashMap<Integer, Integer> streams;
private List<Integer> ports;
/**
* Constructor
*
* @param handler The web socket handler
* @param ports The list of ports that are being forwarded.
*/
public PortForwardResult(WebSocketStreamHandler handler, List<Integer> ports)
throws IOException {
this.handler = handler;
this.streams = new HashMap<>();
this.ports = ports;
}
```

```
/** Initialize the connection. Must be called after the web socket has been opened. */
public void init() throws IOException {
for (int i = 0; i < ports.size(); i++) {
InputStream is = handler.getInputStream(i * 2);
byte[] data = new byte[2];
int readAmount = is.read(data);
if (readAmount != 2) {
throw new IOException("Failed to read port");
}
int port = (data[0] & 0xFF) + (data[1] & 0xFF) * 256;
streams.put(port, i);
}
}
private int findPortIndex(int portNumber) {
Integer ix = streams.get(portNumber);
if (ix == null) {
return -1;
}
return ix.intValue();
}
/**
* Get the output stream for the specified port number (e.g. 80)
*
* @param port The port number to get the stream for.
* @return The OutputStream for the specified port, null if there is no such port.
*/
public OutputStream getOutboundStream(int port) {
int portIndex = findPortIndex(port);
if (portIndex == -1) {
return null;
}
return handler.getOutputStream(portIndex * 2);
}
/**
* Get the error stream for a port number (e.g. 80)
*
* @param port The port number to get the stream for.
* @return The error stream, or null if there is no such port.
*/
public InputStream getErrorStream(int port) {
int portIndex = findPortIndex(port);
if (portIndex == -1) {
return null;
}
return handler.getInputStream(portIndex * 2 + 1);
}
/**
* Get the input stream for a port number (e.g. 80)
*
* @param port The port number to get the stream for.
* @return The input stream, or null if no such port exists.
*/
public InputStream getInputStream(int port) throws IOException {
```

```
int portIndex = findPortIndex(port);
if (portIndex == -1) {
return null;
}
return handler.getInputStream(portIndex * 2);
}
}
}
```

io/kubernetes/client/TreeNode.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client;
import com.google.gson.Gson;
import java.util.ArrayList;
import java.util.List;
/** Helper class to represent directory tree */
public class TreeNode {
boolean isDir;
String name;
boolean isRoot;
List<TreeNode> children;
public TreeNode(boolean isDir, String name, boolean isRoot) {
this.isDir = isDir;
this.name = name;
this.isRoot = isRoot;
this.children = new ArrayList<>();
}
public boolean isDir() {
return isDir;
}
public void setDir(boolean dir) {
isDir = dir;
}
public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}
```

```
public boolean isRoot() {
return isRoot;
}
public void setRoot(boolean root) {
isRoot = root;
}
public List<TreeNode> getChildren() {
return children;
}
public void setChildren(List<TreeNode> children) {
this.children = children;
}
@Override
public String toString() {
return new Gson().toJson(this).toString();
}
}
```

io/kubernetes/client/persister/FilePersister.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.persister;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.HashMap;
import io.kubernetes.client.persister.ConfigPersister;
import org.yaml.snakeyaml.LoaderOptions;
import org.yaml.snakeyaml.Yaml;
import org.yaml.snakeyaml.constructor.SafeConstructor;
public class FilePersister implements ConfigPersister {
File configFile;
public FilePersister(String filename) {
this(new File(filename));
}
public FilePersister(File file) {
this.configFile = file;
}
public void save(
```

```
ArrayList<Object> contexts,
ArrayList<Object> clusters,
ArrayList<Object> users,
Object preferences,
String currentContext)
throws IOException {
HashMap<String, Object> config = new HashMap<>();
config.put("apiVersion", "v1");
config.put("kind", "Config");
config.put("current-context", currentContext);
config.put("preferences", preferences);
config.put("clusters", clusters);
config.put("contexts", contexts);
config.put("users", users);
// Note this is imperfect, should protect against other processes writing this file too...
synchronized (configFile) {
try (FileWriter fw = new FileWriter(configFile)) {
Yaml yaml = new Yaml(new SafeConstructor(new LoaderOptions()));
yaml.dump(config, fw);
fw.flush();
}
}
}
}
```

io/kubernetes/client/persister/ConfigPersister.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.persister;
import java.io.IOException;
import java.util.ArrayList;
public interface ConfigPersister {
void save(
ArrayList<Object> contexts,
ArrayList<Object> clusters,
ArrayList<Object> users,
Object preferences,
String currentContext)
throws IOException;
}
```

io/kubernetes/client/informer/ResourceEventHandler.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer;
import io.kubernetes.client.common.KubernetesObject;
public interface ResourceEventHandler<ApiType extends KubernetesObject> {
void onAdd(ApiType obj);
void onUpdate(ApiType oldObj, ApiType newObj);
void onDelete(ApiType obj, boolean deletedFinalStateUnknown);
}
```

io/kubernetes/client/informer/SharedInformerFactory.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer;
import com.google.gson.reflect.TypeToken;
import io.kubernetes.client.common.KubernetesListObject;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.informer.cache.Cache;
import io.kubernetes.client.informer.impl.DefaultSharedIndexInformer;
import io.kubernetes.client.openapi.ApiClient;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.Configuration;
```

```
import io.kubernetes.client.util.CallGenerator;
import io.kubernetes.client.util.CallGeneratorParams;
import io.kubernetes.client.util.Namespaces;
import io.kubernetes.client.util.Watch;
import io.kubernetes.client.util.Watchable;
import io.kubernetes.client.util.generic.GenericKubernetesApi;
import io.kubernetes.client.util.generic.options.ListOptions;
import java.lang.reflect.Type;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.function.BiConsumer;
import okhttp3.Call;
import org.apache.commons.collections4.MapUtils;
/** SharedInformerFactory class constructs and caches informers for api types. */
public class SharedInformerFactory {
protected Map<Type, SharedIndexInformer> informers;
private Map<Type, Future> startedInformers;
private ExecutorService informerExecutor;
private ApiClient apiClient;
/** Constructor w/ default thread pool. */
/** DEPRECATE: In favor of explicit apiClient constructor to avoid misguiding */
@Deprecated
public SharedInformerFactory() {
this(Configuration.getDefaultApiClient().setReadTimeout(0), Executors.newCachedThreadPool());
}
/** Constructor w/ api client specified and default thread pool. */
public SharedInformerFactory(ApiClient apiClient) {
this(apiClient, Executors.newCachedThreadPool());
}
/**
* Constructor w/ thread pool specified.
*
* @param threadPool specified thread pool
*/
public SharedInformerFactory(ExecutorService threadPool) {
this(Configuration.getDefaultApiClient().setReadTimeout(0), threadPool);
}
/**
* Constructor w/ api client and thread pool specified.
*
* @param client specific api client
* @param threadPool specified thread pool
*/
public SharedInformerFactory(ApiClient client, ExecutorService threadPool) {
if (client.getReadTimeout() != 0) {
throw new IllegalArgumentException("read timeout of ApiClient must be zero");
}
apiClient = client;
informerExecutor = threadPool;
informers = new HashMap<>();
```

```
startedInformers = new HashMap<>();
}
/**
* Shared index informer for shared index informer.
*
* @param <ApiType> the type parameter
* @param <ApiListType> the type parameter
* @param callGenerator the call generator
* @param apiTypeClass the api type class
* @param apiListTypeClass the api list type class
* @return the shared index informer
*/
public synchronized <ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
SharedIndexInformer<ApiType> sharedIndexInformerFor(
CallGenerator callGenerator,
Class<ApiType> apiTypeClass,
Class<ApiListType> apiListTypeClass) {
return sharedIndexInformerFor(callGenerator, apiTypeClass, apiListTypeClass, 0);
}
/**
* Constructs and returns a shared index informer w/ resync period specified. But the informer
* cache will not be overwritten i.e. only the first registered informer will be kept.
*
* @param <ApiType> the type parameter
* @param <ApiListType> the type parameter
* @param callGenerator the call generator
* @param apiTypeClass the api type class
* @param apiListTypeClass the api list type class
* @param resyncPeriodInMillis the resync period in millis
* @return the shared index informer
*/
public synchronized <ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
SharedIndexInformer<ApiType> sharedIndexInformerFor(
CallGenerator callGenerator,
Class<ApiType> apiTypeClass,
Class<ApiListType> apiListTypeClass,
long resyncPeriodInMillis) {
return sharedIndexInformerFor(
callGenerator, apiTypeClass, apiListTypeClass, resyncPeriodInMillis, null);
}
/**
* Shared index informer for shared index informer.
*
* @param <ApiType> the type parameter
* @param <ApiListType> the type parameter
* @param callGenerator the call generator
* @param apiTypeClass the api type class
* @param apiListTypeClass the api list type class
* @param exceptionHandler the exception Handler
* @return the shared index informer
*/
public synchronized <ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
SharedIndexInformer<ApiType> sharedIndexInformerFor(
```

```
CallGenerator callGenerator,
Class<ApiType> apiTypeClass,
Class<ApiListType> apiListTypeClass,
long resyncPeriodInMillis,
BiConsumer<Class<ApiType>, Throwable> exceptionHandler) {
ListerWatcher<ApiType, ApiListType> listerWatcher =
listerWatcherFor(callGenerator, apiTypeClass, apiListTypeClass);
return sharedIndexInformerFor(
listerWatcher, apiTypeClass, resyncPeriodInMillis, exceptionHandler);
}
/**
* Constructs and returns a shared index informer by specifying lister-watcher. But the informer
* cache will not be overwritten on multiple call w/ the the same apiTypeClass i.e. only the first
* registered informer will be kept.
*
* @param <ApiType> the type parameter
* @param <ApiListType> the type parameter
* @param listerWatcher the lister watcher
* @param apiTypeClass the api type class
* @param resyncPeriodInMillis the resync period in millis
* @return the shared index informer
*/
public synchronized <ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
SharedIndexInformer<ApiType> sharedIndexInformerFor(
ListerWatcher<ApiType, ApiListType> listerWatcher,
Class<ApiType> apiTypeClass,
long resyncPeriodInMillis) {
return sharedIndexInformerFor(listerWatcher, apiTypeClass, resyncPeriodInMillis, null);
}
public synchronized <ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
SharedIndexInformer<ApiType> sharedIndexInformerFor(
ListerWatcher<ApiType, ApiListType> listerWatcher,
Class<ApiType> apiTypeClass,
long resyncPeriodInMillis,
BiConsumer<Class<ApiType>, Throwable> exceptionHandler) {
SharedIndexInformer<ApiType> informer =
new DefaultSharedIndexInformer<>(
apiTypeClass, listerWatcher, resyncPeriodInMillis, new Cache<>(), exceptionHandler);
this.informers.putIfAbsent(TypeToken.get(apiTypeClass).getType(), informer);
return informer;
}
/**
* Constructs and returns a shared index informer by specifying a generic api instance. But the
* informer cache will not be overwritten on multiple call w/ the the same apiTypeClass i.e. only
* the first registered informer will be kept.
*
* @param <ApiType> the type parameter
* @param <ApiListType> the type parameter
* @param genericKubernetesApi the generic kubernetes api
* @param apiTypeClass the api type class
* @param resyncPeriodInMillis the resync period in millis
* @return the shared index informer
*/
```

```
public synchronized <ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
SharedIndexInformer<ApiType> sharedIndexInformerFor(
GenericKubernetesApi<ApiType, ApiListType> genericKubernetesApi,
Class<ApiType> apiTypeClass,
long resyncPeriodInMillis) {
return sharedIndexInformerFor(
genericKubernetesApi, apiTypeClass, resyncPeriodInMillis, Namespaces.NAMESPACE_ALL);
}
/**
* Working the same as {@link SharedInformerFactory#sharedIndexInformerFor} above.
*
* <p>Constructs and returns a shared index informer for a specific namespace.
*
* @param <ApiType> the type parameter
* @param <ApiListType> the type parameter
* @param genericKubernetesApi the generic kubernetes api
* @param apiTypeClass the api type class
* @param resyncPeriodInMillis the resync period in millis
* @param namespace the target namespace
* @return the shared index informer
*/
public synchronized <ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
SharedIndexInformer<ApiType> sharedIndexInformerFor(
GenericKubernetesApi<ApiType, ApiListType> genericKubernetesApi,
Class<ApiType> apiTypeClass,
long resyncPeriodInMillis,
String namespace) {
return sharedIndexInformerFor(
genericKubernetesApi, apiTypeClass, resyncPeriodInMillis, namespace, null);
}
/**
* Working the same as {@link SharedInformerFactory#sharedIndexInformerFor} above.
*
* <p>Constructs and returns a shared index informer for a specific namespace.
*
* @param <ApiType> the type parameter
* @param <ApiListType> the type parameter
* @param genericKubernetesApi the generic kubernetes api
* @param apiTypeClass the api type class
* @param resyncPeriodInMillis the resync period in millis
* @param namespace the target namespace
* @param exceptionHandler the exception Handler
* @return the shared index informer
*/
public synchronized <ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
SharedIndexInformer<ApiType> sharedIndexInformerFor(
GenericKubernetesApi<ApiType, ApiListType> genericKubernetesApi,
Class<ApiType> apiTypeClass,
long resyncPeriodInMillis,
String namespace,
BiConsumer<Class<ApiType>, Throwable> exceptionHandler) {
ListerWatcher<ApiType, ApiListType> listerWatcher =
listerWatcherFor(genericKubernetesApi, namespace);
```

```
return sharedIndexInformerFor(
listerWatcher, apiTypeClass, resyncPeriodInMillis, exceptionHandler);
}
private <ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
ListerWatcher<ApiType, ApiListType> listerWatcherFor(
CallGenerator callGenerator,
Class<ApiType> apiTypeClass,
Class<ApiListType> apiListTypeClass) {
if (apiClient.getReadTimeout() > 0) {
// set read timeout zero to ensure client doesn't time out
apiClient.setReadTimeout(0);
}
return new ListerWatcher<ApiType, ApiListType>() {
@Override
public ApiListType list(CallGeneratorParams params) throws ApiException {
Call call = callGenerator.generate(params);
return (ApiListType) apiClient.execute(call, apiListTypeClass).getData();
}
@Override
public Watch<ApiType> watch(CallGeneratorParams params) throws ApiException {
Call call = callGenerator.generate(params);
// bind call with private http client to make sure read timeout is zero.
call = apiClient.getHttpClient().newCall(call.request());
return Watch.createWatch(
apiClient,
call,
TypeToken.getParameterized(Watch.Response.class, apiTypeClass).getType());
}
};
}
private <ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
ListerWatcher<ApiType, ApiListType> listerWatcherFor(
GenericKubernetesApi<ApiType, ApiListType> genericKubernetesApi, String namespace) {
if (apiClient.getReadTimeout() > 0) {
// set read timeout zero to ensure client doesn't time out
apiClient.setReadTimeout(0);
}
// TODO: it seems read timeout is determined by genericKubernetesApi instead of above apiClient.
return new ListerWatcher<ApiType, ApiListType>() {
public ApiListType list(CallGeneratorParams params) throws ApiException {
if (Namespaces.NAMESPACE_ALL.equals(namespace)) {
return genericKubernetesApi
.list(
new ListOptions() {
{
setResourceVersion(params.resourceVersion);
setTimeoutSeconds(params.timeoutSeconds);
}
})
.throwsApiException()
.getObject();
} else {
return genericKubernetesApi
```

```
.list(
namespace,
new ListOptions() {
{
setResourceVersion(params.resourceVersion);
setTimeoutSeconds(params.timeoutSeconds);
}
})
.throwsApiException()
.getObject();
}
}
public Watchable<ApiType> watch(CallGeneratorParams params) throws ApiException {
if (Namespaces.NAMESPACE_ALL.equals(namespace)) {
return genericKubernetesApi.watch(
new ListOptions() {
{
setResourceVersion(params.resourceVersion);
setTimeoutSeconds(params.timeoutSeconds);
}
});
} else {
return genericKubernetesApi.watch(
namespace,
new ListOptions() {
{
setResourceVersion(params.resourceVersion);
setTimeoutSeconds(params.timeoutSeconds);
}
});
}
}
};
}
/**
* Gets existing shared index informer, return null if the requesting informer is never
* constructed.
*
* @param <ApiType> the type parameter
* @param apiTypeClass the api type class
* @return the existing shared index informer
*/
public synchronized <ApiType extends KubernetesObject>
SharedIndexInformer<ApiType> getExistingSharedIndexInformer(Class<ApiType> apiTypeClass) {
return this.informers.get(TypeToken.get(apiTypeClass).getType());
}
/** Start all registered informers. */
public synchronized void startAllRegisteredInformers() {
if (MapUtils.isEmpty(informers)) {
return;
}
informers.forEach(
(informerType, informer) ->
```

```
startedInformers.computeIfAbsent(
informerType, key -> informerExecutor.submit((Runnable) informer::run)));
}
/** Stop all registered informers and shut down the thread pool. */
public synchronized void stopAllRegisteredInformers() {
stopAllRegisteredInformers(true);
}
/**
* Stop all registered informers.
*
* @param shutdownThreadPool whether or not to shut down the thread pool.
*/
public synchronized void stopAllRegisteredInformers(boolean shutdownThreadPool) {
if (MapUtils.isEmpty(informers)) {
return;
}
informers.forEach(
(informerType, informer) -> {
if (startedInformers.remove(informerType) != null) {
informer.stop();
}
});
if (shutdownThreadPool) {
informerExecutor.shutdown();
}
}
}
```

io/kubernetes/client/informer/SharedIndexInformer.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.informer.cache.Indexer;
import java.util.List;
import java.util.Map;
import java.util.function.Function;
/*
```

```
* SharedIndexInformer extends SharedInformer and provides indexer operability additionally.
*/
public interface SharedIndexInformer<ApiType extends KubernetesObject>
extends SharedInformer<ApiType> {
/**
* Add indexers.
*
* @param indexers the indexers
*/
void addIndexers(Map<String, Function<ApiType, List<String>>> indexers);
/**
* getIndexer returns the internal indexer store.
*
* @return the internal indexer store
*/
Indexer<ApiType> getIndexer();
}
```

io/kubernetes/client/informer/ListerWatcher.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer;
import io.kubernetes.client.common.KubernetesListObject;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.util.CallGeneratorParams;
import io.kubernetes.client.util.Watchable;
public interface ListerWatcher<
ApiType extends KubernetesObject, ApiListType extends KubernetesListObject> {
ApiListType list(CallGeneratorParams params) throws ApiException;
Watchable<ApiType> watch(CallGeneratorParams params) throws ApiException;
}
```

io/kubernetes/client/informer/impl/DefaultSharedIndexInformer.java

```
/*
Copyright 2020 The Kubernetes Authors.
```

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
```
package io.kubernetes.client.informer.impl;
import io.kubernetes.client.common.KubernetesListObject;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.informer.ListerWatcher;
import io.kubernetes.client.informer.ResourceEventHandler;
import io.kubernetes.client.informer.SharedIndexInformer;
import io.kubernetes.client.informer.TransformFunc;
import io.kubernetes.client.informer.cache.Cache;
import io.kubernetes.client.informer.cache.Controller;
import io.kubernetes.client.informer.cache.DeltaFIFO;
import io.kubernetes.client.informer.cache.Indexer;
import io.kubernetes.client.informer.cache.ProcessorListener;
import io.kubernetes.client.informer.cache.SharedProcessor;
import io.kubernetes.client.util.Threads;
import java.util.Deque;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ThreadFactory;
import java.util.function.BiConsumer;
import java.util.function.Function;
import org.apache.commons.collections4.CollectionUtils;
import org.apache.commons.lang3.tuple.MutablePair;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class DefaultSharedIndexInformer<
ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
implements SharedIndexInformer<ApiType> {
private static final Logger log = LoggerFactory.getLogger(DefaultSharedIndexInformer.class);
private static final long MINIMUM_RESYNC_PERIOD_MILLIS = 1000L;
// resyncCheckPeriod is how often we want the reflector's resync timer to fire so it can call
// shouldResync to check if any of our listeners need a resync.
private long resyncCheckPeriodMillis;
// defaultEventHandlerResyncPeriod is the default resync period for any handlers added via
// AddEventHandler (i.e. they don't specify one and just want to use the shared informer's
// default
// value).
private long defaultEventHandlerResyncPeriod;
private Indexer<ApiType> indexer;
private SharedProcessor<ApiType> processor;
private Controller<ApiType, ApiListType> controller;
private Thread controllerThread;
private TransformFunc transform;
private volatile boolean started = false;
```

```
private volatile boolean stopped = false;
public DefaultSharedIndexInformer(
Class<ApiType> apiTypeClass,
ListerWatcher<ApiType, ApiListType> listerWatcher,
long resyncPeriod) {
this(apiTypeClass, listerWatcher, resyncPeriod, new Cache<>());
}
public DefaultSharedIndexInformer(
Class<ApiType> apiTypeClass,
ListerWatcher<ApiType, ApiListType> listerWatcher,
long resyncPeriod,
Cache<ApiType> cache) {
this(
apiTypeClass,
listerWatcher,
resyncPeriod,
// down-casting should be safe here because one delta FIFO instance only serves one
// resource
// type
new DeltaFIFO(
(Function<KubernetesObject, String>) cache.getKeyFunc(),
(Cache<KubernetesObject>) cache),
cache,
null);
}
public DefaultSharedIndexInformer(
Class<ApiType> apiTypeClass,
ListerWatcher<ApiType, ApiListType> listerWatcher,
long resyncPeriod,
Cache<ApiType> cache,
BiConsumer<Class<ApiType>, Throwable> exceptionHandler) {
this(
apiTypeClass,
listerWatcher,
resyncPeriod,
// down-casting should be safe here because one delta FIFO instance only serves one
// resource
// type
new DeltaFIFO(
(Function<KubernetesObject, String>) cache.getKeyFunc(),
(Cache<KubernetesObject>) cache),
cache,
exceptionHandler);
}
public DefaultSharedIndexInformer(
Class<ApiType> apiTypeClass,
ListerWatcher<ApiType, ApiListType> listerWatcher,
long resyncPeriod,
DeltaFIFO deltaFIFO,
Indexer<ApiType> indexer) {
this(apiTypeClass, listerWatcher, resyncPeriod, deltaFIFO, indexer, null);
}
public DefaultSharedIndexInformer(
```

```
Class<ApiType> apiTypeClass,
ListerWatcher<ApiType, ApiListType> listerWatcher,
long resyncPeriod,
DeltaFIFO deltaFIFO,
Indexer<ApiType> indexer,
BiConsumer<Class<ApiType>, Throwable> exceptionHandler) {
this.resyncCheckPeriodMillis = resyncPeriod;
this.defaultEventHandlerResyncPeriod = resyncPeriod;
this.processor = new SharedProcessor<>();
this.indexer = indexer;
this.controller =
new Controller<>(
apiTypeClass,
deltaFIFO,
listerWatcher,
this::handleDeltas,
processor::shouldResync,
resyncCheckPeriodMillis,
exceptionHandler);
ThreadFactory threadFactory =
Threads.threadFactory("informer-controller-" + apiTypeClass.getSimpleName());
controllerThread = threadFactory.newThread(controller::run);
}
/** add event callback */
@Override
public void addEventHandler(ResourceEventHandler<ApiType> handler) {
addEventHandlerWithResyncPeriod(handler, defaultEventHandlerResyncPeriod);
}
/** add event callback with a resync period */
@Override
public void addEventHandlerWithResyncPeriod(
ResourceEventHandler<ApiType> handler, long resyncPeriodMillis) {
if (stopped) {
log.info(
"DefaultSharedIndexInformer#Handler was not added to shared informer because it has stopped
already");
return;
}
if (resyncPeriodMillis > 0) {
if (resyncPeriodMillis < MINIMUM_RESYNC_PERIOD_MILLIS) {
log.warn(
"DefaultSharedIndexInformer#resyncPeriod {} is too small. Changing it to the minimum allowed rule of
{}",
resyncPeriodMillis,
MINIMUM_RESYNC_PERIOD_MILLIS);
resyncPeriodMillis = MINIMUM_RESYNC_PERIOD_MILLIS;
}
if (resyncPeriodMillis < this.resyncCheckPeriodMillis) {
if (started) {
log.warn(
"DefaultSharedIndexInformer#resyncPeriod {} is smaller than resyncCheckPeriod {} and the informer
has already started. Changing it to {}",
resyncPeriodMillis,
```

```
resyncCheckPeriodMillis,
resyncCheckPeriodMillis);
resyncPeriodMillis = resyncCheckPeriodMillis;
} else {
// if the event handler's resyncPeriod is smaller than the current
// resyncCheckPeriod,
// update resyncCheckPeriod to match resyncPeriod and adjust the resync periods
// of all
// the listeners accordingly
this.resyncCheckPeriodMillis = resyncPeriodMillis;
}
}
}
ProcessorListener<ApiType> listener =
new ProcessorListener(
handler, determineResyncPeriod(resyncCheckPeriodMillis, this.resyncCheckPeriodMillis));
if (!started) {
this.processor.addListener(listener);
return;
}
this.processor.addAndStartListener(listener);
List<ApiType> objectList = this.indexer.list();
for (Object item : objectList) {
listener.add(new ProcessorListener.AddNotification(item));
}
}
@Override
public String lastSyncResourceVersion() {
if (!started) {
return "";
}
return this.controller.lastSyncResourceVersion();
}
@Override
public void setTransform(TransformFunc transformFunc) {
if (started) {
throw new IllegalStateException("cannot set transform func to a running informer");
}
this.transform = transformFunc;
}
@Override
public void run() {
if (started) {
return;
}
started = true;
this.processor.run();
controllerThread.start();
}
@Override
public void stop() {
if (!started) {
return;
```

```
}
stopped = true;
controller.stop();
controllerThread.interrupt();
processor.stop();
}
@Override
public boolean hasSynced() {
// Waiting for controller initialize.
// Because informer.run() and hasSynced() are called by different threads.
return controller != null && this.controller.hasSynced();
}
/**
* handleDeltas handles deltas and call processor distribute.
*
* @param deltas deltas
*/
public void handleDeltas(Deque<MutablePair<DeltaFIFO.DeltaType, KubernetesObject>> deltas) {
if (CollectionUtils.isEmpty(deltas)) {
return;
}
// from oldest to newest
for (MutablePair<DeltaFIFO.DeltaType, KubernetesObject> delta : deltas) {
DeltaFIFO.DeltaType deltaType = delta.getLeft();
KubernetesObject obj = delta.getRight();
if (transform != null) {
obj = transform.transform(obj);
}
switch (deltaType) {
case Sync:
case Added:
case Updated:
boolean isSync = deltaType == DeltaFIFO.DeltaType.Sync;
Object oldObj = this.indexer.get((ApiType) obj);
if (oldObj != null) {
this.indexer.update((ApiType) obj);
this.processor.distribute(
new ProcessorListener.UpdateNotification(oldObj, obj), isSync);
} else {
this.indexer.add((ApiType) obj);
this.processor.distribute(new ProcessorListener.AddNotification(obj), isSync);
}
break;
case Deleted:
this.indexer.delete((ApiType) obj);
this.processor.distribute(new ProcessorListener.DeleteNotification(obj), false);
break;
}
}
}
@Override
public void addIndexers(Map<String, Function<ApiType, List<String>>> indexers) {
if (started) {
```

```
throw new IllegalStateException("cannot add indexers to a running informer");
}
indexer.addIndexers(indexers);
}
@Override
public Indexer getIndexer() {
return this.indexer;
}
private long determineResyncPeriod(long desired, long check) {
if (desired == 0) {
return desired;
}
if (check == 0) {
return 0;
}
return Math.max(desired, check);
}
}
```

io/kubernetes/client/informer/cache/DeltaFIFO.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.cache;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.openapi.models.V1ObjectMeta;
import java.util.ArrayList;
import java.util.Deque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.NoSuchElementException;
import java.util.Set;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;
import java.util.function.Consumer;
import java.util.function.Function;
import org.apache.commons.collections4.CollectionUtils;
```

```
import org.apache.commons.lang3.StringUtils;
import org.apache.commons.lang3.tuple.MutablePair;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
// DeltaFIFO is a java portable of k/client-go's DeltaFIFO
public class DeltaFIFO {
private static final Logger log = LoggerFactory.getLogger(DeltaFIFO.class);
private Function<KubernetesObject, String> keyFunc;
// `items` maps keys to Deltas.
private Map<String, Deque<MutablePair<DeltaType, KubernetesObject>>> items;
// `queue` maintains FIFO order of keys for consumption in Pop().
// We maintain the property that keys in the `items` and `queue` are
// strictly 1:1 mapping, and that all Deltas in `items` should have
// at least one Delta.
private Deque<String> queue;
// knownObjects list keys that are "known" --- affecting Delete(),
// Replace(), and Resync()
private Store<? extends KubernetesObject> knownObjects;
// populated is true if the first batch of items inserted by Replace() has
// been populated or Delete/Add/Update was called first.
private boolean populated = false;
// initialPopulationCount is the number of items inserted by the first call
// of Replace()
private int initialPopulationCount;
/** lock provides thread safety * */
private ReadWriteLock lock = new ReentrantReadWriteLock();
/** indicates if the store is empty * */
private Condition notEmpty;
/**
* Constructor.
*
* @param keyFunc the key func
* @param knownObjects the known objects
*/
public DeltaFIFO(
Function<KubernetesObject, String> keyFunc, Store<? extends KubernetesObject> knownObjects) {
this.keyFunc = keyFunc;
this.knownObjects = knownObjects;
this.items = new HashMap<>();
this.queue = new LinkedList<>();
this.notEmpty = lock.writeLock().newCondition();
}
/**
* Add items to the delta FIFO.
*
* @param obj the obj
*/
public void add(KubernetesObject obj) {
lock.writeLock().lock();
try {
populated = true;
this.queueActionLocked(DeltaType.Added, obj);
} finally {
```

```
lock.writeLock().unlock();
}
}
/**
* Update items in the delta FIFO.
*
* @param obj the obj
*/
public void update(KubernetesObject obj) {
lock.writeLock().lock();
try {
populated = true;
this.queueActionLocked(DeltaType.Updated, obj);
} finally {
lock.writeLock().unlock();
}
}
/**
* Delete items from the delta FIFO.
*
* @param obj the obj
*/
public void delete(KubernetesObject obj) {
String id = this.keyOf(obj);
lock.writeLock().lock();
try {
this.populated = true;
if (this.knownObjects == null) {
if (!this.items.containsKey(id)) {
// Presumably, this was deleted when a relist happened.
// Don't provide a second report of the same deletion.
return;
}
} else {
// We only want to skip the "deletion" action if the object doesn't
// exist in knownObjects and it doesn't have corresponding item in items.
if (this.knownObjects.getByKey(id) == null && !this.items.containsKey(id)) {
return;
}
}
this.queueActionLocked(DeltaType.Deleted, obj);
} finally {
lock.writeLock().unlock();
}
}
/**
* Replace the item forcibly.
*
* @param list the list
* @param resourceVersion the resource version
*/
public void replace(List<KubernetesObject> list, String resourceVersion) {
lock.writeLock().lock();
```

```
try {
Set<String> keys = new HashSet<>();
for (KubernetesObject obj : list) {
String key = this.keyOf(obj);
keys.add(key);
this.queueActionLocked(DeltaType.Sync, obj);
}
if (this.knownObjects == null) {
for (Map.Entry<String, Deque<MutablePair<DeltaType, KubernetesObject>>> entry :
this.items.entrySet()) {
if (keys.contains(entry.getKey())) {
continue;
}
KubernetesObject deletedObj = null;
MutablePair<DeltaType, KubernetesObject> delta =
entry.getValue().peekLast(); // get newest
if (delta != null) {
deletedObj = delta.getRight();
}
this.queueActionLocked(
DeltaType.Deleted, new DeletedFinalStateUnknown(entry.getKey(), deletedObj));
}
if (!this.populated) {
this.populated = true;
this.initialPopulationCount = list.size();
}
return;
}
// Detect deletions not already in the queue.
List<String> knownKeys = this.knownObjects.listKeys();
int queueDeletion = 0;
for (String knownKey : knownKeys) {
if (keys.contains(knownKey)) {
continue;
}
KubernetesObject deletedObj = this.knownObjects.getByKey(knownKey);
if (deletedObj == null) {
log.warn(
"Key {} does not exist in known objects store, placing DeleteFinalStateUnknown marker without object",
knownKey);
}
queueDeletion++;
this.queueActionLocked(
DeltaType.Deleted, new DeletedFinalStateUnknown(knownKey, deletedObj));
}
if (!this.populated) {
this.populated = true;
this.initialPopulationCount = list.size() + queueDeletion;
}
} finally {
lock.writeLock().unlock();
}
}
```

```
/**
* Re-sync the delta FIFO. First, It locks the queue to block any more write operation until it
* finishes processing all the pending items in the queue.
*/
public void resync() {
lock.writeLock().lock();
try {
if (this.knownObjects == null) {
return;
}
List<String> keys = this.knownObjects.listKeys();
for (String key : keys) {
syncKeyLocked(key);
}
} finally {
lock.writeLock().unlock();
}
}
/**
* List keys list.
*
* @return the list
*/
public List<String> listKeys() {
lock.readLock().lock();
try {
List<String> keyList = new ArrayList<>(items.size());
for (Map.Entry<String, Deque<MutablePair<DeltaType, KubernetesObject>>> entry :
items.entrySet()) {
keyList.add(entry.getKey());
}
return keyList;
} finally {
lock.readLock().unlock();
}
}
/**
* Get object.
*
* @param obj the obj
* @return the object
*/
public Deque<MutablePair<DeltaType, KubernetesObject>> get(KubernetesObject obj) {
String key = this.keyOf(obj);
return this.getByKey(key);
}
/**
* Gets get by key.
*
* @param key the key
* @return the get by key
*/
public Deque<MutablePair<DeltaType, KubernetesObject>> getByKey(String key) {
```

```
lock.readLock().lock();
try {
Deque<MutablePair<DeltaType, KubernetesObject>> deltas = this.items.get(key);
if (deltas != null) {
// returning a shallow copy
return new LinkedList<>(deltas);
}
} finally {
lock.readLock().unlock();
}
return null;
}
/**
* List list.
*
* @return the list
*/
public List<Deque<MutablePair<DeltaType, KubernetesObject>>> list() {
lock.readLock().lock();
List<Deque<MutablePair<DeltaType, KubernetesObject>>> objects = new ArrayList<>();
try {
// TODO: make a generic deep copy utility
for (Map.Entry<String, Deque<MutablePair<DeltaType, KubernetesObject>>> entry :
items.entrySet()) {
Deque<MutablePair<DeltaType, KubernetesObject>> copiedDeltas =
new LinkedList<>(entry.getValue());
objects.add(copiedDeltas);
}
} finally {
lock.readLock().unlock();
}
return objects;
}
/**
* Pop deltas.
*
* @param func the func
* @return the deltas
* @throws Exception the exception
*/
public Deque<MutablePair<DeltaType, KubernetesObject>> pop(
Consumer<Deque<MutablePair<DeltaType, KubernetesObject>>> func) throws InterruptedException {
lock.writeLock().lock();
try {
while (true) {
while (queue.isEmpty()) {
notEmpty.await();
}
// there should have data now
String id = this.queue.removeFirst();
if (this.initialPopulationCount > 0) {
this.initialPopulationCount--;
}
```

```
if (!this.items.containsKey(id)) {
// Item may have been deleted subsequently.
continue;
}
Deque<MutablePair<DeltaType, KubernetesObject>> deltas = this.items.get(id);
this.items.remove(id);
func.accept(deltas);
// Don't make any copyDeltas here
return deltas;
}
} finally {
lock.writeLock().unlock();
}
}
/**
* Has synced boolean.
*
* @return the boolean
*/
public boolean hasSynced() {
lock.readLock().lock();
try {
return this.populated && this.initialPopulationCount == 0;
} finally {
lock.readLock().unlock();
}
}
/** queueActionLocked appends to the delta list for the object. Caller must hold the lock. */
private void queueActionLocked(DeltaType actionType, KubernetesObject obj) {
String id = this.keyOf(obj);
Deque<MutablePair<DeltaType, KubernetesObject>> deltas = items.get(id);
if (deltas == null) {
deltas = new LinkedList<>();
deltas.add(new MutablePair(actionType, obj));
} else {
deltas.add(new MutablePair<DeltaType, KubernetesObject>(actionType, obj));
}
Deque<MutablePair<DeltaType, KubernetesObject>> combinedDeltaList = combineDeltas(deltas);
boolean exist = items.containsKey(id);
if (combinedDeltaList != null && combinedDeltaList.size() > 0) {
if (!exist) {
this.queue.add(id);
}
this.items.put(id, combinedDeltaList);
notEmpty.signalAll();
} else {
this.items.remove(id);
}
}
// KeyOf exposes f's keyFunc, but also detects the key of a Deltas object or
// DeletedFinalStateUnknown objects.
private String keyOf(KubernetesObject obj) {
KubernetesObject innerObj = obj;
```

```
if (obj instanceof Deque) {
Deque<MutablePair<DeltaType, KubernetesObject>> deltas =
(Deque<MutablePair<DeltaType, KubernetesObject>>) obj;
if (deltas.size() == 0) {
throw new NoSuchElementException("0 length Deltas object; can't get key");
}
innerObj = deltas.peekLast().getRight();
}
if (innerObj instanceof DeletedFinalStateUnknown) {
return ((DeletedFinalStateUnknown) innerObj).key;
}
return keyFunc.apply(innerObj);
}
/** Add Sync delta. Caller must hold the lock. */
private void syncKeyLocked(String key) {
KubernetesObject obj = this.knownObjects.getByKey(key);
if (obj == null) {
return;
}
String id = this.keyOf(obj);
Deque<MutablePair<DeltaType, KubernetesObject>> deltas = this.items.get(id);
if (deltas != null && !(CollectionUtils.isEmpty(deltas))) {
return;
}
this.queueActionLocked(DeltaType.Sync, obj);
}
// re-listing and watching can deliver the same update multiple times in any
// order. This will combine the most recent two deltas if they are the same.
private Deque<MutablePair<DeltaType, KubernetesObject>> combineDeltas(
Deque<MutablePair<DeltaType, KubernetesObject>> deltas) {
if (deltas.size() < 2) {
return deltas;
}
int size = deltas.size();
MutablePair<DeltaType, KubernetesObject> d1 = deltas.pollLast();
MutablePair<DeltaType, KubernetesObject> d2 = deltas.pollLast();
MutablePair<DeltaType, KubernetesObject> out = isDuplicate(d1, d2);
if (out != null) {
deltas.add(out);
} else {
deltas.add(d2);
deltas.add(d1);
}
return deltas;
}
/**
* If d1 & d2 represent the same event, returns the delta that ought to be kept.
*
* @param d1 the elder one
* @param d2 the most one
* @return the one ought to be kept
*/
private MutablePair<DeltaType, KubernetesObject> isDuplicate(
```

```
MutablePair<DeltaType, KubernetesObject> d1, MutablePair<DeltaType, KubernetesObject> d2) {
MutablePair<DeltaType, KubernetesObject> deletionDelta = isDeletionDup(d1, d2);
// TODO: remove this after the cause of memory leakage is confirmed
// Squashing deltas w/ the same resource version, note that is a temporary fix that eases memory
// intensity.
if (deletionDelta != null) {
return deletionDelta;
}
if (d1.getLeft() != DeltaType.Deleted
&& d2.getLeft() != DeltaType.Deleted
&& StringUtils.equals(
d1.getRight().getMetadata().getResourceVersion(),
d2.getRight().getMetadata().getResourceVersion())) {
return d1;
}
return null;
}
/**
* keep the one with the most information if both are deletions.
*
* @param d1 the most one
* @param d2 the elder one
* @return the most one
*/
private MutablePair<DeltaType, KubernetesObject> isDeletionDup(
MutablePair<DeltaType, KubernetesObject> d1, MutablePair<DeltaType, KubernetesObject> d2) {
if (!d1.getLeft().equals(DeltaType.Deleted) || !d2.getLeft().equals(DeltaType.Deleted)) {
return null;
}
Object obj = d2.getRight();
if (obj instanceof DeletedFinalStateUnknown) {
return d1;
}
return d2;
}
// Note: this should only used in test
Map<String, Deque<MutablePair<DeltaType, KubernetesObject>>> getItems() {
return items;
}
// DeletedFinalStateUnknown is placed into a DeltaFIFO in the case where
// an object was deleted but the watch deletion event was missed. In this
// case we don't know the final "resting" state of the object, so there's
// a chance the included `Obj` is stale.
public static final class DeletedFinalStateUnknown<ApiType extends KubernetesObject>
implements KubernetesObject {
private String key;
private ApiType obj;
DeletedFinalStateUnknown(String key, ApiType obj) {
this.key = key;
this.obj = obj;
}
String getKey() {
return key;
```

```
}
/**
* Gets get obj.
*
* @return the get obj
*/
public ApiType getObj() {
return obj;
}
@Override
public V1ObjectMeta getMetadata() {
return this.obj.getMetadata();
}
@Override
public String getApiVersion() {
return this.obj.getApiVersion();
}
@Override
public String getKind() {
return this.obj.getKind();
}
}
public enum DeltaType {
Added,
Updated,
Deleted,
Sync
}
}
```

io/kubernetes/client/informer/cache/ReflectorRunnable.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.cache;
import io.kubernetes.client.common.KubernetesListObject;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.informer.EventType;
import io.kubernetes.client.informer.ListerWatcher;
```

```
import io.kubernetes.client.informer.exception.WatchExpiredException;
import io.kubernetes.client.openapi.ApiException;
import io.kubernetes.client.openapi.models.V1ListMeta;
import io.kubernetes.client.openapi.models.V1ObjectMeta;
import io.kubernetes.client.util.CallGeneratorParams;
import io.kubernetes.client.util.Strings;
import io.kubernetes.client.util.Watchable;
import java.io.IOException;
import java.net.ConnectException;
import java.net.HttpURLConnection;
import java.time.Duration;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.function.BiConsumer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class ReflectorRunnable<
ApiType extends KubernetesObject, ApiListType extends KubernetesListObject>
implements Runnable {
public static Duration REFLECTOR_WATCH_CLIENTSIDE_TIMEOUT = Duration.ofMinutes(5);
public static Duration REFLECTOR_WATCH_CLIENTSIDE_MAX_TIMEOUT = Duration.ofMinutes(5 *
2);
private static final Logger log = LoggerFactory.getLogger(ReflectorRunnable.class);
private String lastSyncResourceVersion;
private boolean isLastSyncResourceVersionUnavailable;
private Watchable<ApiType> watch;
private ListerWatcher<ApiType, ApiListType> listerWatcher;
private DeltaFIFO store;
private Class<ApiType> apiTypeClass;
private AtomicBoolean isActive = new AtomicBoolean(true);
/* visible for testing */ final BiConsumer<Class<ApiType>, Throwable> exceptionHandler;
public ReflectorRunnable(
Class<ApiType> apiTypeClass,
ListerWatcher<ApiType, ApiListType> listerWatcher,
DeltaFIFO store) {
this(apiTypeClass, listerWatcher, store, null);
}
public ReflectorRunnable(
Class<ApiType> apiTypeClass,
ListerWatcher<ApiType, ApiListType> listerWatcher,
DeltaFIFO store,
BiConsumer<Class<ApiType>, Throwable> exceptionHandler) {
this.listerWatcher = listerWatcher;
this.store = store;
this.apiTypeClass = apiTypeClass;
this.exceptionHandler =
exceptionHandler == null ? ReflectorRunnable::defaultWatchErrorHandler : exceptionHandler;
}
/**
* run first lists all items and get the resource version at the moment of call, and then use the
* resource version to watch.
*/
```

```
public void run() {
log.info("{}#Start listing and watching...", apiTypeClass);
try {
ApiListType list =
listerWatcher.list(
new CallGeneratorParams(Boolean.FALSE, getRelistResourceVersion(), null));
V1ListMeta listMeta = list.getMetadata();
String resourceVersion = listMeta.getResourceVersion();
List<? extends KubernetesObject> items = list.getItems();
if (log.isDebugEnabled()) {
log.debug("{}#Extract resourceVersion {} list meta", apiTypeClass, resourceVersion);
}
this.syncWith(items, resourceVersion);
this.lastSyncResourceVersion = resourceVersion;
this.isLastSyncResourceVersionUnavailable = false;
if (log.isDebugEnabled()) {
log.debug("{}#Start watching with {}...", apiTypeClass, lastSyncResourceVersion);
}
while (true) {
if (!isActive.get()) {
closeWatch();
return;
}
try {
if (log.isDebugEnabled()) {
log.debug(
"{}#Start watch with resource version {}", apiTypeClass, lastSyncResourceVersion);
}
long jitteredWatchTimeoutSeconds =
Double.valueOf(REFLECTOR_WATCH_CLIENTSIDE_TIMEOUT.getSeconds() * (1 + Math.random()))
.longValue();
Watchable<ApiType> newWatch =
listerWatcher.watch(
new CallGeneratorParams(
Boolean.TRUE,
lastSyncResourceVersion,
Long.valueOf(jitteredWatchTimeoutSeconds).intValue()));
synchronized (this) {
if (!isActive.get()) {
newWatch.close();
continue;
}
watch = newWatch;
}
watchHandler(newWatch);
} catch (WatchExpiredException e) {
// Watch calls were failed due to expired resource-version. Returning
// to unwind the list-watch loops so that we can respawn a new round
// of list-watching.
log.info("{}#Watch expired, will re-list-watch soon", this.apiTypeClass);
return;
} catch (Throwable t) {
if (isConnectException(t)) {
```

```
// If this is "connection refused" error, it means that most likely
// apiserver is not responsive. It doesn't make sense to re-list all
// objects because most likely we will be able to restart watch where
// we ended. If that's the case wait and resend watch request.
log.info("{}#Watch get connect exception, retry watch", this.apiTypeClass);
try {
Thread.sleep(1000L);
} catch (InterruptedException e) {
// no-op
}
continue;
}
if ((t instanceof RuntimeException)
&& t.getMessage() != null
&& t.getMessage().contains("IO Exception during hasNext")) {
log.info("{}#Read timeout retry list and watch", this.apiTypeClass);
// IO timeout should be taken as a normal case
return;
}
this.exceptionHandler.accept(apiTypeClass, t);
return;
} finally {
closeWatch();
}
}
} catch (ApiException e) {
if (e.getCode() == HttpURLConnection.HTTP_GONE) {
log.info(
"ResourceVersion {} expired, will retry w/o resourceVersion at the next time",
getRelistResourceVersion());
isLastSyncResourceVersionUnavailable = true;
} else {
this.exceptionHandler.accept(apiTypeClass, e);
}
} catch (Throwable t) {
this.exceptionHandler.accept(apiTypeClass, t);
}
}
public void stop() {
try {
isActive.set(false);
closeWatch();
} catch (Throwable t) {
this.exceptionHandler.accept(apiTypeClass, t);
}
}
private synchronized void closeWatch() throws IOException {
if (watch != null) {
watch.close();
watch = null;
}
}
private void syncWith(List<? extends KubernetesObject> items, String resourceVersion) {
```

```
this.store.replace(
(List<KubernetesObject>) items, resourceVersion); // down-casting is safe here
}
public String getLastSyncResourceVersion() {
return lastSyncResourceVersion;
}
public boolean isLastSyncResourceVersionUnavailable() {
return isLastSyncResourceVersionUnavailable;
}
private String getRelistResourceVersion() {
if (isLastSyncResourceVersionUnavailable) {
// Since this reflector makes paginated list requests, and all paginated list requests skip
// the watch cache
// if the lastSyncResourceVersion is unavailable, we set ResourceVersion="" and list again to
// re-establish reflector
// to the latest available ResourceVersion, using a consistent read from etcd.
return "";
}
if (Strings.isNullOrEmpty(lastSyncResourceVersion)) {
// For performance reasons, initial list performed by reflector uses "0" as resource version
// to allow it to
// be served from the watch cache if it is enabled.
return "0";
}
return lastSyncResourceVersion;
}
private void watchHandler(Watchable<ApiType> watch) {
while (watch.hasNext()) {
io.kubernetes.client.util.Watch.Response<ApiType> item = watch.next();
Optional<EventType> eventType = EventType.findByType(item.type);
if (eventType.isEmpty()) {
log.error("unrecognized event {}", item);
continue;
}
if (eventType.get() == EventType.ERROR) {
if (item.status != null && item.status.getCode() == HttpURLConnection.HTTP_GONE) {
log.info(
"ResourceVersion {} and Watch connection expired: {} , will retry w/o resourceVersion next time",
getRelistResourceVersion(),
item.status.getMessage());
isLastSyncResourceVersionUnavailable = true;
throw new WatchExpiredException();
} else {
String errorMessage =
String.format("got ERROR event and its status: %s", item.status.toString());
log.error(errorMessage);
throw new RuntimeException(errorMessage);
}
}
ApiType obj = item.object;
V1ObjectMeta meta = obj.getMetadata();
String newResourceVersion = meta.getResourceVersion();
switch (eventType.get()) {
```

```
case ADDED:
store.add(obj);
break;
case MODIFIED:
store.update(obj);
break;
case DELETED:
store.delete(obj);
break;
case BOOKMARK:
break;
// A `Bookmark` means watch has synced here, just update the resourceVersion
}
lastSyncResourceVersion = newResourceVersion;
if (log.isDebugEnabled()) {
log.debug("{}#Receiving resourceVersion {}", apiTypeClass, lastSyncResourceVersion);
}
}
}
static <ApiType extends KubernetesObject> void defaultWatchErrorHandler(
Class<ApiType> watchingApiTypeClass, Throwable t) {
log.error("{}#Reflector loop failed unexpectedly", watchingApiTypeClass, t);
}
private boolean isConnectException(Throwable t) {
if (t instanceof ConnectException) {
return true;
}
// ApiException can nest a ConnectException
Throwable cause = t.getCause();
return cause instanceof ConnectException;
}
}
```

io/kubernetes/client/informer/cache/Lister.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.cache;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.util.Strings;
import java.util.List;
```

```
/** Lister interface is used to list cached items from a running informer. */
public class Lister<ApiType extends KubernetesObject> {
private String namespace;
private String indexName;
private Indexer<ApiType> indexer;
public Lister(Indexer<ApiType> indexer) {
this(indexer, null, Caches.NAMESPACE_INDEX);
}
public Lister(Indexer<ApiType> indexer, String namespace) {
this(indexer, namespace, Caches.NAMESPACE_INDEX);
}
public Lister(Indexer<ApiType> indexer, String namespace, String indexName) {
this.indexer = indexer;
this.namespace = namespace;
this.indexName = indexName;
}
public List<ApiType> list() {
if (Strings.isNullOrEmpty(namespace)) {
return indexer.list();
} else {
return indexer.byIndex(this.indexName, namespace);
}
}
public ApiType get(String name) {
String key = name;
if (!Strings.isNullOrEmpty(namespace)) {
key = namespace + "/" + name;
}
return indexer.getByKey(key);
}
public Lister<ApiType> namespace(String namespace) {
return new Lister<ApiType>(this.indexer, namespace, Caches.NAMESPACE_INDEX);
}
}
```

io/kubernetes/client/informer/cache/Cache.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.cache;
import io.kubernetes.client.common.KubernetesObject;
```

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.function.Function;
import org.apache.commons.collections4.CollectionUtils;
import org.apache.commons.collections4.MapUtils;
/**
* Cache is a java port of k/client-go's ThreadSafeStore. It basically saves and indexes all the
* entries.
*/
// TODO(yue9944882): Cache is very similar to a Map, replace/inherit w/ Map interface
public class Cache<ApiType extends KubernetesObject> implements Indexer<ApiType> {
/** keyFunc defines how to map objects into indices */
private Function<ApiType, String> keyFunc;
/** indexers stores index functions by their names */
private Map<String, Function<ApiType, List<String>>> indexers = new HashMap<>();
/** items stores object instances */
private Map<String, ApiType> items = new HashMap<>();
/** indices stores objects' keys by their indices */
private Map<String, Map<String, Set<String>>> indices = new HashMap<>();
public Cache() {
this(
Caches.NAMESPACE_INDEX,
Caches::metaNamespaceIndexFunc,
Caches::deletionHandlingMetaNamespaceKeyFunc);
}
/**
* Constructor.
*
* @param indexName the index name, an unique name representing the index
* @param indexFunc the index func by which we map multiple object to an index for querying
* @param keyFunc the key func by which we map one object to an unique key for storing
*/
public Cache(
String indexName,
Function<ApiType, List<String>> indexFunc,
Function<ApiType, String> keyFunc) {
this.indexers.put(indexName, indexFunc);
this.keyFunc = keyFunc;
this.indices.put(indexName, new HashMap<>());
}
/**
* Add objects.
*
* @param obj the obj
*/
@Override
public void add(ApiType obj) {
String key = keyFunc.apply(obj);
```

```
synchronized (this) {
ApiType oldObj = this.items.get(key);
this.items.put(key, obj);
this.updateIndices(oldObj, obj, key);
}
}
/**
* Update the object.
*
* @param obj the obj
*/
@Override
public void update(ApiType obj) {
String key = keyFunc.apply(obj);
synchronized (this) {
ApiType oldObj = this.items.get(key);
this.items.put(key, obj);
updateIndices(oldObj, obj, key);
}
}
/**
* Delete the object.
*
* @param obj the obj
*/
@Override
public void delete(ApiType obj) {
String key = keyFunc.apply(obj);
synchronized (this) {
boolean exists = this.items.containsKey(key);
if (exists) {
this.deleteFromIndices(this.items.get(key), key);
this.items.remove(key);
}
}
}
/**
* Replace the content in the cache completely.
*
* @param list the list
* @param resourceVersion the resource version
*/
@Override
public synchronized void replace(List<ApiType> list, String resourceVersion) {
Map<String, ApiType> newItems = new HashMap<>();
for (ApiType item : list) {
String key = keyFunc.apply(item);
newItems.put(key, item);
}
this.items = newItems;
// rebuild any index
this.indices = new HashMap<>();
for (Map.Entry<String, ApiType> itemEntry : items.entrySet()) {
```

```
this.updateIndices(null, itemEntry.getValue(), itemEntry.getKey());
}
}
/** Resync. */
@Override
public void resync() {
// Do nothing by default
}
/**
* List keys.
*
* @return the list
*/
@Override
public synchronized List<String> listKeys() {
List<String> keys = new ArrayList<>(this.items.size());
for (Map.Entry<String, ApiType> entry : this.items.entrySet()) {
keys.add(entry.getKey());
}
return keys;
}
/**
* Get object t.
*
* @param obj the obj
* @return the t
*/
@Override
public ApiType get(ApiType obj) {
String key = this.keyFunc.apply(obj);
synchronized (this) {
return this.getByKey(key);
}
}
/**
* List all objects in the cache.
*
* @return the list
*/
@Override
public synchronized List<ApiType> list() {
List<ApiType> itemList = new ArrayList<>(this.items.size());
for (Map.Entry<String, ApiType> entry : this.items.entrySet()) {
itemList.add(entry.getValue());
}
return itemList;
}
/**
* Gets get by key.
*
* @param key the key
* @return the get by key
*/
```

```
@Override
public synchronized ApiType getByKey(String key) {
return this.items.get(key);
}
/**
* Get objects .
*
* @param indexName the index name
* @param obj the obj
* @return the list
*/
@Override
public synchronized List<ApiType> index(String indexName, ApiType obj) {
if (!this.indexers.containsKey(indexName)) {
throw new IllegalArgumentException(String.format("index %s doesn't exist!", indexName));
}
Function<ApiType, List<String>> indexFunc = this.indexers.get(indexName);
List<String> indexKeys = indexFunc.apply((ApiType) obj);
Map<String, Set<String>> index = this.indices.get(indexName);
if (MapUtils.isEmpty(index)) {
return new ArrayList<>();
}
Set<String> returnKeySet = new HashSet<>();
for (String indexKey : indexKeys) {
Set<String> set = index.get(indexKey);
if (CollectionUtils.isEmpty(set)) {
continue;
}
returnKeySet.addAll(set);
}
List<ApiType> items = new ArrayList<>(returnKeySet.size());
for (String absoluteKey : returnKeySet) {
items.add(this.items.get(absoluteKey));
}
return items;
}
/**
* Index keys list.
*
* @param indexName the index name
* @param indexKey the index key
* @return the list
*/
@Override
public synchronized List<String> indexKeys(String indexName, String indexKey) {
if (!this.indexers.containsKey(indexName)) {
throw new IllegalArgumentException(String.format("index %s doesn't exist!", indexName));
}
Map<String, Set<String>> index = this.indices.get(indexName);
Set<String> set = index.get(indexKey);
if (set == null) {
return Collections.EMPTY_LIST;
}
```

```
List<String> keys = new ArrayList<>(set.size());
for (String key : set) {
keys.add(key);
}
return keys;
}
/**
* By index list.
*
* @param indexName the index name
* @param indexKey the index key
* @return the list
*/
@Override
public synchronized List<ApiType> byIndex(String indexName, String indexKey) {
if (!this.indexers.containsKey(indexName)) {
throw new IllegalArgumentException(String.format("index %s doesn't exist!", indexName));
}
Map<String, Set<String>> index = this.indices.get(indexName);
Set<String> set = index.get(indexKey);
if (set == null) {
return Collections.EMPTY_LIST;
}
List<ApiType> items = new ArrayList<>(set.size());
for (String key : set) {
items.add(this.items.get(key));
}
return items;
}
/**
* Return the indexers registered with the cache.
*
* @return registered indexers
*/
@Override
public Map<String, Function<ApiType, List<String>>> getIndexers() {
return indexers;
}
/**
* Add additional indexers to the cache.
*
* @param newIndexers indexers to add
*/
@Override
public void addIndexers(Map<String, Function<ApiType, List<String>>> newIndexers) {
if (!items.isEmpty()) {
throw new IllegalStateException("cannot add indexers to a non-empty cache");
}
Set<String> oldKeys = indexers.keySet();
Set<String> newKeys = newIndexers.keySet();
Set<String> intersection = new HashSet<>(oldKeys);
intersection.retainAll(newKeys);
if (!intersection.isEmpty()) {
```

```
throw new IllegalArgumentException("indexer conflict: " + intersection);
}
for (Map.Entry<String, Function<ApiType, List<String>>> indexEntry : newIndexers.entrySet()) {
addIndexFunc(indexEntry.getKey(), indexEntry.getValue());
}
}
/**
* updateIndices modifies the objects location in the managed indexes, if this is an update, you
* must provide an oldObj.
*
* <p>Note: updateIndices must be called from a function that already has a lock on the cache.
*
* @param oldObj the old obj
* @param newObj the new obj
* @param key the key
*/
public void updateIndices(ApiType oldObj, ApiType newObj, String key) {
// if we got an old object, we need to remove it before we can add
// it again.
if (oldObj != null) {
deleteFromIndices(oldObj, key);
}
for (Map.Entry<String, Function<ApiType, List<String>>> indexEntry : indexers.entrySet()) {
String indexName = indexEntry.getKey();
Function<ApiType, List<String>> indexFunc = indexEntry.getValue();
List<String> indexValues = indexFunc.apply(newObj);
if (CollectionUtils.isEmpty(indexValues)) {
continue;
}
Map<String, Set<String>> index =
this.indices.computeIfAbsent(indexName, k -> new HashMap<>());
for (String indexValue : indexValues) {
Set<String> indexSet = index.computeIfAbsent(indexValue, k -> new HashSet<>());
indexSet.add(key);
}
}
}
/**
* deleteFromIndices removes the object from each of the managed indexes.
*
* <p>Note: It is intended to be called from a function that already has a lock on the cache.
*
* @param oldObj the old obj
* @param key the key
*/
private void deleteFromIndices(ApiType oldObj, String key) {
for (Map.Entry<String, Function<ApiType, List<String>>> indexEntry : this.indexers.entrySet()) {
Function<ApiType, List<String>> indexFunc = indexEntry.getValue();
List<String> indexValues = indexFunc.apply(oldObj);
if (CollectionUtils.isEmpty(indexValues)) {
continue;
}
Map<String, Set<String>> index = this.indices.get(indexEntry.getKey());
```

```
if (index == null) {
continue;
}
for (String indexValue : indexValues) {
Set<String> indexSet = index.get(indexValue);
if (indexSet != null) {
indexSet.remove(key);
if (indexSet.isEmpty()) {
index.remove(indexValue);
}
}
}
}
}
/**
* Add index func.
*
* @param indexName the index name
* @param indexFunc the index func
*/
public void addIndexFunc(String indexName, Function<ApiType, List<String>> indexFunc) {
this.indices.put(indexName, new HashMap<>());
this.indexers.put(indexName, indexFunc);
}
public Function<ApiType, String> getKeyFunc() {
return keyFunc;
}
public void setKeyFunc(Function<ApiType, String> keyFunc) {
this.keyFunc = keyFunc;
}
}
```

io/kubernetes/client/informer/cache/ProcessorListener.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.cache;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.informer.ResourceEventHandler;
```

```
import io.kubernetes.client.informer.exception.BadNotificationException;
import java.time.Duration;
import java.time.OffsetDateTime;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/**
* ProcessorListener implements Runnable interface. It's supposed to run in background and actually
* executes its event handler on notification.
*/
public class ProcessorListener<ApiType extends KubernetesObject> implements Runnable {
private static final Logger log = LoggerFactory.getLogger(ProcessorListener.class);
// resyncPeriod is how frequently the listener wants a full resync from the shared informer.
// This
// value may differ from requestedResyncPeriod if the shared informer adjusts it to align with
// the
// informer's overall resync check period.
private long resyncPeriod;
private OffsetDateTime nextResync;
private BlockingQueue<Notification> queue;
private ResourceEventHandler<ApiType> handler;
public ProcessorListener(ResourceEventHandler<ApiType> handler, long resyncPeriod) {
this.resyncPeriod = resyncPeriod;
this.handler = handler;
this.queue = new LinkedBlockingQueue<>();
determineNextResync(OffsetDateTime.now());
}
@Override
public void run() {
while (true) {
try {
Notification obj = queue.take();
if (obj instanceof UpdateNotification) {
UpdateNotification notification = (UpdateNotification) obj;
try {
this.handler.onUpdate(
(ApiType) notification.getOldObj(), (ApiType) notification.getNewObj());
} catch (Throwable t) {
// Catch all exceptions here so that listeners won't quit unexpectedly
log.error("failed invoking UPDATE event handler: {}", t);
continue;
}
} else if (obj instanceof AddNotification) {
AddNotification notification = (AddNotification) obj;
try {
this.handler.onAdd((ApiType) notification.getNewObj());
} catch (Throwable t) {
// Catch all exceptions here so that listeners won't quit unexpectedly
log.error("failed invoking ADD event handler: {}", t);
continue;
}
} else if (obj instanceof DeleteNotification) {
```

```
Object deletedObj = ((DeleteNotification) obj).getOldObj();
try {
if (deletedObj instanceof DeltaFIFO.DeletedFinalStateUnknown) {
this.handler.onDelete(
((DeltaFIFO.DeletedFinalStateUnknown<ApiType>) deletedObj).getObj(), true);
} else {
this.handler.onDelete((ApiType) deletedObj, false);
}
} catch (Throwable t) {
// Catch all exceptions here so that listeners won't quit unexpectedly
log.error("failed invoking DELETE event handler: {}", t);
continue;
}
} else {
throw new BadNotificationException("unrecognized notification");
}
} catch (InterruptedException e) {
log.error("processor interrupted: {}", e);
return;
}
}
}
public void add(Notification<ApiType> obj) {
if (obj == null) {
return;
}
this.queue.add(obj);
}
public void determineNextResync(OffsetDateTime now) {
this.nextResync = now.plus(Duration.ofMillis(this.resyncPeriod));
}
public boolean shouldResync(OffsetDateTime now) {
return this.resyncPeriod != 0 && (now.isAfter(this.nextResync) || now.equals(this.nextResync));
}
public static class Notification<ApiType> {}
public static final class UpdateNotification<ApiType> extends Notification<ApiType> {
private ApiType oldObj;
private ApiType newObj;
public UpdateNotification(ApiType oldObj, ApiType newObj) {
this.oldObj = oldObj;
this.newObj = newObj;
}
ApiType getOldObj() {
return oldObj;
}
ApiType getNewObj() {
return newObj;
}
}
public static final class AddNotification<ApiType> extends Notification<ApiType> {
private ApiType newObj;
public AddNotification(ApiType newObj) {
this.newObj = newObj;
```

```
}
ApiType getNewObj() {
return newObj;
}
}
public static final class DeleteNotification<ApiType> extends Notification<ApiType> {
private ApiType oldObj;
public DeleteNotification(ApiType oldObj) {
this.oldObj = oldObj;
}
ApiType getOldObj() {
return oldObj;
}
}
}
```

io/kubernetes/client/informer/cache/Caches.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.cache;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.openapi.models.V1ObjectMeta;
import io.kubernetes.client.util.Strings;
import java.util.Collections;
import java.util.List;
/** A set of helper utilities for constructing a cache. */
public class Caches {
/** NAMESPACE_INDEX is the default index function for caching objects */
public static final String NAMESPACE_INDEX = "namespace";
/**
* deletionHandlingMetaNamespaceKeyFunc checks for DeletedFinalStateUnknown objects before
calling
* metaNamespaceKeyFunc.
*
* @param <ApiType> the type parameter
* @param object specific object
* @return the key
*/
public static <ApiType extends KubernetesObject> String deletionHandlingMetaNamespaceKeyFunc(
ApiType object) {
```

```
if (object instanceof DeltaFIFO.DeletedFinalStateUnknown) {
DeltaFIFO.DeletedFinalStateUnknown<ApiType> deleteObj =
(DeltaFIFO.DeletedFinalStateUnknown<ApiType>) object;
return deleteObj.getKey();
}
return metaNamespaceKeyFunc(object);
}
/**
* MetaNamespaceKeyFunc is a convenient default KeyFunc which knows how to make keys for API
* objects which implement HasMetadata Interface. The key uses the format <namespace>/<name>
* unless <namespace> is empty, then it's just <name>.
*
* @param obj specific object
* @return the key
*/
public static String metaNamespaceKeyFunc(KubernetesObject obj) {
V1ObjectMeta metadata = obj.getMetadata();
if (!Strings.isNullOrEmpty(metadata.getNamespace())) {
return metadata.getNamespace() + "/" + metadata.getName();
}
return metadata.getName();
}
/**
* metaNamespaceIndexFunc is a default index function that indexes based on an object's namespace.
*
* @param obj specific object
* @return the indexed value
*/
public static List<String> metaNamespaceIndexFunc(KubernetesObject obj) {
V1ObjectMeta metadata = obj.getMetadata();
if (metadata == null) {
return Collections.emptyList();
}
return Collections.singletonList(metadata.getNamespace());
}
}
```

io/kubernetes/client/informer/cache/Indexer.java


```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
```

```
package io.kubernetes.client.informer.cache;
import java.util.List;
import java.util.Map;
import java.util.function.Function;
/** Indexer extends Store interface and adds index/de-index methods. */
public interface Indexer<ApiType> extends Store<ApiType> {
/**
* Retrieve list of objects that match on the named indexing function.
*
* @param indexName specific indexing function
* @return matched objects
*/
List<ApiType> index(String indexName, ApiType obj);
/**
* IndexKeys returns the set of keys that match on the named indexing function.
*
* @param indexName specific indexing function
* @param indexKey specific index key
* @return matched keys
*/
List<String> indexKeys(String indexName, String indexKey);
/**
* ByIndex lists object that match on the named indexing function with the exact key.
*
* @param indexName specific indexing function
* @param indexKey specific index key
* @return matched objects
*/
List<ApiType> byIndex(String indexName, String indexKey);
/**
* Return the indexers registered with the store.
*
* @return registered indexers
*/
Map<String, Function<ApiType, List<String>>> getIndexers();
/**
* Add additional indexers to the store.
*
* @param indexers indexers to add
*/
void addIndexers(Map<String, Function<ApiType, List<String>>> indexers);
}
```

io/kubernetes/client/informer/cache/SharedProcessor.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
```

```
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.cache;
import io.kubernetes.client.common.KubernetesObject;
import java.time.Duration;
import java.time.OffsetDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;
import org.apache.commons.collections4.CollectionUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/*
* SharedProcessor class manages all the registered ProcessorListener and distributes notifications.
*/
public class SharedProcessor<ApiType extends KubernetesObject> {
private static final Logger log = LoggerFactory.getLogger(SharedProcessor.class);
private ReadWriteLock lock = new ReentrantReadWriteLock();
private List<ProcessorListener<ApiType>> listeners;
private List<ProcessorListener<ApiType>> syncingListeners;
private ExecutorService executorService;
private final Duration timeout;
public SharedProcessor() {
this(Executors.newCachedThreadPool());
}
public SharedProcessor(ExecutorService threadPool) {
this(threadPool, Duration.ofMinutes(1));
}
public SharedProcessor(ExecutorService threadPool, Duration timeout) {
this.listeners = new ArrayList<>();
this.syncingListeners = new ArrayList<>();
this.executorService = threadPool;
this.timeout = timeout;
}
/**
* addAndStartListener first adds the specific processorListener then starts the listener with
* executor.
*
* @param processorListener specific processor listener
*/
public void addAndStartListener(final ProcessorListener<ApiType> processorListener) {
lock.writeLock().lock();
try {
```

```
addListenerLocked(processorListener);
executorService.execute(processorListener);
} finally {
lock.writeLock().unlock();
}
}
/**
* addListener adds the specific processorListener, but not start it.
*
* @param processorListener specific processor listener
*/
public void addListener(final ProcessorListener<ApiType> processorListener) {
lock.writeLock().lock();
try {
addListenerLocked(processorListener);
} finally {
lock.writeLock().unlock();
}
}
private void addListenerLocked(final ProcessorListener<ApiType> processorListener) {
this.listeners.add(processorListener);
this.syncingListeners.add(processorListener);
}
/** starts the processor listeners. */
public void run() {
lock.readLock().lock();
try {
if (CollectionUtils.isEmpty(listeners)) {
return;
}
for (ProcessorListener listener : listeners) {
executorService.execute(listener);
}
} finally {
lock.readLock().unlock();
}
}
/**
* distribute the object among listeners.
*
* @param obj specific obj
* @param isSync is sync or not
*/
public void distribute(ProcessorListener.Notification<ApiType> obj, boolean isSync) {
lock.readLock().lock();
try {
if (isSync) {
for (ProcessorListener<ApiType> listener : syncingListeners) {
listener.add(obj);
}
} else {
for (ProcessorListener<ApiType> listener : listeners) {
listener.add(obj);
```

```
}
}
} finally {
lock.readLock().unlock();
}
}
public boolean shouldResync() {
lock.writeLock().lock();
boolean resyncNeeded = false;
try {
this.syncingListeners = new ArrayList<>(this.listeners.size());
OffsetDateTime now = OffsetDateTime.now();
for (ProcessorListener listener : this.listeners) {
if (listener.shouldResync(now)) {
resyncNeeded = true;
this.syncingListeners.add(listener);
listener.determineNextResync(now);
}
}
} finally {
lock.writeLock().unlock();
}
return resyncNeeded;
}
public void stop() {
lock.writeLock().lock();
try {
listeners = null;
} finally {
lock.writeLock().unlock();
}
// Interrupts running listeners by signalling InterruptedException
executorService.shutdownNow();
try {
// Hold until all the listeners exits
if (!executorService.awaitTermination(timeout.toMillis(), TimeUnit.MILLISECONDS)) {
log.warn(
"SharedProcessors wasn't gracefully terminated, there can be listener thread leakage");
}
} catch (InterruptedException e) {
log.error("Graceful shutdown process of SharedProcessors was interrupted");
}
}
}
```

io/kubernetes/client/informer/cache/Controller.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
```

You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
```
package io.kubernetes.client.informer.cache;
import io.kubernetes.client.common.KubernetesListObject;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.informer.ListerWatcher;
import io.kubernetes.client.informer.ResyncRunnable;
import io.kubernetes.client.util.Threads;
import java.util.Deque;
import java.util.concurrent.Executors;
import java.util.concurrent.RejectedExecutionException;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.ScheduledFuture;
import java.util.concurrent.TimeUnit;
import java.util.function.BiConsumer;
import java.util.function.Consumer;
import java.util.function.Supplier;
import org.apache.commons.lang3.tuple.MutablePair;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/**
* Controller is a java port of k/client-go's informer#Controller. It plumbs reflector and the queue
* implementation and it runs re-sync function periodically.
*/
public class Controller<
ApiType extends KubernetesObject, ApiListType extends KubernetesListObject> {
private static final Logger log = LoggerFactory.getLogger(Controller.class);
private static final long DEFAULT_PERIOD = 1000L;
/** resync fifo internal in millis */
private long fullResyncPeriod;
/** Queue stores deltas produced by reflector */
private DeltaFIFO queue;
private ListerWatcher<ApiType, ApiListType> listerWatcher;
private ReflectorRunnable<ApiType, ApiListType> reflector;
private Supplier<Boolean> resyncFunc;
/** how we actually process items from the queue */
private Consumer<Deque<MutablePair<DeltaFIFO.DeltaType, KubernetesObject>>> processFunc;
private ScheduledExecutorService reflectExecutor;
private ScheduledExecutorService resyncExecutor;
private ScheduledFuture resyncFuture;
private Class<ApiType> apiTypeClass;
private ScheduledFuture reflectorFuture;
/* visible for testing */ BiConsumer<Class<ApiType>, Throwable> exceptionHandler;
public Controller(
Class<ApiType> apiTypeClass,
DeltaFIFO queue,
ListerWatcher<ApiType, ApiListType> listerWatcher,
```

```
Consumer<Deque<MutablePair<DeltaFIFO.DeltaType, KubernetesObject>>> processFunc,
Supplier<Boolean> resyncFunc,
long fullResyncPeriod) {
this(apiTypeClass, queue, listerWatcher, processFunc, resyncFunc, fullResyncPeriod, null);
}
public Controller(
Class<ApiType> apiTypeClass,
DeltaFIFO queue,
ListerWatcher<ApiType, ApiListType> listerWatcher,
Consumer<Deque<MutablePair<DeltaFIFO.DeltaType, KubernetesObject>>> processFunc,
Supplier<Boolean> resyncFunc,
long fullResyncPeriod,
BiConsumer<Class<ApiType>, Throwable> exceptionHandler) {
this.queue = queue;
this.listerWatcher = listerWatcher;
this.apiTypeClass = apiTypeClass;
this.processFunc = processFunc;
this.resyncFunc = resyncFunc;
this.fullResyncPeriod = fullResyncPeriod;
this.exceptionHandler = exceptionHandler;
// starts one daemon thread for reflector
this.reflectExecutor =
Executors.newSingleThreadScheduledExecutor(
Threads.threadFactory("controller-reflector-" + apiTypeClass.getName() + "-%d"));
// starts one daemon thread for resync
this.resyncExecutor =
Executors.newSingleThreadScheduledExecutor(
Threads.threadFactory("controller-resync-" + apiTypeClass.getName() + "-%d"));
}
public Controller(
Class<ApiType> apiTypeClass,
DeltaFIFO queue,
ListerWatcher<ApiType, ApiListType> listerWatcher,
Consumer<Deque<MutablePair<DeltaFIFO.DeltaType, KubernetesObject>>> popProcessFunc) {
this(apiTypeClass, queue, listerWatcher, popProcessFunc, null, 0);
}
public void run() {
log.info("informer#Controller: ready to run resync & reflector runnable");
// start the resync runnable
if (fullResyncPeriod > 0) {
ResyncRunnable resyncRunnable = new ResyncRunnable(queue, resyncFunc);
resyncFuture =
resyncExecutor.scheduleAtFixedRate(
resyncRunnable::run, fullResyncPeriod, fullResyncPeriod, TimeUnit.MILLISECONDS);
} else {
log.info("informer#Controller: resync skipped due to 0 full resync period");
}
synchronized (this) {
// TODO(yue9944882): proper naming for reflector
reflector = newReflector();
try {
reflectorFuture =
reflectExecutor.scheduleWithFixedDelay(
```

```
reflector::run, 0L, DEFAULT_PERIOD, TimeUnit.MILLISECONDS);
} catch (RejectedExecutionException e) {
// submitting reflector list-watching job can fail due to concurrent invocation of
// `shutdown`. handling exception with a warning then return.
log.warn("reflector list-watching job exiting because the thread-pool is shutting down");
return;
}
}
// start the process loop
this.processLoop();
}
/* visible for testing */ ReflectorRunnable<ApiType, ApiListType> newReflector() {
return new ReflectorRunnable<>(apiTypeClass, listerWatcher, queue, exceptionHandler);
}
/** stops the resync thread pool firstly, then stop the reflector */
public void stop() {
synchronized (this) {
if (reflectorFuture != null) {
reflector.stop();
reflectorFuture.cancel(true);
}
}
reflectExecutor.shutdownNow();
resyncExecutor.shutdownNow();
}
/** returns true if the queue has been resycned */
public boolean hasSynced() {
return this.queue.hasSynced();
}
/** returns the latest resource version watched by controller */
public String lastSyncResourceVersion() {
if (reflector == null) {
return "";
}
return reflector.getLastSyncResourceVersion();
}
/** processLoop drains the work queue. */
private void processLoop() {
while (true) {
try {
this.queue.pop(this.processFunc);
} catch (InterruptedException t) {
log.error("DefaultController#processLoop get interrupted {}", t.getMessage(), t);
return;
} catch (Throwable t) {
log.error("DefaultController#processLoop recovered from crashing {}", t.getMessage(), t);
}
}
}
}
```

io/kubernetes/client/informer/cache/Store.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.cache;
import java.util.List;
public interface Store<ApiType> {
/**
* add inserts an item into the store.
*
* @param obj specific obj
*/
void add(ApiType obj);
/**
* update sets an item in the store to its updated state.
*
* @param obj specific obj
*/
void update(ApiType obj);
/**
* delete removes an item from the store.
*
* @param obj specific obj
*/
void delete(ApiType obj);
/**
* Replace will delete the contents of 'c', using instead the given list.
*
* @param list list of objects
* @param resourceVersion specific resource version
*/
void replace(List<ApiType> list, String resourceVersion);
/** resync will send a resync event for each item. */
void resync();
/**
* listKeys returns a list of all the keys of the object currently in the store.
*
* @return list of all keys
*/
List<String> listKeys();
/**
* get returns the requested item.
*
* @param obj specific obj
```

```
* @return the requested item if exist
*/
Object get(ApiType obj);
/**
* getByKey returns the request item with specific key.
*
* @param key specific key
* @return the request item
*/
ApiType getByKey(String key);
/**
* list returns a list of all the items.
*
* @return list of all the items
*/
List<ApiType> list();
}
```

io/kubernetes/client/informer/EventType.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer;
import java.util.Arrays;
import java.util.Map;
import java.util.Optional;
import java.util.function.Function;
import java.util.stream.Collectors;
public enum EventType {
ADDED,
MODIFIED,
DELETED,
BOOKMARK,
ERROR;
private static final Map<String, EventType> TYPES =
Arrays.stream(EventType.values()).collect(Collectors.toMap(Enum::name, Function.identity()));
/**
* returns the corresponding EventType by type.
*
* @param type specific code
* @return corresponding EventType
```

```
* @deprecated will be removed in a future release. use : findByType
*/
@Deprecated
public static EventType getByType(String type) {
if (type != null && type.length() > 0) {
for (EventType eventType : EventType.values()) {
if (eventType.name().equalsIgnoreCase(type)) {
return eventType;
}
}
}
return null;
}
/**
* returns the corresponding EventType by type, wrapped in an {@link Optional}
*
* @param type specific code
* @return an Optional describing the EventType
*/
public static Optional<EventType> findByType(String type) {
return Optional.ofNullable(TYPES.get(String.valueOf(type).toUpperCase()));
}
}
```

io/kubernetes/client/informer/ResyncRunnable.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer;
import io.kubernetes.client.informer.cache.DeltaFIFO;
import java.util.function.Supplier;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
/**
* ResyncRunnable class implements Runnable interface. It calls the resync function of {@link
* DeltaFIFO}
*/
public class ResyncRunnable implements Runnable {
private static final Logger log = LoggerFactory.getLogger(ResyncRunnable.class);
private final DeltaFIFO store;
private final Supplier<Boolean> shouldResyncFunc;
```

```
public ResyncRunnable(DeltaFIFO store, Supplier<Boolean> shouldResyncFunc) {
this.store = store;
this.shouldResyncFunc = shouldResyncFunc;
}
public void run() {
log.debug("ResyncRunnable#resync ticker tick");
if (shouldResyncFunc == null || Boolean.TRUE.equals(shouldResyncFunc.get())) {
log.debug("ResyncRunnable#force resync");
this.store.resync();
}
}
}
```

io/kubernetes/client/informer/SharedInformer.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer;
import io.kubernetes.client.common.KubernetesObject;
/*
* SharedInformer defines basic methods of a informer.
*/
public interface SharedInformer<ApiType extends KubernetesObject> {
/**
* Add event handler.
*
* @param handler the handler
*/
void addEventHandler(ResourceEventHandler<ApiType> handler);
/**
* addEventHandlerWithResyncPeriod adds an event handler to the shared informer using the
* specified resync period. Events to a single handler are delivered sequentially, but there is no
* coordination between different handlers.
*
* @param handler the event handler
* @param resyncPeriod the specific resync period
*/
void addEventHandlerWithResyncPeriod(ResourceEventHandler<ApiType> handler, long
resyncPeriod);
/** run starts the shared informer, which will be stopped until stop() is called. */
void run();
```

```
/** stop stops the shared informer. */
void stop();
/** hasSynced returns true if the shared informer's store has synced. */
boolean hasSynced();
/**
* Last sync resource version string.
*
* @return the string
*/
// LastSyncResourceVersion is the resource version observed when last synced with the underlying
// store. The value returned is not synchronized with access to the underlying store and is not
// thread-safe.
String lastSyncResourceVersion();
/**
* The TransformFunc is called for each object which is about to be stored. This function is
* intended for you to take the opportunity to remove, transform, or normalize fields. One use
* case is to strip unused metadata fields out of objects to save on RAM cost.
*
* <p>Must be set before starting the informer.
*
* <p>Note: Since the object given to the handler may be already shared with other goroutines, it
* is advisable to copy the object being transform before mutating it at all and returning the
* copy to prevent data races.
*
* @param transformFunc the transform function
*/
void setTransform(TransformFunc transformFunc);
}
```

io/kubernetes/client/informer/exception/BadNotificationException.java

```
/*
Copyright 2020 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.exception;
public class BadNotificationException extends RuntimeException {
public BadNotificationException(String message) {
super(message);
}
}
```

io/kubernetes/client/informer/exception/WatchExpiredException.java

```
/*
Copyright 2021 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.exception;
/** A ERROR watch item was returned due to stale resource version. */
public class WatchExpiredException extends RuntimeException {}
```

io/kubernetes/client/informer/exception/ObjectTransformException.java

```
/*
Copyright 2022 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/
package io.kubernetes.client.informer.exception;
public class ObjectTransformException extends RuntimeException {
public ObjectTransformException(String message) {
super(message);
}
}
```

io/kubernetes/client/informer/TransformFunc.java

```
/*
Copyright 2022 The Kubernetes Authors.
Licensed under the Apache License, Version 2.0 (the "License");
```

you may not use this file except in compliance with the License.
You may obtain a copy of the License at
http://www.apache.org/licenses/LICENSE-2.0
Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
```
*/
package io.kubernetes.client.informer;
import io.kubernetes.client.common.KubernetesObject;
import io.kubernetes.client.informer.exception.ObjectTransformException;
/*
* TransformFunc allows for transforming an object before it will be processed
* and put into the controller cache and before the corresponding handlers will
* be called on it.
* TransformFunc (similarly to ResourceEventHandler functions) should be able
* to correctly handle the tombstone of type DeletedFinalStateUnknown
*
* The most common usage pattern is to clean-up some parts of the object to
* reduce component memory usage if a given component doesn't care about them.
* given controller doesn't care for them
*/
public interface TransformFunc {
/**
* @param the original object to be transformed
* @return the transformed object
*/
KubernetesObject transform(KubernetesObject object) throws ObjectTransformException;
}
```